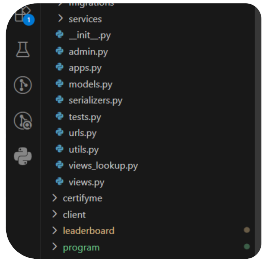
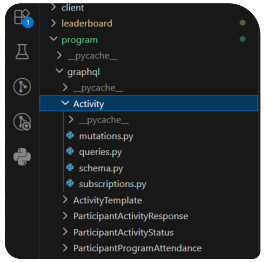




this is my backend structure attachments is my functionality in rest of the project graphql is used but in my attachments project i used rest api now i want to connect my attachments app with main leaderboar project and program app i will share all code below then tell me step by step what to do

```
from django.contrib import admin
from django import forms

from .models import SecureAttachment
from .services.sharepoint import (
    upload_file_to_sharepoint,
    create_view_only_link,
)


# ----------------------------
# Admin Form
# ----------------------------
class SecureAttachmentAdminForm(forms.ModelForm):
    # 🔑 Upload field (ONLY for creation)
    upload_file = forms.FileField(required=True)
```

```
    class Meta:
        model = SecureAttachment
        fields = (
            "upload_file",
            "file_name",
            "description",
            "clients",
            "programs",
            "sessions",
        )


# ----------------------------
# Admin Configuration
# ----------------------------
@admin.register(SecureAttachment)
class SecureAttachmentAdmin(admin.ModelAdmin):
    form = SecureAttachmentAdminForm

    list_display = (
        "file_name",
        "uploaded_by",
        "created_at",
    )

    readonly_fields = (
        "sharepoint_url",
        "sharepoint_item_id",
```

```
    "created_at",
)

filter_horizontal = (
    "clients",
    "programs",
    "sessions",
)

# ----------------------------
# Hide upload_file on edit
# ----------------------------
def get_form(self, request, obj=None, **kwargs):
    form = super().get_form(request, obj, **kwargs)
    if obj:  # Editing existing attachment
        form.base_fields.pop("upload_file", None)
    return form

# ----------------------------
# Save logic
# ----------------------------
def save_model(self, request, obj, form, change):
    """
    Uploads file to SharePoint ONLY on create.
    On edit, only metadata (clients/programs/sessions) is updated.
    """

    # If editing, DO NOT re-upload file
    if change:
```

```
        super().save_model(request, obj, form, change)
        return

    # ----------------------------
    # Create flow (new attachment)
    # ----------------------------
    uploaded_file = form.cleaned_data["upload_file"]

    uploaded = upload_file_to_sharepoint(
        file_stream=uploaded_file.file,
        filename=uploaded_file.name,
        content_type=uploaded_file.content_type,
        file_size=uploaded_file.size,
    )

    item_id = uploaded["id"]
    secure_link = create_view_only_link(item_id)

    obj.file_name = uploaded_file.name
    obj.sharepoint_item_id = item_id
    obj.sharepoint_url = secure_link
    obj.uploaded_by = request.user

    super().save_model(request, obj, form, change)
```

attachments/admin.py

```
from django.apps import AppConfig


class AttachmentsConfig(AppConfig):
```

```
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'attachments'
```

attachments/apps.py

```
from django.db import models
from django.conf import settings
from django.utils import timezone


class SecureAttachment(models.Model):

    ASSET_TYPE_CHOICES = (
        ("file", "File"),
        ("url", "URL"),
    )

    asset_type = models.CharField(
        max_length=50,
        choices=ASSET_TYPE_CHOICES,
        default="file"
    )

    file_name = models.CharField(max_length=255)
    description = models.TextField(blank=True)

    # SharePoint fields (only for file)
    sharepoint_item_id = models.CharField(
        max_length=255,
        unique=True,
        null=True,
```

```
        blank=True,
    )
    sharepoint_url = models.URLField(null=True, blank=True)

    # URL-only field
    external_url = models.URLField(max_length=2000, null=True,
blank=True)

    uploaded_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="uploaded_attachments"
    )

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    clients = models.ManyToManyField(
        "client.Client",
        blank=True,
        related_name="attachments"
    )

    programs = models.ManyToManyField(
        "program.Program",
        blank=True,
        related_name="attachments"
```

```
    )

    sessions = models.ManyToManyField(
        "program.Session",
        blank=True,
        related_name="attachments"
    )

    is_active = models.BooleanField(default=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return self.file_name

class SecureAttachmentAssignment(models.Model):
    attachment = models.ForeignKey(
        "attachments.SecureAttachment",
        on_delete=models.CASCADE,
        related_name="assignments"
    )
    client = models.ForeignKey(
        "client.Client",
        on_delete=models.CASCADE,
        related_name="secure_attachment_assignments"
    )
    program = models.ForeignKey(
        "program.Program",
```

```
        on_delete=models.CASCADE,
        related_name="secure_attachment_assignments"
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ("attachment", "client", "program")
        ordering = ["-created_at"]

    def __str__(self):
        return f"{self.attachment.file_name} -> {self.client.name} /
{self.program.name}"
attachments/models.py
from rest_framework import serializers

from .models import SecureAttachment, SecureAttachmentAssignment
from client.models import Client
from program.models import Program


#
==========================================================
=======
# BASIC LOOKUP SERIALIZERS
#
==========================================================
=======
class ClientSerializer(serializers.ModelSerializer):
    class Meta:
```

```
        model = Client
        fields = ["id", "name"]


class ProgramSerializer(serializers.ModelSerializer):
    class Meta:
        model = Program
        fields = ["id", "name", "client"]


#
==============================================
=======
# UPLOAD SERIALIZER
#
==============================================
=======
class SecureAttachmentUploadSerializer(serializers.Serializer):
    name = serializers.CharField(required=False, allow_blank=True)
    description = serializers.CharField(required=False, allow_blank=True)

    external_url = serializers.URLField(
        required=False,
        allow_blank=True,
        max_length=2000
    )

    # Keep temporarily for upload compatibility
    clients = serializers.PrimaryKeyRelatedField(
```

```
        queryset=Client.objects.all(),
        many=True,
        required=False
    )

    programs = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        many=True,
        required=False
    )

    def validate(self, data):
        file = self.context["request"].FILES.get("file")
        external_url = data.get("external_url")
        name = data.get("name")

        if not file and not external_url:
            raise serializers.ValidationError("Provide file or external_url.")

        # URL-only upload requires name
        if external_url and not file and not name:
            raise serializers.ValidationError("Name is required when
uploading external_url.")

        return data


#
==============================================
```

```
=======
# ASSIGNMENT SERIALIZERS
#
==============================================
=======
class
SecureAttachmentAssignmentSerializer(serializers.ModelSerializer):
    client = ClientSerializer(read_only=True)
    program = ProgramSerializer(read_only=True)

    class Meta:
        model = SecureAttachmentAssignment
        fields = ["id", "client", "program", "created_at"]


class SecureAttachmentAssignActionSerializer(serializers.Serializer):
    client_id = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        source="client"
    )
    program_id = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        source="program"
    )

    def validate(self, data):
        client = data["client"]
        program = data["program"]
```

```
        # Safety check: selected program must belong to selected client
        if getattr(program, "client_id", None) != client.id:
            raise serializers.ValidationError(
                {"program_id": "Selected program does not belong to
selected client."}
            )

        return data



#
==========================================================
=======
# LIST SERIALIZER
#
==========================================================
=======
class SecureAttachmentListSerializer(serializers.ModelSerializer):
    # Legacy support (temporary)
    clients = ClientSerializer(many=True, read_only=True)
    programs = ProgramSerializer(many=True, read_only=True)

    # New exact pair mapping
    assignments = SecureAttachmentAssignmentSerializer(many=True,
read_only=True)

    class Meta:
        model = SecureAttachment
        fields = (
```

```
            "id",
            "file_name",
            "description",
            "asset_type",
            "sharepoint_url",
            "external_url",
            "clients",
            "programs",
            "assignments",
            "created_at",
            "updated_at",
        )


#
==============================================
=======
# LEGACY PATCH SERIALIZER (temporary)
#
==============================================
=======
class SecureAttachmentUpdateSerializer(serializers.Serializer):
    clients = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        many=True,
        required=False
    )
    programs = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
```

```
        many=True,
        required=False
    )

class SecureAttachmentReplaceSerializer(serializers.Serializer):
    external_url = serializers.URLField(
        required=False,
        allow_blank=True,
        max_length=2000
    )

    def validate(self, data):
        request = self.context.get("request")
        file = request.FILES.get("file") if request else None
        external_url = (data.get("external_url") or "").strip()

        if not file and not external_url:
            raise serializers.ValidationError(
                "Provide a new file or external_url."
            )

        if file and external_url:
            raise serializers.ValidationError(
                "Provide either a new file or external_url, not both."
            )

        return data
attachments/serializers.py
from django.urls import path
```

```
from .views import (
    SecureAttachmentUploadAPIView,
    SecureAttachmentReplaceAPIView,
    SecureAttachmentListAPIView,
    SecureAttachmentDetailAPIView,
    SecureAttachmentAssignAPIView,
    SecureAttachmentUnassignAPIView,
    ProgramsByClientAPIView,
)
from .views_lookup import (
    ClientListAPIView,
    ProgramListAPIView,
)
from graphene_django.views import GraphQLView
from django.views.decorators.csrf import csrf_exempt
from .graphql.schema import schema
from attachments.views import csrf_token_view

urlpatterns = [
    path("upload/", SecureAttachmentUploadAPIView.as_view(),
name="secure-attachment-upload"),
    path("<int:pk>/replace/",
SecureAttachmentReplaceAPIView.as_view(), name="attachment-
replace"),
    path("", SecureAttachmentListAPIView.as_view(), name="secure-
attachment-list"),
    path("clients/", ClientListAPIView.as_view()),
    path("programs/", ProgramListAPIView.as_view()),
    path("programs-by-client/", ProgramsByClientAPIView.as_view()),
```

```
    path("<int:pk>/assign/", SecureAttachmentAssignAPIView.as_view(),
name="attachment-assign"),
    path("<int:pk>/unassign/",
SecureAttachmentUnassignAPIView.as_view(), name="attachment-
unassign"),
    path("<int:pk>/", SecureAttachmentDetailAPIView.as_view(),
name="attachment-detail"),
    path("graphql/", csrf_exempt(GraphQLView.as_view(schema=schema,
graphiql=True))),
    path("csrf/", csrf_token_view),
]
```

attachments/urls.py

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status, permissions
from django.db import transaction, IntegrityError
from django.shortcuts import get_object_or_404
from rest_framework.parsers import MultiPartParser, FormParser

from .models import SecureAttachment, SecureAttachmentAssignment
from .serializers import (
    SecureAttachmentUploadSerializer,
    SecureAttachmentListSerializer,
    SecureAttachmentUpdateSerializer,
    SecureAttachmentAssignActionSerializer,
    SecureAttachmentReplaceSerializer,
    ProgramSerializer,
)
from .services.sharepoint import (
```

```
    upload_file_to_sharepoint,
    create_view_only_link,
    delete_file_from_sharepoint,
)
from program.models import Program
from django.http import JsonResponse
from django.middleware.csrf import get_token
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt


def csrf_token_view(request):
    return JsonResponse({
        "csrfToken": get_token(request)
    })


def get_attachment_queryset():
    return
SecureAttachment.objects.filter(is_active=True).prefetch_related(
        "clients",
        "programs",
        "assignments__client",
        "assignments__program",
    )


@method_decorator(csrf_exempt, name='dispatch')
class SecureAttachmentUploadAPIView(APIView):
```

```
    authentication_classes = []
    permission_classes = [permissions.AllowAny]
    parser_classes = [MultiPartParser, FormParser]

    def initial(self, request, *args, **kwargs):
        request._dont_enforce_csrf_checks = True
        super().initial(request, *args, **kwargs)

    def post(self, request):
        try:
            data = request.data.copy()
            uploaded_file = request.FILES.get("file")

            serializer = SecureAttachmentUploadSerializer(
                data=data,
                context={"request": request},
                partial=True
            )

            if not serializer.is_valid():
                return Response(serializer.errors, status=400)

            validated = serializer.validated_data

            with transaction.atomic():
                # ==============================
                # FILE UPLOAD (SharePoint)
                # ==============================
                if uploaded_file:
```

```
                existing_file = SecureAttachment.objects.filter(
                    asset_type="file",
                    file_name=uploaded_file.name,
                    is_active=True
                ).first()

                if existing_file:
                    return Response(
                        {"error": "The file you are trying to upload already
exists."},
                        status=status.HTTP_409_CONFLICT
                    )

                upload_result = upload_file_to_sharepoint(
                    file_stream=uploaded_file.file,
                    filename=uploaded_file.name,
                    content_type=uploaded_file.content_type,
                    file_size=uploaded_file.size,
                )

                item_id = upload_result["id"]

                existing_item = SecureAttachment.objects.filter(
                    sharepoint_item_id=item_id
                ).first()

                if existing_item:
                    return Response(
                        {"error": "The file you are trying to upload already
```

```
exists."},
                status=status.HTTP_409_CONFLICT
            )

        view_url = create_view_only_link(item_id)

        attachment = SecureAttachment.objects.create(
            asset_type="file",
            file_name=uploaded_file.name,
            description=validated.get("description", ""),
            sharepoint_item_id=item_id,
            sharepoint_url=view_url,
            uploaded_by=request.user if
request.user.is_authenticated else None,
        )

    # ==============================
    # URL ONLY
    # ==============================
    else:
        attachment = SecureAttachment.objects.create(
            asset_type="url",
            file_name=validated["name"],
            description=validated.get("description", ""),
            external_url=validated["external_url"],
            uploaded_by=request.user if
request.user.is_authenticated else None,
        )
```

```
            # TEMPORARY legacy sync
            if "clients" in validated:
                attachment.clients.set(validated["clients"])

            if "programs" in validated:
                attachment.programs.set(validated["programs"])

            # NEW exact pair mapping:
            # only create if exactly one client + one program are supplied
            upload_clients = validated.get("clients", [])
            upload_programs = validated.get("programs", [])

            if len(upload_clients) == 1 and len(upload_programs) == 1:
                selected_client = upload_clients[0]
                selected_program = upload_programs[0]

                if getattr(selected_program, "client_id", None) ==
selected_client.id:
                    SecureAttachmentAssignment.objects.get_or_create(
                        attachment=attachment,
                        client=selected_client,
                        program=selected_program,
                    )

        # Re-fetch with prefetch so serializer gets fresh assignments
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            SecureAttachmentListSerializer(attachment).data,
```

```
            status=201,
        )

    except IntegrityError:
        return Response(
            {"error": "The file you are trying to upload already exists."},
            status=status.HTTP_409_CONFLICT
        )

    except Exception as e:
        print("ERROR in upload API:", repr(e))
        return Response(
            {"error": str(e)},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )

@method_decorator(csrf_exempt, name='dispatch')
class SecureAttachmentReplaceAPIView(APIView):
    authentication_classes = []
    permission_classes = [permissions.AllowAny]
    parser_classes = [MultiPartParser, FormParser]

    def initial(self, request, *args, **kwargs):
        request._dont_enforce_csrf_checks = True
        super().initial(request, *args, **kwargs)

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)
```

```
        serializer = SecureAttachmentReplaceSerializer(
            data=request.data,
            context={"request": request},
            partial=True,
        )

        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        uploaded_file = request.FILES.get("file")
        external_url = (validated.get("external_url") or "").strip()

        try:
            with transaction.atomic():
                # Keep old values only for safety/logging if needed
                old_sharepoint_item_id = attachment.sharepoint_item_id
                old_sharepoint_url = attachment.sharepoint_url

                # ====================================
                # CASE 1: REPLACE WITH NEW FILE
                # ====================================
                if uploaded_file:
                    # Optional duplicate name check (recommended)
                    # Allow same current attachment, block others with same
active file name
                    existing_file = SecureAttachment.objects.filter(
                        asset_type="file",
```

```
                    file_name=uploaded_file.name,
                    is_active=True
                ).exclude(pk=attachment.pk).first()

                if existing_file:
                    return Response(
                        {"error": "Another active file with this file name already
exists."},
                        status=status.HTTP_409_CONFLICT
                    )

                upload_result = upload_file_to_sharepoint(
                    file_stream=uploaded_file.file,
                    filename=uploaded_file.name,
                    content_type=uploaded_file.content_type,
                    file_size=uploaded_file.size,
                )

                new_item_id = upload_result["id"]

                existing_item = SecureAttachment.objects.filter(
                    sharepoint_item_id=new_item_id
                ).exclude(pk=attachment.pk).first()

                if existing_item:
                    return Response(
                        {"error": "The uploaded SharePoint item already exists
in another attachment."},
                        status=status.HTTP_409_CONFLICT
```

```
                )

            new_view_url = create_view_only_link(new_item_id)

            # IMPORTANT:
            # Keep SAME attachment row, SAME id, SAME
assignments
            # Update only content-related fields
            attachment.asset_type = "file"

            # Requirement says keep existing name unchanged
            # So DO NOT set attachment.file_name =
uploaded_file.name
            # Keep old attachment.file_name as-is

            attachment.sharepoint_item_id = new_item_id
            attachment.sharepoint_url = new_view_url
            attachment.external_url = None
            attachment.save()

            # SAFETY RECOMMENDATION:
            # For v1, do NOT delete old SharePoint file automatically.
            # If you want later, we can add delayed cleanup.
            # old_sharepoint_item_id can be used later if needed.

        # ====================================
        # CASE 2: REPLACE WITH NEW URL
        # ====================================
        elif external_url:
```

```
                attachment.asset_type = "url"
                attachment.external_url = external_url
                attachment.sharepoint_item_id = None
                attachment.sharepoint_url = None
                attachment.save()

                # SAFETY RECOMMENDATION:
                # If old attachment was SharePoint file, do NOT auto-
delete for v1.
                # That prevents accidental data loss.

            # Re-fetch with prefetch so serializer gets fresh assignments
            attachment = get_attachment_queryset().get(pk=attachment.pk)

            return Response(
                {
                    "status": "replaced",
                    "attachment":
SecureAttachmentListSerializer(attachment).data,
                },
                status=200,
            )

        except IntegrityError:
            return Response(
                {"error": "A conflicting attachment already exists."},
                status=status.HTTP_409_CONFLICT
            )
```

```
        except Exception as e:
            print("ERROR in replace API:", repr(e))
            return Response(
                {"error": str(e)},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR
            )
class SecureAttachmentListAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def get(self, request):
        client_id = request.GET.get("client")
        program_id = request.GET.get("program")

        queryset = get_attachment_queryset()

        # TEMPORARY legacy filter support if query params are used
        if client_id and str(client_id).isdigit():
            queryset = queryset.filter(clients__id=int(client_id))

        if program_id and str(program_id).isdigit():
            queryset = queryset.filter(programs__id=int(program_id))

        queryset = queryset.distinct()
        serializer = SecureAttachmentListSerializer(queryset, many=True)
        return Response(serializer.data, status=200)


class SecureAttachmentDetailAPIView(APIView):
    permission_classes = [permissions.AllowAny]
```

```
    def patch(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer = SecureAttachmentUpdateSerializer(data=request.data,
partial=True)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data

        if "clients" in validated:
            attachment.clients.set(validated["clients"])

        if "programs" in validated:
            attachment.programs.set(validated["programs"])

        attachment.save()

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            SecureAttachmentListSerializer(attachment).data,
            status=200,
        )

    def delete(self, request, pk):
```

```
        attachment = get_object_or_404(SecureAttachment, pk=pk)

        try:
            with transaction.atomic():
                # If it's a SharePoint file, delete from SharePoint first
                if attachment.asset_type == "file" and
attachment.sharepoint_item_id:

delete_file_from_sharepoint(attachment.sharepoint_item_id)

                # Hard delete from DB
                attachment.delete()

            return Response({"status": "deleted"}, status=200)

        except Exception as e:
            return Response(
                {"error": f"Delete failed: {str(e)}"},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR
            )


class SecureAttachmentAssignAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)
```

```
        serializer =
SecureAttachmentAssignActionSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        client = validated["client"]
        program = validated["program"]

        # Create exact pair assignment
        assignment, created =
SecureAttachmentAssignment.objects.get_or_create(
            attachment=attachment,
            client=client,
            program=program,
        )

        # TEMPORARY: keep legacy M2M fields in sync for display
compatibility
        attachment.clients.add(client)
        attachment.programs.add(program)

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "assigned",
                "created": created,
```

```
            "attachment":
SecureAttachmentListSerializer(attachment).data,
        },
        status=200,
    )



class SecureAttachmentUnassignAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer =
SecureAttachmentAssignActionSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        client = validated["client"]
        program = validated["program"]

        deleted_count, _ = SecureAttachmentAssignment.objects.filter(
            attachment=attachment,
            client=client,
            program=program,
        ).delete()
```

```
        # TEMPORARY: rebuild legacy M2M fields from remaining exact
assignments
        remaining_assignments =
SecureAttachmentAssignment.objects.filter(
            attachment=attachment
        )

        remaining_client_ids = list(
            remaining_assignments.values_list("client_id",
flat=True).distinct()
        )
        remaining_program_ids = list(
            remaining_assignments.values_list("program_id",
flat=True).distinct()
        )

        attachment.clients.set(remaining_client_ids)
        attachment.programs.set(remaining_program_ids)

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "unassigned",
                "deleted": deleted_count > 0,
                "attachment":
SecureAttachmentListSerializer(attachment).data,
            },
```

```
            status=200,
        )


class ProgramsByClientAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def get(self, request):
        client_id = request.GET.get("client")

        if not client_id:
            return Response([], status=200)

        programs = Program.objects.filter(client_id=client_id)
        serializer = ProgramSerializer(programs, many=True)
        return Response(serializer.data)
```

attachments/views.py

```
import graphene
from graphene_django.types import DjangoObjectType
from client.models import Client
from program.models import Program
from attachments.models import SecureAttachment

class ClientType(DjangoObjectType):
    class Meta:
        model = Client
        fields = ("id", "name")

class ProgramType(DjangoObjectType):
```

```
    class Meta:
        model = Program
        fields = ("id", "name", "client")

class AttachmentType(DjangoObjectType):
    class Meta:
        model = SecureAttachment
        fields = ("id", "file_name", "description", "sharepoint_url", "clients",
"programs")

class Query(graphene.ObjectType):
    attachments = graphene.List(
        AttachmentType,
        client_id=graphene.Int(),
        program_id=graphene.Int()
    )

    clients = graphene.List(ClientType)
    programs_by_client = graphene.List(ProgramType,
client_id=graphene.Int())

    def resolve_attachments(self, info, client_id=None,
program_id=None):
        qs =
SecureAttachment.objects.filter(is_active=True).prefetch_related("clients
", "programs")

        if client_id:
            qs = qs.filter(clients__id=client_id)
```

```
        if program_id:
            qs = qs.filter(programs__id=program_id)

        return qs

    def resolve_clients(self, info):
        return Client.objects.all()

    def resolve_programs_by_client(self, info, client_id):
        return Program.objects.filter(client_id=client_id)
```

attachments/graphql/queries.py

```
import graphene
from .queries import Query

schema = graphene.Schema(query=Query)
```

attachments/graphql/schema.py

now leaderboard code

```
import pytz
import graphene
from datetime import datetime
from django.conf import settings
from graphql_auth import mutations
from graphql_jwt import Refresh, Verify

class ObtainJSONWebToken(mutations.ObtainJSONWebToken):
    @classmethod
    def resolve(cls, root, info, **kwargs):
```

```
        user = info.context.user

        if user.is_authenticated:
            user.last_login =
datetime.now(tz=pytz.timezone(settings.TIME_ZONE))
            user.save()

        return super().resolve(root, info, **kwargs)

class Mutation(graphene.ObjectType):
    token_auth = ObtainJSONWebToken.Field()
    token_refresh = Refresh.Field()
    token_verify = Verify.Field()

schema = graphene.Schema(mutation=Mutation)
this is leaderboard/graphql/mutations.py
# leaderboard.graphql.schema.py

import graphene
import channels_graphql_ws

from graphql_auth.schema import UserQuery, MeQuery

from graphql_auth import mutations
from leaderboard.graphql.mutations import Mutation as
JWTAuthMutation

## paswordless
from users.passwordless import RegisterInviteUser
```

```
##  apis
from client.graphql.queries import Query as ClientQueries
from program.graphql.queries import Query as ProgramQueries
from client.graphql.mutations import Mutation as ClientMutations
from program.graphql.mutations import Mutation as
ProgramMutations
from client.graphql.subscriptions import Subscription as
ClientSubscriptions
from program.graphql.subscriptions import Subscription as
ProgramSubscriptions
from whatsapp.graphql.queries import Query as WAqueries
from whatsapp.graphql.mutations import Mutation as WAmutations
from whatsapp.graphql.subscriptions import Subscription as
WAsubscriptions

##

class AuthMutation(graphene.ObjectType):
    register_invite_user = RegisterInviteUser.Field()
    register = mutations.Register.Field()
    verify_account = mutations.VerifyAccount.Field()
    resend_activation_email = mutations.ResendActivationEmail.Field()
    send_password_reset_email =
mutations.SendPasswordResetEmail.Field()
    password_reset = mutations.PasswordReset.Field()
    # password_set = mutations.PasswordSet.Field() # For passwordless
registration
    password_change = mutations.PasswordChange.Field()
    update_account = mutations.UpdateAccount.Field()
```

```
    archive_account = mutations.ArchiveAccount.Field()
    delete_account = mutations.DeleteAccount.Field()
    send_secondary_email_activation =
mutations.SendSecondaryEmailActivation.Field()
    verify_secondary_email = mutations.VerifySecondaryEmail.Field()
    swap_emails = mutations.SwapEmails.Field()
    remove_secondary_email = mutations.RemoveSecondaryEmail.Field()
    verify_token = mutations.VerifyToken.Field()
    refresh_token = mutations.RefreshToken.Field()
    revoke_token = mutations.RevokeToken.Field()

class Query(
            UserQuery,
            MeQuery,
            ClientQueries,
            ProgramQueries,
            WAqueries,
            graphene.ObjectType):
    pass

class Mutation(
                AuthMutation,
                JWTAuthMutation,
                ClientMutations,
                ProgramMutations,
                WAmutations,
                graphene.ObjectType):
    pass
```

```
class Subscription(
                ClientSubscriptions,
                ProgramSubscriptions,
                WAsubscriptions,
                graphene.ObjectType):
    pass


schema = graphene.Schema(query=Query, mutation=Mutation,
subscription=Subscription)

class
MyGraphqlWsConsumer(channels_graphql_ws.GraphqlWsConsumer):
    schema = schema

    async def on_connect(self, payload):
        pass
this is leaderboard/graphql/schema.py

now program code
import graphene
from .types import *
from ..models import *
import program.graphql.Program.schema
import program.graphql.Activity.schema
import program.graphql.ParticipantProgramEnrollment.schema
import program.graphql.ParticipantSessionAttendance.schema
import program.graphql.Session.schema
import program.graphql.Topic.schema
```

```
import program.graphql.ParticipantActivityResponse.schema
import program.graphql.ParticipantActivityStatus.schema
import program.graphql.UserProgramPreference.schema
import program.graphql.ScorecardConfiguration.schema
import program.graphql.ScorecardActivity.schema
import program.graphql.ParticipantProgramAttendance.schema
import program.graphql.ActivityTemplate.schema
import program.graphql.ParticipantReplacement.schema
import program.graphql.PaymentNotificationRule.schema
import program.graphql.PaymentNotificationRun.schema
import program.graphql.Review.schema
class Mutation(
        program.graphql.Program.schema.Mutation,
        program.graphql.Activity.schema.Mutation,

program.graphql.ParticipantProgramEnrollment.schema.Mutation,

program.graphql.ParticipantSessionAttendance.schema.Mutation,
        program.graphql.Session.schema.Mutation,
        program.graphql.Topic.schema.Mutation,

program.graphql.ParticipantActivityResponse.schema.Mutation,
        program.graphql.ParticipantActivityStatus.schema.Mutation,
        program.graphql.UserProgramPreference.schema.Mutation,
        program.graphql.ScorecardConfiguration.schema.Mutation,
        program.graphql.ScorecardActivity.schema.Mutation,

program.graphql.ParticipantProgramAttendance.schema.Mutation,
        program.graphql.ActivityTemplate.schema.Mutation,
```

```
            program.graphql.ParticipantReplacement.schema.Mutation,
            program.graphql.PaymentNotificationRule.schema.Mutation,
            program.graphql.PaymentNotificationRun.schema.Mutation,
            program.graphql.Review.schema.Mutation,
            graphene.ObjectType):
    pass
this is program/graphql/mutations.py
import graphene
from .types import *
from ..models import *
import program.graphql.Activity.schema
import program.graphql.ParticipantActivityResponse.schema
import program.graphql.ParticipantProgramEnrollment.schema
import program.graphql.ParticipantSessionAttendance.schema
import program.graphql.Program.schema
import program.graphql.Session.schema
import program.graphql.Topic.schema
import program.graphql.ParticipantActivityStatus.schema
import program.graphql.UserProgramPreference.schema
import program.graphql.ScorecardConfiguration.schema
import program.graphql.ScorecardActivity.schema
import program.graphql.ParticipantProgramAttendance.schema
import program.graphql.ActivityTemplate.schema
import program.graphql.ParticipantReplacement.schema
import program.graphql.PaymentNotificationRule.schema
import program.graphql.PaymentNotificationRun.schema
import program.graphql.Review.schema
```

```
class Query(    program.graphql.Activity.schema.Query,
                program.graphql.ParticipantActivityResponse.schema.Query,

program.graphql.ParticipantProgramEnrollment.schema.Query,

program.graphql.ParticipantSessionAttendance.schema.Query,
                program.graphql.Program.schema.Query,
                program.graphql.Session.schema.Query,
                program.graphql.Topic.schema.Query,
                program.graphql.ParticipantActivityStatus.schema.Query,
                program.graphql.UserProgramPreference.schema.Query,
                program.graphql.ScorecardConfiguration.schema.Query,
                program.graphql.ScorecardActivity.schema.Query,

program.graphql.ParticipantProgramAttendance.schema.Query,
                program.graphql.ActivityTemplate.schema.Query,
                program.graphql.ParticipantReplacement.schema.Query,
                program.graphql.PaymentNotificationRule.schema.Query,
                program.graphql.PaymentNotificationRun.schema.Query,
                program.graphql.Review.schema.Query,
                graphene.ObjectType):
    pass
program/graphql/queries.py
import graphene
from .queries import Query
from .mutations import Mutation
from .subscriptions import Subscription

schema = graphene.Schema( query=Query,mutation=Mutation,
```

```
subscription=Subscription)
program/graphql/schema.py
import graphene
import django_filters
from ..models import *
from client.models import Client
from graphene import relay
from django.db.models import Q
from graphene_django.types import DjangoObjectType
from leaderboard.customConnections.extendedConnections import
ExtendedConnection
from django.contrib.auth import get_user_model
from graphene_django.filter import DjangoFilterConnectionField
from django.core.exceptions import ObjectDoesNotExist
import json
from ..journey_helpers import *
from ..journey_builder import compute_visibility
User = get_user_model()

class ProgramActivityType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    reason = graphene.String()
    total_responses = graphene.Int()
    class Meta:
        model = Activity
        interfaces = (relay.Node,)
        filter_fields = {
            "type": ["exact"],
```

```
            "description": ["exact", "icontains", "istartswith"],
        }
        connection_class = ExtendedConnection
    def resolve_visible(self, info):
        return compute_activity_vis(self, info.context.user)["accessible"]

    def resolve_accessible(self, info):
        return compute_activity_vis(self, info.context.user)["accessible"]
    def resolve_reason(self, info):
        return getattr(self, '_journey_reason', None)

    def resolve_total_responses(self, info):
        return ParticipantActivityResponse.objects.filter(activity = self,
status= "Complete").count()

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        activity_ids = Activity.objects.filter(
            Q(pre_program__enrolled_employees__participant=user) |
            Q(post_program__enrolled_employees__participant=user) |

Q(pre_session__program__enrolled_employees__participant=user) |

Q(post_session__program__enrolled_employees__participant=user) |
```

```
Q(pre_topics__topic_sessions__program__enrolled_employees__participa
nt=user) |

Q(post_topics__topic_sessions__program__enrolled_employees__particip
ant=user)
            ).distinct().values_list('id', flat=True)

        return queryset.filter(id__in=activity_ids).distinct()

class CustomUserFilterSet(django_filters.FilterSet):
    class Meta:
        model = User
        fields = {
            "id": ["exact"],
            "username": ["exact", "icontains", "istartswith"],
            "email": ["exact", "icontains"],
            "first_name": ["exact", "icontains", "istartswith"],
            "last_name": ["exact", "icontains", "istartswith"],
        }

class UserType(DjangoObjectType):
    class Meta:
        model = User
        fields = "__all__"
        filterset_class = CustomUserFilterSet
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection

class ActivityWithStageType(graphene.ObjectType):
```

```
    activity = graphene.Field(lambda: ProgramActivityType)
    stage = graphene.String()


class ProgramType(DjangoObjectType):
    activities = graphene.List(ActivityWithStageType)

    # Expose ManyToMany as a connection field
    account_manager = DjangoFilterConnectionField(UserType)
    pre_activities = graphene.List(ProgramActivityType)
    post_activities = graphene.List(ProgramActivityType)

    class Meta:
        model = Program
        fields = "__all__"
        interfaces = (relay.Node,)
        filter_fields = {
            "name": ["exact", "icontains", "istartswith"],
            "creator__id": ["exact"],
            "start_date": ["exact", "lt", "gt"],
            "end_date": ["exact", "lt", "gt"],
            "status": ["exact"],
            "client__id": ["exact"],
        }
        connection_class = ExtendedConnection

    def resolve_account_manager(self, info, **kwargs):
        return self.client.admins.filter(is_staff=True)
```

```
    def resolve_activities(self, info):
        sessions = self.sessions.all()
        topics = set()
        for session in sessions:
            for t in session.topic.all():
                topics.add(t)

        seen = set()
        results = []

        for topic in topics:
            for pre_act in topic.pre_activity.all():
                if pre_act.id not in seen:
                    results.append(
                        ActivityWithStageType(activity=pre_act, stage="Pre")
                    )
                    seen.add(pre_act.id)

            for post_act in topic.post_activity.all():
                if post_act.id not in seen:
                    results.append(
                        ActivityWithStageType(activity=post_act, stage="Post")
                    )
                    seen.add(post_act.id)

        return results
    def resolve_sessions(self, info):
        """Override sessions resolution to inject context"""
        sessions = self.sessions.all().order_by('start_date')
```

```
        # Inject program context
        for session in sessions:
            session._journey_program = self

        return sessions
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        return queryset.filter(Q(creator=user) |
Q(enrolled_employees__participant=user) | Q(client__id__in =
info.context.user.client_admins.all())).distinct()

class SessionType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()

    pre_activities = graphene.List(ProgramActivityType)
    post_activities = graphene.List(ProgramActivityType)
    rating = graphene.Int()
    class Meta:
        model = Session
        interfaces = (relay.Node,)
        filter_fields = {
            "program__id": ["exact"],
            "topic__id": ["exact"],
```

```
            "start_date": ["exact", "lt", "gt"],
            "end_date": ["exact", "lt", "gt"],
            "location": ["exact", "icontains", "istartswith"],
            "status": ["exact"],
        }
        connection_class = ExtendedConnection

    def resolve_visible(self, info):
        return compute_session_vis(self, info.context.user)["accessible"]

    def resolve_accessible(self, info):
        return compute_session_vis(self, info.context.user)["accessible"]
    def resolve_rating(self, info):
        return Review().rating(program_id=None,session_id=self.id)

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        return
queryset.filter(Q(program__enrolled_employees__participant=user) |
Q(attendees__participant=user)).distinct()

class ProgramActivityTemplateType(DjangoObjectType):
    class Meta:
        model = ActivityTemplate
        interfaces = (relay.Node,)
```

```
        filter_fields = {
            "type": ["exact"],
            "description": ["exact", "icontains", "istartswith"],
            "is_feedback":["exact"]
        }
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return None



class TopicType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    pre_activities = graphene.List(ProgramActivityType)
    post_activities = graphene.List(ProgramActivityType)
    class Meta:
        model = Topic
        interfaces = (relay.Node,)
        filter_fields = {
            "name": ["exact", "icontains", "istartswith"],
            "creator__id": ["exact"],
        }
        connection_class = ExtendedConnection
```

```
    def resolve_visible(self, info):
        return compute_topic_vis(self, info.context.user)["accessible"]

    def resolve_accessible(self, info):
        return compute_topic_vis(self, info.context.user)["accessible"]

    preActivityResponses = DjangoFilterConnectionField(
        'program.graphql.types.ParticipantActivityResponseType'
    )
    postActivityResponses = DjangoFilterConnectionField(
        'program.graphql.types.ParticipantActivityResponseType'
    )

    def resolve_preActivityResponses(self, info, **kwargs):
        return ParticipantActivityResponse.objects.filter(
            topic=self,
            activty_status="pre_activity",
            participant=info.context.user
        )

    def resolve_postActivityResponses(self, info, **kwargs):
        return ParticipantActivityResponse.objects.filter(
            topic=self,
            activty_status="post_activity",
            participant=info.context.user
        )

    @classmethod
    def get_queryset(cls, queryset, info):
```

```
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        enrolled_programs =
Program.objects.filter(enrolled_employees__participant=user)

        sessions = Session.objects.filter(program__in=enrolled_programs)

        topics_in_sessions =
Topic.objects.filter(topic_sessions__in=sessions).distinct()

        return queryset.filter(Q(id__in=topics_in_sessions.values_list('id',
flat=True)) | Q(creator=user)).distinct()
class ParticipantActivityStatusType(DjangoObjectType):
    class Meta:
        model = ParticipantActivityStatus
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "activity__id": ["exact"],
            "status": ["exact"],
            "completion_date": ["exact", "lt", "gt"],
        }
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
```

```
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)


class ParticipantProgramEnrollmentType(DjangoObjectType):
    attendance_percentage = graphene.Float()

    class Meta:
        model = ParticipantProgramEnrollment
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "program__id": ["exact"],
            "enrollment_date": ["exact", "lt", "gt"],
        }
        connection_class = ExtendedConnection

    def resolve_attendance_percentage(self, info):
        participant = self.participant
        program = self.program
        total_sessions = Session.objects.filter(program=program).count()
        attended_sessions = ParticipantSessionAttendance.objects.filter(
            participant=participant, session__program=program,
status="present"
        ).count()
        if total_sessions == 0:
            return 0
        return round((attended_sessions / total_sessions) * 100, 2)
```

```
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class ParticipantReplacementType(DjangoObjectType):

    class Meta:
        model = ParticipantReplacement
        interfaces = (relay.Node,)
        filter_fields = {
            "old_participant__id": ["exact"],
            "new_participant__id": ["exact"],
            "program__id": ["exact"],
            "old_enrollment_date": ["exact", "lt", "gt"],
            "replacement_date": ["exact", "lt", "gt"],
            "action":["exact"]
        }
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)
```

```
class ParticipantSessionAttendanceType(DjangoObjectType):
    class Meta:
        model = ParticipantSessionAttendance
        fields = ('id', 'status', 'participant', 'session','attendance_date')
        # participant = graphene.Field(UserType)
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "session__id": ["exact"],
            "status": ["exact"],
            "attendance_date": ["exact", "lt", "gt"],

        }

        connection_class = ExtendedConnection
        # def resolve_participant(self, info):
        #     return self.participant  # Your ForeignKey to User
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class ParticipantProgramAttendanceType(DjangoObjectType):
    class Meta:
        model = ParticipantProgramAttendance
        fields = ('id', 'status', 'participant', 'program','attendance_date')
        # participant = graphene.Field(UserType)
```

```
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "program__id": ["exact"],
            "status": ["exact"],
            "attendance_date": ["exact", "lt", "gt"],

        }

        connection_class = ExtendedConnection
        # def resolve_participant(self, info):
        #     return self.participant  # Your ForeignKey to User
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class ParticipantActivityResponseType(DjangoObjectType):
    class Meta:
        model = ParticipantActivityResponse
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "activity__id": ["exact"],
            "topic__id":["exact"],
            "session__id":["exact"],
            "program__id":["exact"],
```

```
            "submission_date": ["exact", "lt", "gt"],
        }
        connection_class = ExtendedConnection

    fetch = graphene.String()

    def resolve_fetch(self, info):
        if self.status != "Complete" and
Activity.objects.get(id=self.activity_id).mapp_survey:
            user_email = self.participant.email
            survey_ids = Activity.objects.get(
                id=self.activity_id
            ).mapp_survey
            response_date_gte = self.created_at

            if survey_ids:
                responses = (

ParticipantActivityResponse.fetch_mapp_survey_responses_on_refresh(
                        user_email, survey_ids, response_date_gte,
query_called=True
                    )
                )
                if isinstance(responses, str):
                    responses = json.loads(responses)
                if responses and len(responses) > 0:
                    self.response_data = responses
                    self.status = "Complete"
                    self.save()
```

```
        return self.status
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class UserProgramPreferenceType(DjangoObjectType):
    class Meta:
        model=UserProgramPreference
        fields=(
            "default_program",
            "user",
            "client",
            "role",
            "role_program_json"
        )
        filter_fields = {
            "default_program": ['exact'],
            "user": ['exact'],
            "client":['exact'],
            "role":['exact']
        }
        interfaces = (relay.Node, )
        connection_class = ExtendedConnection

    @classmethod
```

```
    def get_queryset(cls, queryset, info, **kwargs):
        if not info.context.user.is_staff:
            return queryset.filter(Q(user = info.context.user))
        else:
            return queryset

class ScorecardConfigurationType(DjangoObjectType):
    class Meta:
        model = ScorecardConfiguration

        filter_fields = {
            "program__id": ["exact"],
            "name": ["exact", "icontains", "istartswith"],
            "is_draft": ["exact"],
            "created_at": ["exact", "lt", "gt"],
            "updated_at": ["exact", "lt", "gt"],
            "program__client__id": ["exact"],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        if not info.context.user.is_staff:
            return queryset.filter(
                    Q(program__client__in=info.context.user.client_admins.all())
|

Q(program__enrolled_employees__participant=info.context.user)
```

```
            ).distinct()
        else:
            return queryset

class ScorecardActivityType(DjangoObjectType):
    class Meta:
        model = ScorecardActivity

        filter_fields = {
            "scorecard__id": ["exact"],
            "activity__id": ["exact"],
            "session__id":["exact"],
            "topic__id":["exact"],
            "program__id":["exact"],
            "weight_percentage": ["exact", "lt", "gt"],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        if not info.context.user.is_staff:
            return queryset.filter(

Q(scorecard__program__client__id__in=info.context.user.client_admins.al
l()) |

Q(scorecard__program__enrolled_employees__participant=info.context.
user)
            ).distinct()
```

```
        else:
            return queryset


class PaymentNotificationRuleType(DjangoObjectType):
    class Meta:
        model = PaymentNotificationRule
        fields = '__all__'
        filter_fields = {
            'program__id': ['exact'],
            'target_type': ['exact'],
            'when': ['exact'],
            'created_by__id': ['exact'],
            'created_at': ['gte', 'lte'],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)



class PaymentNotificationRunType(DjangoObjectType):
    class Meta:
        model = PaymentNotificationRun
```

```
        fields = '__all__'
        filter_fields = {
            'rule__id': ['exact'],
            'rule__program__id': ['exact'],
            'state': ['exact'],
            'fire_at': ['gte', 'lte'],
            'sent_at': ['gte', 'lte'],
            'created_at': ['gte', 'lte'],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)
class ReviewType(DjangoObjectType):
    class Meta:
        model = Review
        fields = '__all__'
        filter_fields = {
            'additional_feedback': ['exact'],
            'submitted_by__id': ['exact'],
            'program__id': ['exact'],
            'session__id': ['exact'],
            'created_at': ['gte', 'lte'],
        }
        interfaces = (relay.Node,)
```

```
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)
program/graphql/types.py
from leaderboard.decorators import graphql_permission_required
import graphene
from django.utils.dateparse import parse_datetime
from django.contrib.auth import get_user_model
from ...models import Program, Activity,ACTIVITY_TYPE_CHOICES
from ..types import ProgramActivityType
from leaderboard.events import send_subscription_and_notification
from django.db.models import Q
from django.db.models import Prefetch

User = get_user_model()

class CreateActivity(graphene.Mutation):
    class Arguments:
        type = graphene.String(required=True)
        name = graphene.String(required = True)
        template_id = graphene.Int()
        description = graphene.String()
        deadline = graphene.String()
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
```

```
        mapp_survey_name = graphene.String()
        mapp_tool = graphene.String()
        is_feedback = graphene.Boolean()
        learning_resource_id = graphene.Int()

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, type,
name,template_id=None,description=None, deadline=None,
mapp_survey=None, is_360=None,mapp_survey_name = None,
is_feedback=False,mapp_tool = None,learning_resource_id = None):
        try:
            deadline_parsed = parse_datetime(deadline) if deadline else
None

            activity = Activity(
                type=type,
                name = name,
                template_id = template_id,
                description=description if description else '',
                deadline=deadline_parsed,
                mapp_survey=mapp_survey,
                is_360=is_360,
                mapp_survey_name = mapp_survey_name,
                is_feedback = is_feedback,
```

```
            mapp_tool = mapp_tool,
            learning_resource_id = learning_resource_id
        )
        activity.full_clean()
        activity.save()

        send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": activity,
                "mutation": "CREATE",
            },
            # notification_event_type="survey_token_created",
            # notification_payload={
            #     "object": token,
            #     "user_id": user.id,
            # },
        )
        return CreateActivity(activity=activity, success=True,
message="Activity created successfully")
    except Program.DoesNotExist:
        return CreateActivity(activity=None, success=False,
message="Program not found.")
    except Exception as e:
        return CreateActivity(activity=None, success=False,
message=str(e))
```

```
class UpdateActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
        template_id = graphene.Int()
        name = graphene.String()
        type = graphene.String()
        description = graphene.String()
        deadline = graphene.String()  # ISO 8601 datetime string
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        is_feedback = graphene.Boolean()
        mapp_tool = graphene.String()
        learning_resource_id = graphene.Int()

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id, template_id=None,name=None,
type=None, description=None, deadline=None, mapp_survey=None,
is_360=None,mapp_survey_name = None,
is_feedback=None,mapp_tool = None,learning_resource_id = None):
        try:
            activity = Activity.objects.get(pk=id)
```

```
        # Handle type with choices validation
        if type is not None:
            activity.type = type

        # Handle deadline parsing
        if deadline is not None:
            deadline_parsed = parse_datetime(deadline)
            if not deadline_parsed:
                return UpdateActivity(activity=None, success=False,
message="Invalid deadline format.")
            activity.deadline = deadline_parsed

        # Update simple fields (None will set NULL)
        if name is not None:
            activity.name = name
        if description is not None:
            activity.description = description
        activity.mapp_survey = mapp_survey
        activity.mapp_survey_name = mapp_survey_name
        activity.mapp_tool = mapp_tool
        activity.is_360 = is_360
        if template_id:
            activity.template_id = template_id

        if is_feedback is not None:
            activity.is_feedback = is_feedback
        if learning_resource_id is not None:
            activity.learning_resource_id = learning_resource_id
```

```
            activity.full_clean()
            activity.save()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "UPDATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )

            participants = User.objects.filter(
                program_enrollments__program__in=Program.objects.filter(
                    Q(pre_activity=activity) |
                    Q(post_activity=activity) |
                    Q(sessions__pre_activity=activity) |
                    Q(sessions__post_activity=activity) |
                    Q(sessions__topic__pre_activity=activity) |
                    Q(sessions__topic__post_activity=activity)
                )
            ).distinct()
```

```
            for participant in participants:
                send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                    subscription_class="ActivitySubscription",
                    group=participant.username,
                    subscription_payload={
                        "object": activity,
                        "mutation": "UPDATE",
                    },
                    # notification_event_type="survey_token_created",
                    # notification_payload={
                    #     "object": token,
                    #     "user_id": user.id,
                    # },
                )
            return UpdateActivity(activity=activity, success=True,
message="Activity updated successfully")
        except Activity.DoesNotExist:
            return UpdateActivity(activity=None, success=False,
message="Activity not found.")
        except Exception as e:
            return UpdateActivity(activity=None, success=False,
message=str(e))


class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
```

```
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity.delete()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return DeleteActivity(success=True, message="Activity deleted
successfully")
        except Activity.DoesNotExist:
            return DeleteActivity(success=False, message="Activity not
```

```
found")


class Mutation(graphene.ObjectType):
    create_activity = CreateActivity.Field()
    update_activity = UpdateActivity.Field()
    delete_activity = DeleteActivity.Field()

"""
Example:
mutation {
  createActivity(
    type: "quiz"
    description: "A quiz for the program"
    deadline: "2025-12-10T17:00:00Z"
    mappSurvey: 123
    is360: true
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360
```

```
    }
  }
}

mutation {
  updateActivity(
    id: "5"
    type: "audio"
    description: "Updated description"
    deadline: "2025-12-15T10:30:00Z"
    mappSurvey: 456
    is360: false
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360
    }
  }
}

mutation {
  deleteActivity(id: "5") {
```

```
    success
    message
  }
}
"""
this is program/graphql/activity/mutation.py
import graphene
import os
from ..types import ProgramActivityType
from ...models import Activity, ParticipantActivityResponse
from graphql_jwt.decorators import login_required
from graphene_django.filter import DjangoFilterConnectionField
from leaderboard.settings import domain, MApp_group_surveys


class CreateSurveyTokenType(graphene.ObjectType):
    token = graphene.String()
    token_id = graphene.ID()
    valid = graphene.Boolean()
    invited = graphene.Boolean()

class UserSurveyInput(graphene.InputObjectType):
    first_name = graphene.String(required=False)
    last_name = graphene.String(required=False)
    email = graphene.String(
        description="Email is required for sending an email invitation.",
        required=False,
    )
    respondentType = graphene.String(required=False)
```

```
class MAppSurveyType(graphene.ObjectType):
    id = graphene.Int()
    name = graphene.String()

class Query(graphene.ObjectType):
    all_activities = DjangoFilterConnectionField(ProgramActivityType,
orderBy=graphene.List(of_type=graphene.String), description = "Get all
Sessions.")
    activity_by_id = graphene.Field(ProgramActivityType,
id=graphene.Int(required=True))
    create_survey_token =
graphene.String(activity_id=graphene.Int(required=True),invite=graphe
ne.Boolean(required=False),message=graphene.String(required=False),t
opic_id =graphene.Int(required=False),session_id
=graphene.Int(required=False),program_id
=graphene.Int(required=False),activty_status =
graphene.String(required=False, description = "please add activity
status from pre_activity or post_activity"),user_input =
UserSurveyInput(required=False,description = "add user input data
when 360 invite are send") )
    mapp_surveys = graphene.List(MAppSurveyType)
    fetch_mapp_tool_used_date = graphene.JSONString(
        user_email=graphene.String(required=True),
        module_name=graphene.String(required=True),
        date_start=graphene.DateTime(required=False),
        date_end=graphene.DateTime(required=False)
    )
```

```
    @login_required
    def resolve_fetch_mapp_tool_used_date(self, info, user_email,
module_name, date_start=None, date_end=None):
        return Activity.fetch_mapp_tool_used_date(user_email,
module_name, date_start, date_end)

    @login_required
    def resolve_mapp_surveys(root, info):
        survey_dict = Activity.get_cached_surveys()
        return survey_dict

    @login_required
    def resolve_all_activities(self, info, **kwargs):
        return Activity.objects.all()

    @login_required
    def resolve_activity_by_id(self, info, id):
        try:
            return Activity.objects.get(id=id)
        except Activity.DoesNotExist:
            return None

    @login_required
    def resolve_create_survey_token(self, info, activity_id, invite=False,
message=None,topic_id = None, session_id = None,program_id =
None,activty_status=None,user_input=None):
        user = info.context.user
        participant_activity_response_obj =
ParticipantActivityResponse.objects.filter(
```

```
            participant=info.context.user,
            activity_id=activity_id,
            topic_id=topic_id,
            session_id=session_id,
            program_id=program_id,
            activty_status=activty_status
        )

        if not user_input and not invite and
participant_activity_response_obj.exists():
            return participant_activity_response_obj.first().token_link
        if user_input:

user_input["responseSourceFor_id"]=participant_activity_response_obj.fi
rst().response_data["responseSourceFor_id"]
        else:
            user_input = {
                "first_name": getattr(user, "first_name", "") or "",
                "last_name": getattr(user, "last_name", "") or "",
                "email": getattr(user, "email", "") or "",
                "mobileNumber": getattr(user.profile, "phone_number", "") if
hasattr(user, "profile") else None
            }

        # is_360_survey = Activity.objects.get(id = activity_id).is_360
        token_data = ""
        try:

            token_data = Activity.create_survey_token_in_mapp(
```

```
            user_data=user_input,
            survey_id=Activity.objects.get(id = activity_id).mapp_survey,
            invite=invite,
            message=message
        )

        if Activity.objects.get(id = activity_id).mapp_survey in
MApp_group_surveys:
            token_link = os.path.join(domain,'survey/group-
survey/survey/token',token_data)
        else:
            token_link =
os.path.join(domain,'survey/survey/token',token_data)

        if not invite:
            ParticipantActivityResponse.objects.create(participant =
info.context.user,activity_id = activity_id, status = 'Pending', topic_id =
topic_id, session_id=session_id,program_id=program_id,activty_status
= activty_status, token_link = token_link)
        if Activity.objects.get(id = activity_id).mapp_survey in
MApp_group_surveys:
            return os.path.join(domain,'survey/group-
survey/survey/token',token_data)
        return os.path.join(domain,'survey/survey/token',token_data)
    except Exception as e:
        raise Exception(f"Failed to create MApp survey token: {str(e)}")

"""
Example:
```

```
query{
  allActivities(type:ASSIGNMENT){
    edges{
      node {
        id
        type
        description
        deadline

      }
  }
}
}


query{
  activityById(id:1){
    id
    type
    deadline
  }
}

query {
  Program(id: 1) {
    activities {
      activity {
        id
```

```
        description
        type
      }
      stage
    }
  }
}


query{
  createSurveyToken(surveyId:4)
}

"""
this is program/graphql/activity/queries.py
import graphene
from .queries import Query
from .mutations import Mutation
from .subscriptions import Subscription

schema = graphene.Schema(query=Query, mutation=Mutation,
subscription=Subscription)
program/graphql/activity/schema.py
```

those are my my code files now tell me if it is possible i am working in team project so i want to do it without breaking anything else now tell me how to do this step by step proparlly is it possible and tell me how to check if it is succesfully connected or not do it very preciselly and give me everything step by step with propar path

Yes — **it is absolutely possible**, and you can do it **safely without breaking the team project** if you do it in **2 phases**:

✅ Best Safe Plan (Recommended)

Because your project already uses **GraphQL everywhere**, and your `attachments` app already has a **working REST API**, the safest integration is:

Phase 1 (Safe / Non-breaking)

Expose attachments through the main GraphQL schema
➡️ This lets your frontend use **one GraphQL endpoint** (`/graphql/`) like the rest of the project.

Phase 2 (Optional / Next Step)

Connect attachments directly to `Activity` model
➡️ This is needed only if you want:

- select attachment while creating activity
- save attachment against activity
- query activity with its attachments

🚨 Very Important (Current Situation)

Right now:

- `attachments` app has **REST API**

- `attachments` app also has a **separate GraphQL endpoint** at:
  - `/attachments/graphql/`
- Main project GraphQL endpoint is:
  - `/graphql/`

Since the rest of your app uses `/graphql/`, **best practice is to merge attachment queries into the main leaderboard GraphQL schema**.

✅ What You Should Do (Safe Team-Friendly Approach)

I'll give you the exact step-by-step in the **safest order**:

PHASE 1 — CONNECT ATTACHMENTS TO MAIN GRAPHQL (NO MODEL BREAKING)

This phase will **NOT disturb existing REST upload/list APIs**.
It only adds attachment queries to the main `/graphql/`.

STEP 1) Keep your REST APIs as they are

Do not remove or change these files now:

- `attachments/views.py`
- `attachments/serializers.py`
- `attachments/urls.py`

These are already working.
We will **reuse them** and only **add GraphQL integration**.

✅ Safe for team project.

STEP 2) Create a proper GraphQL module structure inside attachments

Your current attachment GraphQL files are:

- `attachments/graphql/queries.py`
- `attachments/graphql/schema.py`

This is okay, but to match your project style, I strongly recommend creating:

- `attachments/graphql/queries.py`
- `attachments/graphql/mutations.py`
- `attachments/graphql/schema.py`

Even if mutations are empty at first.

Create file:

`attachments/graphql/mutations.py`

```python
import graphene
```

```python
pass
```

✅ This keeps structure consistent with `program/graphql`.

STEP 3) Improve your attachment GraphQL types (VERY IMPORTANT)

Your current `attachments/graphql/queries.py` is too basic and uses only:

- `file_name`
- `description`
- `sharepoint_url`
- `clients`
- `programs`

But your actual model has more useful fields:

- `asset_type`
- `external_url`
- `assignments`
- `created_at`
- `updated_at`

Also, for future frontend integration, we should expose those too.

Replace your file:

`attachments/graphql/queries.py`

with this **safer improved version**:

```
import graphene
from graphene_django.types import DjangoObjectType

from client.models import Client
from program.models import Program
from attachments.models import SecureAttachment, SecureAttachmentAssignment


class ClientType(DjangoObjectType):
    class Meta:
        model = Client
        fields = ("id", "name")


class ProgramType(DjangoObjectType):
    class Meta:
        model = Program
        fields = ("id", "name", "client")


class AttachmentAssignmentType(DjangoObjectType):
    class Meta:
```

STEP 4) Update attachments GraphQL schema file

Replace:

`attachments/graphql/schema.py`

with:

```
import graphene
from .queries import Query
from .mutations import Mutation

schema = graphene.Schema(query=Query, mutation=Mutation)
```

✅ This makes it consistent.

STEP 5) Merge attachments GraphQL into main leaderboard GraphQL schema

This is the **main integration step**.

Your main file:

`leaderboard/graphql/schema.py`

currently imports:

- `ClientQueries`
- `ProgramQueries`
- `WAqueries`
- and mutations/subscriptions

We will **add AttachmentsQueries and AttachmentsMutations**.

Update imports in:

`leaderboard/graphql/schema.py`

Add these imports:

```python
from attachments.graphql.queries import Query as AttachmentsQueries
```

Python Run

Then update your `Query` class

CURRENT:

```python
class Query(
            UserQuery,
            MeQuery,
            ClientQueries,
            ProgramQueries,
            WAqueries,
            graphene.ObjectType):
    pass
```

CHANGE TO:

```
class Query(
            UserQuery,
            MeQuery,
            ClientQueries,
            ProgramQueries,
            AttachmentsQueries,
            WAqueries,
            graphene.ObjectType):
    pass
```

Then update your `Mutation` class

CURRENT:

```
class Mutation(
                AuthMutation,
                JWTAuthMutation,
                ClientMutations,
                ProgramMutations,
                WAmutations,
                graphene.ObjectType):
    pass
```

CHANGE TO:

```
class Mutation(
                AuthMutation,
```

```
                JWTAuthMutation,
                ClientMutations,
                ProgramMutations,
                AttachmentsMutations,
                WAmutations,
                graphene.ObjectType):
    pass
```

✅ Result After Step 5

Now your **main** `/graphql/` **endpoint** will support:

- `attachments`
- `attachmentById`
- `clients`
- `programsByClient`

⚠️ Note: `clients` and `programsByClient` names may clash if other apps already define same field names.

STEP 6) Avoid GraphQL field name conflicts (VERY IMPORTANT)

This is a **team-project safety issue**.

In your attachments GraphQL, you currently expose:

- `clients`
- `programs_by_client`

But `client` app may already expose similar names.

To avoid conflicts, rename attachment helper fields.

Safer version (recommended)

Change in `attachments/graphql/queries.py`:

Replace:

```python
clients = graphene.List(ClientType)
```

Python Run

With:

```python
attachment_clients = graphene.List(ClientType)
attachment_programs_by_client = graphene.List(ProgramType, client_id=graphene.Int(required=True))
```

Python Run

Then replace resolvers:

```python
def resolve_attachment_clients(self, info):
    return Client.objects.all()

def resolve_attachment_programs_by_client(self, info, client_id):
    return Program.objects.filter(client_id=client_id)
```

✅ Final safer `attachments/graphql/queries.py` field names

Use:

- `attachments`
- `attachmentById`
- `attachmentClients`
- `attachmentProgramsByClient`

Because Graphene converts snake_case → camelCase.

So frontend query becomes:

- `attachments`
- `attachmentById`
- `attachmentClients`
- `attachmentProgramsByClient`

✅ Much safer.

STEP 7) Restart server and test main GraphQL

After changes:

```bash
```

Then open your main GraphQL:

- `http://127.0.0.1:8000/graphql/`

STEP 8) Test if attachment is successfully connected to main GraphQL

Run this query in **main GraphQL endpoint** (`/graphql/`):

```
query {
  attachments {
    id
    fileName
    description
    assetType
    sharepointUrl
    externalUrl
    createdAt
    updatedAt
    clients {
      id
      name
    }
    programs {
      id
      name
    }
    assignments {
      id
      createdAt
      client {
```

✅ If it works, you are successfully connected.

You should get data like:

```
{
  "data": {
    "attachments": [
      {
        "id": "1",
        "fileName": "test.pdf",
        "description": "sample file",
        ...
      }
    ]
  }
}
```

STEP 9) Also test filtered query

```
query {
  attachments(clientId: 1, programId: 2) {
    id
    fileName
    sharepointUrl
  }
}
```

If filtered data comes correctly, then your attachment app is properly connected with main project GraphQL.

PHASE 2 — CONNECT ATTACHMENTS TO ACTIVITY (Needed for your actual goal)

From your earlier requirement, you want:

> when creating activity in program/session, show dropdown of uploaded files and attach selected file to that activity

For that, **Phase 1 alone is not enough**.

You must also connect `SecureAttachment` with `Activity` model.

IMPORTANT DESIGN CHOICE (Safest)

❌ Do NOT modify existing `SecureAttachmentAssignment`

That model is for:

- attachment ↔ client ↔ program

It is not for activity-level relation.

✅ Best safe way:

Add a **new ManyToMany relation** between `Activity` and `SecureAttachment`

This is simplest and least risky.

STEP 10) Update `program/models.py` → `Activity` model

You did not share full `Activity` model file, but wherever your `Activity` model is defined, add this field:

In `program/models.py` (inside `Activity` model)

```
attachments = models.ManyToManyField(
    "attachments.SecureAttachment",
    blank=True,
    related_name="activities"
)
```

Example:

```
class Activity(models.Model):
    # existing fields...
    type = models.CharField(max_length=100)
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True, null=True)

    attachments = models.ManyToManyField(
        "attachments.SecureAttachment",
        blank=True,
```

```
        related_name="activities"
    )

    # rest of model...
```

STEP 11) Create migration

```bash
python manage.py makemigrations
```

You should see migration for `program` app only.

Example:

- `program/migrations/00xx_add_attachments_to_activity.py`

✅ This is safe and reversible.

STEP 12) Expose activity attachments in GraphQL type

Your `ProgramActivityType` is in:

`program/graphql/types.py`

Inside `ProgramActivityType`, add a field manually for attachments.

Add import at top:

```python
from attachments.models import SecureAttachment
```

Python Run

⚠️ But importing `AttachmentType` from `queries.py` is not ideal because of circular structure.

Better approach:

Move GraphQL types into a separate file.

SAFER STRUCTURE (Recommended)

Create:

`attachments/graphql/types.py`

```python
from graphene_django.types import DjangoObjectType
from client.models import Client
from program.models import Program
from attachments.models import SecureAttachment, SecureAttachmentAssignment


class ClientType(DjangoObjectType):
    class Meta:
        model = Client
        fields = ("id", "name")
```

```
class ProgramType(DjangoObjectType):
    class Meta:
        model = Program
        fields = ("id", "name", "client")


class AttachmentAssignmentType(DjangoObjectType):
    class Meta:
        model = SecureAttachmentAssignment
```

STEP 13) Update `attachments/graphql/queries.py` to import from `types.py`

Replace class definitions with imports:

```
import graphene
from client.models import Client
from program.models import Program
from attachments.models import SecureAttachment
from .types import AttachmentType, ClientType, ProgramType
```

And remove duplicate type classes.

STEP 14) Add attachments field to `ProgramActivityType`

In:

`program/graphql/types.py`

Add imports at top:

Python Run

Then inside `ProgramActivityType` add:

Python Run

So:

```python
class ProgramActivityType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    reason = graphene.String()
    total_responses = graphene.Int()
    attachments = graphene.List(AttachmentType)
```

Add resolver:

```python
def resolve_attachments(self, info):
    return self.attachments.filter(is_active=True).prefetch_related(
        "clients",
        "programs",
        "assignments__client",
        "assignments__program",
    )
```

STEP 15) Update createActivity mutation to accept attachment IDs

Your file:

`program/graphql/Activity/mutations.py`

Add new argument in `CreateActivity.Arguments`

Add:

Python Run

So it becomes:

```python
class Arguments:
    type = graphene.String(required=True)
    name = graphene.String(required=True)
    template_id = graphene.Int()
    description = graphene.String()
    deadline = graphene.String()
    mapp_survey = graphene.Int()
    is_360 = graphene.Boolean()
    mapp_survey_name = graphene.String()
    mapp_tool = graphene.String()
    is_feedback = graphene.Boolean()
    learning_resource_id = graphene.Int()
    attachment_ids = graphene.List(graphene.Int)
```

Update mutate signature

Change:

```python
def mutate(root, info, type, name, template_id=None, description=None, deadline=None, mapp_survey
```

Python Run

To:

```python
def mutate(
    root,
    info,
    type,
    name,
```

```
    template_id=None,
    description=None,
    deadline=None,
    mapp_survey=None,
    is_360=None,
    mapp_survey_name=None,
    is_feedback=False,
    mapp_tool=None,
    learning_resource_id=None,
    attachment_ids=None,
):
```

STEP 16) Import SecureAttachment in mutation file

At top of:

`program/graphql/Activity/mutations.py`

Add:

Python Run

STEP 17) Save selected attachments after activity creation

Inside `CreateActivity.mutate`, after:

Python Run

Add:

```python
if attachment_ids:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

Final create block should look like:

```python
activity.full_clean()
activity.save()

if attachment_ids:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

STEP 18) Update updateActivity mutation also (recommended)

Same file:

`program/graphql/Activity/mutations.py`

Add in `UpdateActivity.Arguments`:

Python Run

Update mutate signature:

```
def mutate(root, info, id, template_id=None, name=None, type=None, description=None, deadline=Non
```

Python Run

Then before `activity.full_clean()` or after `activity.save()`:

Recommended **after save**:

```
if attachment_ids is not None:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
```

```
    )
    activity.attachments.set(attachments)
```

⚠️ Use `is not None` so:

- if frontend sends `[]` → clears attachments
- if frontend omits field → leaves unchanged

STEP 19) Test createActivity with attachment IDs

Now test in `/graphql/`

```
mutation {
  createActivity(
    type: "quiz"
    name: "Test Activity With File"
    description: "Testing attachment"
    attachmentIds: [1, 2]
  ) {
    success
    message
    activity {
      id
      name
      attachments {
        id
        fileName
        assetType
        sharepointUrl
        externalUrl
      }
```

```
    }
  }
}
```

✅ If successful, you should get:

```
{
  "data": {
    "createActivity": {
      "success": true,
      "message": "Activity created successfully",
      "activity": {
        "id": "123",
        "name": "Test Activity With File",
        "attachments": [
          {
            "id": "1",
            "fileName": "abc.pdf"
          }
        ]
      }
    }
  }
}
```

STEP 20) Test existing activity query

Your current `activityById` query already returns `ProgramActivityType`.

So now test:

```
query {
  activityById(id: 123) {
    id
    name
    attachments {
      id
      fileName
      assetType
      sharepointUrl
      externalUrl
    }
  }
}
```

If it returns attachments → fully connected.

⚠️ VERY IMPORTANT ABOUT YOUR FRONTEND ERROR

You earlier had this error:

> `No activity id returned from createActivity`

This is because your mutation returns:

```
createActivity {
  success
```

```
  message
  activity {
    id
  }
}
```

So frontend must read:

JavaScript

NOT

JavaScript

Correct response structure is:

```
{
  "data": {
    "createActivity": {
      "success": true,
      "message": "Activity created successfully",
      "activity": {
        "id": "123"
      }
    }
```

```
  }
}
```

So in frontend, use:

```
</> TypeScript
```

This is likely the exact reason for your previous console error.

✅ SAFEST FILES YOU WILL CHANGE

Here is the **exact file list** you should touch.

PHASE 1 files

1. Create

- `attachments/graphql/mutations.py`

2. Create (recommended)

- `attachments/graphql/types.py`

3. Update

- `attachments/graphql/queries.py`

4. Update

- `attachments/graphql/schema.py`

5. Update

- `leaderboard/graphql/schema.py`

PHASE 2 files

6. Update

- `program/models.py`
  (inside `Activity` model only)

7. Migration auto-created

- `program/migrations/xxxx_add_attachments_to_activity.py`

8. Update

- `program/graphql/types.py`

9. Update

- `program/graphql/Activity/mutations.py`

✅ How To Do This Without Breaking Team Project

Follow this exact order:

Safe order:

1. **Create new GraphQL files first** in attachments

2. **Integrate into leaderboard schema**
3. **Test only queries first**
4. **Do NOT change frontend yet**
5. **Then add Activity model relation**
6. **Run migration**
7. **Then update activity GraphQL types**
8. **Then update createActivity/updateActivity mutations**
9. **Then test in GraphiQL**
10. **Then update frontend**

This minimizes breakage.

✅ How To Verify Success (Checklist)

CHECK 1 — Main GraphQL endpoint loads

Open:

- `/graphql/`

If GraphiQL loads without import/schema errors → good.

CHECK 2 — attachments query visible in docs

In GraphiQL docs explorer, check if these fields exist:

- `attachments`

- `attachmentById`
- `attachmentClients`
- `attachmentProgramsByClient`

CHECK 3 — query attachments works

```
query {
  attachments {
    id
    fileName
  }
}
```

CHECK 4 — activity now exposes attachments

```
query {
  activityById(id: 1) {
    id
    name
    attachments {
      id
      fileName
    }
```

```
  }
```

GraphQL

CHECK 5 — create activity with attachment IDs works

```graphql
mutation {
  createActivity(
    type: "quiz"
    name: "GraphQL Attachment Test"
    attachmentIds: [1]
  ) {
    success
    message
    activity {
      id
      name
      attachments {
        id
        fileName
      }
    }
  }
}
```

CHECK 6 — DB verification in Django shell (optional but best)

Run:

```bash
```

Then:

```python
from program.models import Activity
a = Activity.objects.get(id=123)
```

If attachment queryset comes → saved correctly.

⚠️ One More Important Issue (Possible Circular Import)

Because `attachments.graphql.types` may import `Program`, and `program.graphql.types` imports `AttachmentType`, there is a **chance of circular import**.

If circular import happens:

Instead of importing `AttachmentType` directly into `program/graphql/types.py`, use a **manual GraphQL object type** or move attachment GraphQL types to a neutral place.

But in many Django Graphene setups, it still works if imports are clean.

SAFEST ANTI-CIRCULAR ALTERNATIVE

If circular import happens in:

`program/graphql/types.py`

Instead of:

Python Run

Create a lightweight type there:

```python
class ActivityAttachmentType(graphene.ObjectType):
    id = graphene.Int()
    file_name = graphene.String()
    description = graphene.String()
    asset_type = graphene.String()
    sharepoint_url = graphene.String()
    external_url = graphene.String()
```

Then:

```python
attachments = graphene.List(ActivityAttachmentType)

def resolve_attachments(self, info):
```

Python Run

This avoids circular imports completely.

✅ For team project, this is actually safer.

🔥 My Recommendation (Best for Your Team Project)

Do this in 2 pull-request style stages

PR 1 (Safe)

- Add attachment queries into main `/graphql/`
- No model changes
- No frontend breakage

PR 2

- Add `Activity.attachments`
- Add create/update activity mutation support
- Then update frontend dropdown

This is the cleanest team-safe approach.

✅ Final Answer: Is it possible?

YES — 100% possible

And the **best safe architecture** is:

- Keep **REST** for:
  - upload
  - replace
  - assign/unassign
  - delete
- Use **main GraphQL** for:
  - listing attachments in frontend dropdown

- fetching attachments in activity details
- attaching selected files to activity via `attachmentIds`

This is the most practical approach for your current codebase.

🚀 What I Can Do Next (Highly Recommended)

If you want, in my **next reply** I can give you **exact copy-paste ready code** for these files only:

1. `attachments/graphql/types.py`
2. `attachments/graphql/queries.py`
3. `attachments/graphql/mutations.py`
4. `attachments/graphql/schema.py`
5. `leaderboard/graphql/schema.py` (**exact edited version**)
6. `program/graphql/types.py` (**exact patch**)
7. `program/graphql/Activity/mutations.py` (**exact patch**)

👉 That will be **production-safe copy-paste code**, with **minimal changes only**, matching your current project structure.

If you want, send me **your full** `program/models.py` **Activity model**, and I'll give you the **exact line where to add the** `attachments` **field** so you don't touch anything wrong.

env3.11) D:\Leaderboard_BE\Leaderboard_BE>python manage.py runserver

```
Watching for file changes with StatReloader
Performing system checks...

Exception in thread django-main-thread:
Traceback (most recent call last):
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\threading.py", line 1045, in _bootstrap_inner
    self.run()
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\threading.py", line 982, in run
    self._target(*self._args, **self._kwargs)
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\utils\autoreload.py", line 64, in wrapper
    fn(*args, **kwargs)
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-
packages\django\core\management\commands\runserver.py", line
133, in inner_run
    self.check(display_num_errors=True)
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\core\management\base.py", line 485, in
check
    all_issues = checks.run_checks(
                 ^^^^^^^^^^^^^^^^^^^
```

```
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\core\checks\registry.py", line 88, in
run_checks
    new_errors = check(app_configs=app_configs, databases=databases)

^^^
```

```
                                ^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\urls\resolvers.py", line 735, in url_patterns
    patterns = getattr(self.urlconf_module, "urlpatterns",
self.urlconf_module)
                    ^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\utils\functional.py", line 47, in __get__
    res = instance.__dict__[self.name] = self.func(instance)
                                ^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\urls\resolvers.py", line 728, in
urlconf_module
    return import_module(self.urlconf_name)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\importlib\__init__.py", line 126, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

^^^^^^^^^^^^^^^^^^^
```

```
  File "<frozen importlib._bootstrap>", line 690, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 940, in
exec_module
  File "<frozen importlib._bootstrap>", line 241, in
_call_with_frames_removed
  File "D:\Leaderboard_BE\Leaderboard_BE\leaderboard\urls.py", line 51,
in <module>
    path("api/attachments/", include("attachments.urls")),
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\urls\conf.py", line 38, in include
    urlconf_module = import_module(urlconf_module)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\importlib\__init__.py", line 126, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

^^^^^^^^^^^^^^^^^^^
```

```
_call_with_frames_removed
  File "D:\Leaderboard_BE\Leaderboard_BE\attachments\urls.py", line
17, in <module>
    from .graphql.schema import schema
  File
"D:\Leaderboard_BE\Leaderboard_BE\attachments\graphql\schema.py",
line 5, in <module>
    schema = graphene.Schema(query=Query, mutation=Mutation)

^^^
```

```
^^^
```

^^^

✅ Why this error happens

In **your Graphene version**, an empty mutation class like:

```python
class Mutation(graphene.ObjectType):
```

Python Run

is **not allowed**.

That is why you get:

> **AssertionError: Mutation fields must be a mapping...**

Because Graphene expects **at least one mutation field**, but your `Mutation` has **none**.

✅ SAFEST FIX (Recommended)

Since right now you are only adding **queries** for attachments, and you are **not using GraphQL mutations** in attachments yet:

👉 Remove mutation from `attachments/graphql/schema.py`

✅ Step-by-step fix

STEP 1) Open this file

`attachments/graphql/schema.py`

You currently have:

```
import graphene
from .queries import Query
from .mutations import Mutation

schema = graphene.Schema(query=Query, mutation=Mutation)
```

STEP 2) Replace it with this

```
import graphene
from .queries import Query
```

✅ That's it

This is the **best fix** for now.

Because:

- You only need **queries** at this stage
- Your upload/create/delete still uses **REST API**
- No need to force GraphQL mutations in attachments yet

❌ Do NOT do this right now

Do **not** keep this:

```python
from .mutations import Mutation
```

Python Run

unless your `Mutation` class has at least one actual mutation field.

✅ Optional cleanup (not required)

You can keep this file if you want:

`attachments/graphql/mutations.py`

```python
import graphene

class Mutation(graphene.ObjectType):
```

Python Run

That file is okay to keep **for future use**, but **do not include it in schema yet**.

✅ Why this is safe for team project

This change only affects:

- `attachments/graphql/schema.py`

It does **not** touch:

- main `/graphql/`
- program app
- existing REST APIs
- existing frontend

So this is a **very safe non-breaking fix**.

✅ After fixing, run again

Bash

✅ If server starts, next check this immediately

Open:

- `http://127.0.0.1:8000/api/attachments/graphql/`

(or whatever your attachments GraphQL URL is)

Then run:

```
query {
  attachments {
    id
    fileName
  }
}
```

If that works, attachments GraphQL is fixed.

🔥 Important: This error is ONLY for the separate attachments GraphQL endpoint

This error happened because in:

- `attachments/urls.py`

you have:

```python
```

So Django loads `attachments/graphql/schema.py` during URL import.

That's why the server crashes before starting.

✅ Exact root cause summary

Problem:

- `attachments/graphql/schema.py` includes `mutation=Mutation`
- `attachments/graphql/mutations.py` has empty `Mutation`

Fix:

- remove `mutation=Mutation` from `attachments/graphql/schema.py`

✅ Final correct file

`attachments/graphql/schema.py`

```python
import graphene
from .queries import Query
```

🚀 What to do after this

Once server starts, send me these **3 files exactly as they are now after your change**:

1. `attachments/graphql/queries.py`
2. `attachments/graphql/schema.py`
3. `leaderboard/graphql/schema.py`

Then I will give you the **next exact safe step**:

👉 merge attachments into main `/graphql/`

without field conflicts and **without breaking other team modules**

⚠️ One more important thing

Because your `attachments/urls.py` still has its own GraphQL endpoint:

```python
```

You can keep it for now, but **later**, once main `/graphql/` works, I may recommend either:

- keeping it temporarily, or
- removing it to avoid confusion

For now: **keep it**.

If you want, next I can give you the **exact corrected code for** `attachments/graphql/queries.py` (copy-paste ready, conflict-safe for team project) before you touch `leaderboard/graphql/schema.py`.

AssertionError: Found different types with the same name in the schema: ClientType, ClientType.
[29/Mar/2026 12:58:05] "POST /api HTTP/1.1" 500 29695
D:\Leaderboard_BE\Leaderboard_BE\attachments\graphql\queries.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

System check identified some issues:

WARNINGS:
?: settings.ACCOUNT_EMAIL_REQUIRED is deprecated, use: settings.ACCOUNT_SIGNUP_FIELDS = ['email*', 'username*', 'password1*', 'password2*']

System check identified 1 issue (0 silenced).
March 29, 2026 - 12:58:48
Django version 5.0, using settings 'leaderboard.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

Internal Server Error: /api
Traceback (most recent call last):
  File "C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python311\Lib\site-packages\django\core\handlers\exception.py", line 55, in inner

```
    response = get_response(request)
               ^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\django\core\handlers\base.py", line 197, in
_get_response
    response = wrapped_callback(request, *callback_args,
**callback_kwargs)

^^^^^^^^^^^^^^^^^^^
```

```
1\Lib\site-packages\graphene_django\settings.py", line 127, in
__getattr__
    val = perform_import(val, attr)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphene_django\settings.py", line 66, in
perform_import
    return import_from_string(val, setting_name)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphene_django\settings.py", line 80, in
import_from_string
    module = importlib.import_module(module_path)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\importlib\__init__.py", line 126, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

^^^^^^^^^^^^^^^^^^^
```

```
exec_module
  File "<frozen importlib._bootstrap>", line 241, in
_call_with_frames_removed
  File
"D:\Leaderboard_BE\Leaderboard_BE\leaderboard\graphql\schema.py",
line 76, in <module>
    schema = graphene.Schema(query=Query, mutation=Mutation,
subscription=Subscription)

^^^
```

```
^^^
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphql\type\definition.py", line 212, in
define_field_map
    field_map = field_map()
                ^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphene\types\typemap.py", line 275, in
construct_fields_for_type
    map = self.reducer(map, field.type)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphene\types\typemap.py", line 88, in reducer
    return self.graphene_reducer(map, type)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphene\types\typemap.py", line 93, in
graphene_reducer
    return self.reducer(map, type.of_type)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python31
1\Lib\site-packages\graphene\types\typemap.py", line 88, in reducer
    return self.graphene_reducer(map, type)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "C:\Users\SangramBhosale\AppData\Local\Programs\Python\Python311\Lib\site-packages\graphene\types\typemap.py", line 97, in graphene_reducer
assert _type.graphene_type == type, (
^^^

- `ProgramType`
- maybe later `AttachmentType` if another app uses same name

✅ CORRECT TEAM-SAFE STRATEGY (START AGAIN PROPERLY)

We will do this **the safest way**:

DO NOT create new `ClientType` or `ProgramType` in attachments app

Instead:

- Use **only attachment query**
- Return only **simple attachment fields**
- **Do NOT expose nested** `clients` / `programs` / `assignments` **yet**
- First make main `/api` GraphQL work safely
- Then after that, we connect activity

This is the safest and least likely to break team project.

🔥 DO THIS NOW — EXACT RESET + SAFE VERSION

STEP 1) Fix `attachments/graphql/schema.py`

File:

`attachments/graphql/schema.py`

Replace with:

```python
import graphene
from .queries import Query
```

✅ No mutation for now.

STEP 2) REPLACE `attachments/graphql/queries.py` COMPLETELY

Delete whatever is currently there and replace with this exact code:

File:

`attachments/graphql/queries.py`

```python
import graphene
from graphene_django.types import DjangoObjectType

from attachments.models import SecureAttachment


# IMPORTANT:
# Use a UNIQUE GraphQL type name to avoid conflict with any other app.
class SecureAttachmentType(DjangoObjectType):
    class Meta:
        model = SecureAttachment
```

```
        fields = (
            "id",
            "file_name",
            "description",
            "asset_type",
            "sharepoint_url",
            "external_url",
            "created_at",
            "updated_at",
            "is_active",
        )
```

✅ Why this version is correct

This version avoids **all dangerous duplicate names**:

- ❌ no `ClientType`
- ❌ no `ProgramType`
- ❌ no `AttachmentType` generic name
- ✅ uses `SecureAttachmentType` (unique and safe)

Also query field names are safe:

- `attachments`
- `attachmentById`

These are unlikely to conflict.

STEP 3) DO NOT TOUCH `attachments/graphql/mutations.py`

You can keep the file if it exists, but it should **NOT** be used right now.

No need to import it anywhere.

STEP 4) KEEP `attachments/urls.py` AS IS (FOR NOW)

This part can stay:

```python
```

That is okay.

STEP 5) TEST ATTACHMENTS GRAPHQL ALONE FIRST (VERY IMPORTANT)

Before touching main schema, test **only** attachments GraphQL endpoint.

Start server:

```bash
```

Then open:

- `http://127.0.0.1:8000/api/attachments/graphql/`

(or whatever your attachment URL is in your `leaderboard/urls.py`)

Run this exact query:

```
query {
  attachments {
    id
    fileName
    description
    assetType
    sharepointUrl
    externalUrl
    createdAt
    updatedAt
  }
}
```

✅ Expected result

You should get data like:

```
{
  "data": {
    "attachments": [
      {
        "id": "1",
        "fileName": "test.pdf",
        "description": "sample",
        "assetType": "file"
      }
```

```
      ]
    }
  }
```

🚨 DO NOT MERGE INTO MAIN `/api` UNTIL THIS WORKS

First make sure:

- server starts
- `/api/attachments/graphql/` works

Only then move to main GraphQL.

STEP 6) NOW SAFELY MERGE INTO MAIN GRAPHQL (`leaderboard/graphql/schema.py`)

You already have:

- `ClientQueries`
- `ProgramQueries`
- etc.

We will add **only attachment queries**.

File:

`leaderboard/graphql/schema.py`

Add this import:

Python Run

Then update your main `Query` class

Find this:

```python
class Query(
            UserQuery,
            MeQuery,
            ClientQueries,
            ProgramQueries,
            WAqueries,
            graphene.ObjectType):
    pass
```

Replace with:

```python
class Query(
            UserQuery,
            MeQuery,
            ClientQueries,
            ProgramQueries,
```

```
            AttachmentsQueries,
            WAqueries,
            graphene.ObjectType):
    pass
```

⚠️ IMPORTANT: DO NOT ADD ATTACHMENTS MUTATIONS

Do **NOT** add this:

Python Run

Do **NOT** add it to main Mutation class.

Because:

- you don't need it now
- empty mutation caused previous crash
- REST is already handling attachment create/update/delete

STEP 7) Restart and test MAIN GraphQL

Run:

Bash

Then open your main GraphQL endpoint:

- probably `http://127.0.0.1:8000/api`
  or whatever your GraphQLView path is

Run this exact query:

```
query {
  attachments {
    id
    fileName
    assetType
    sharepointUrl
    externalUrl
  }
}
```

✅ If this works, attachments are safely connected to main GraphQL

At this point:

- REST upload still works
- main GraphQL can fetch attachments
- nothing in team schema should break

🔥 VERY IMPORTANT: WHAT NOT TO DO NOW

For now **DO NOT** do these things yet:

❌ Do NOT define these in attachments GraphQL:

- `ClientType`
- `ProgramType`
- `AttachmentType` (generic name)

❌ Do NOT expose nested fields yet:

- `clients`
- `programs`
- `assignments`

Because those can create:

- duplicate type conflicts
- circular import issues
- schema instability in team project

✅ NEXT SAFE STEP (After main GraphQL works)

Once `attachments` query works in main `/api`, then we connect `Activity` to attachments.

That requires:

- add `attachments = models.ManyToManyField(...)` in `Activity`
- expose `attachments` on `ProgramActivityType`
- add `attachment_ids` to `createActivity`

But **not before main query works cleanly**.

🔥 WHY YOUR MAIN `/api` CRASHED

Because your main schema imports all app schemas into one GraphQL type map.

When you added another:

- `ClientType`
- `ProgramType`

inside `attachments/graphql/queries.py`

Graphene saw:

- `client.graphql...ClientType`
- `attachments.graphql...ClientType`

and crashed.

So the fix is **not a random patch** — the fix is:

Use unique GraphQL type names only

For attachments use:

- `SecureAttachmentType`

and nothing else for now.

✅ CLEAN SAFE FILES YOU SHOULD HAVE RIGHT NOW

`attachments/graphql/schema.py`

```python
import graphene
from .queries import Query
```

`attachments/graphql/queries.py`

```python
import graphene
from graphene_django.types import DjangoObjectType

from attachments.models import SecureAttachment


class SecureAttachmentType(DjangoObjectType):
    class Meta:
        model = SecureAttachment
        fields = (
            "id",
            "file_name",
            "description",
            "asset_type",
```

```
            "sharepoint_url",
            "external_url",
            "created_at",
            "updated_at",
            "is_active",
        )
```

`leaderboard/graphql/schema.py`

Only add:

```python
class Query(
            UserQuery,
            MeQuery,
            ClientQueries,
            ProgramQueries,
            AttachmentsQueries,
            WAqueries,
            graphene.ObjectType):
    pass
```

No mutation changes.

✅ TEST ORDER (DO EXACTLY THIS)

1. Save `attachments/graphql/queries.py`
2. Save `attachments/graphql/schema.py`
3. Run server
4. Test `/api/attachments/graphql/`
5. If working, then edit `leaderboard/graphql/schema.py`
6. Run server again
7. Test main `/api`

🚨 If you still get error after doing EXACTLY this

Then the likely issue is:

- you still have old code in `attachments/graphql/queries.py`
- or `leaderboard/graphql/schema.py` still imports `AttachmentsMutations`
- or another type named `AttachmentType` already exists somewhere

If that happens, send me **exact current contents only** of these 3 files:

1. `attachments/graphql/queries.py`
2. `attachments/graphql/schema.py`
3. `leaderboard/graphql/schema.py`

and I'll give you the **exact line-by-line correction**.

✅ Best next move

Do the above **exactly** first.

After that, I'll give you the **safe Step 2**:

connect `Activity` ↔ `SecureAttachment`

with **copy-paste ready code** for:

- `program/models.py`
- `program/graphql/types.py`
- `program/graphql/Activity/mutations.py`

That part I can give you cleanly once this schema conflict is gone.

If you want, after you test this, reply with just:

> **attachments query working in main api now**

and I'll give you the **next exact activity integration steps** with no extra confusion.

thanks above steps are working just correct both for /api and /api/attachment/graphql
from django.db import models
import json
import re

```
from django.contrib.auth.models import User
from client.models import Client  # Adjust import path as needed
from django.contrib.auth import get_user_model
import requests
from leaderboard.auth import get_access_token
from leaderboard.settings import GraphQL_Playground
from django.db.models import Q
import requests
from django.core.validators import MinValueValidator,
MaxValueValidator
from django.core.exceptions import ValidationError
from typing import Dict, Any, List
from django.core.cache import cache
from .formulas import validate_formula, eval_formula,
build_invitees_all_done_map, build_360_maps
from datetime import timedelta, datetime, time
from django.utils import timezone
import uuid
from program.cities import cities
from attachments.models import SecureAttachment

User = get_user_model()

import base64

LEVEL_CHOICES = [
    ('session', 'Session'),
    ('program', 'Program'),
    ('topic', 'Topic'),
```

```
    ]

SESSION_TYPE_CHOICES = [
    ('onsite', 'Onsite'),
    ('online', 'Online'),
    ('hybrid', 'Hybrid'),
]

MAPP_TOOL_CHOICES = [
    # Module-level (returns all sub-metrics)
    ('soul', 'Soul (All)'),
    ('idp', 'IDP (All)'),
    ('okr', 'OKR (All)'),
    ('readiness', 'Readiness Zone (All)'),
    ('feedback', 'Feedback (All)'),
    ('coaching', 'Coaching Conversation (All)'),
    ('kudos', 'Kudos (All)'),
    ('meeting', 'Meeting (All)'),

    # Sub-metric level (returns single count)
    # Soul
    ('soul_values', 'Soul - Values'),
    ('soul_strength', 'Soul - Strength'),
    ('soul_personality', 'Soul - Personality'),
    ('soul_knowledge', 'Soul - Knowledge'),
    ('soul_impact_narrative', 'Soul - Impact Narrative'),

    # IDP
    ('idp_total', 'IDP - Total'),
```

```
    ('idp_completed', 'IDP - Completed'),
    ('idp_inprogress', 'IDP - In Progress'),
    ('idp_pending', 'IDP - Pending'),
    ('idp_revise', 'IDP - Revise'),
    ('idp_draft', 'IDP - Draft'),

    # OKR
    ('okr_complete', 'OKR Complete'),
    # ('okr_owner', 'OKR - Owner'),
    # ('okr_collabrator', 'OKR - Collaborator'),
    # ('okr_assigned', 'OKR - Assigned'),

    # Readiness
    ('readiness_given', 'Readiness - Given'),
    ('readiness_received', 'Readiness - Received'),

    # Feedback
    ('feedback_given', 'Feedback - Given'),
    ('feedback_received', 'Feedback - Received'),

    # Coaching
    # ('coaching_coachee', 'Coaching - Coachee'),
    ('coaching_coaching', 'Coaching - Completed'),
    # ('coaching_incomplete_coachee', 'Coaching - Incomplete
Coachee'),
    # ('coaching_incomplete_coaching', 'Coaching - Incomplete
Coaching'),

    # Kudos/Badges
```

```
    ('kudos_issued', 'Kudos - Issued'),
    ('kudos_received', 'Kudos - Received'),

    # Meetings
    # ('meetings_total', '1:1 Meeting - Total'),
    ('meetings_done', 'Meeting Done'),
    # ('meetings_upcoming', '1:1 Meeting - Upcoming'),
]

MAPP_TOOL_LABEL_MAP = dict(MAPP_TOOL_CHOICES) #adding this
just to take column name

ACTIVITY_CHOICES = [("pre_activity", "pre_activity"), ("post_activity",
"post_activity")]
ACTIVITY_TYPE_CHOICES = [
        ('quiz', 'Quiz'),
        ('audio', 'Audio'),
        ('video', 'Video'),
        ('survey', 'Survey'),
        ('assignment', 'Assignment'),
        ('mapp_tool', 'MApp Tool')
    ]

def from_global_id(global_id):
    """Decode GraphQL Global ID → numeric ID"""
    try:
        decoded = base64.b64decode(global_id).decode('utf-8')
        return int(decoded.split(':')[1])
    except:
```

```
        return None

class ActivityTemplate(models.Model):
    mapp_survey=models.IntegerField(null=True, blank=True)
    mapp_survey_name = models.CharField(max_length=255,
blank=True, null=True,verbose_name="MApp Survey Name")
    mapp_tool = models.CharField(max_length=50,
choices=MAPP_TOOL_CHOICES,null=True, blank=True)
    type = models.CharField(max_length=20,
choices=ACTIVITY_TYPE_CHOICES)
    description = models.TextField(null=True, blank=True)

    is_360 = models.BooleanField(null=True, blank=True)
    name = models.CharField(max_length=200)
    is_feedback = models.BooleanField(default=False)
    learning_resource = models.ForeignKey(SecureAttachment,
on_delete=models.CASCADE, null=True, blank=True)

    def __str__(self):
        return self.name


class Activity(models.Model):
    mapp_survey=models.IntegerField(null=True, blank=True)
    mapp_survey_name = models.CharField(max_length=255,
blank=True, null=True,verbose_name="MApp Survey Name")
    mapp_tool = models.CharField(max_length=50,
choices=MAPP_TOOL_CHOICES,null=True, blank=True)
    type = models.CharField(max_length=20,
```

```
choices=ACTIVITY_TYPE_CHOICES)
    description = models.TextField(null=True, blank=True)
    deadline = models.DateTimeField(null=True, blank=True)
    is_360 = models.BooleanField(null=True, blank=True)
    name = models.CharField(max_length=200)
    is_feedback = models.BooleanField(default=False)
    template = models.ForeignKey(
            ActivityTemplate,
            on_delete=models.PROTECT,
            null=True,
            blank=True,
            related_name="activities"
        )
    learning_resource = models.ForeignKey(SecureAttachment,
on_delete=models.CASCADE, null=True, blank=True)

    def __str__(self):
        return self.name
        # return f"{self.get_type_display()} Activity"

    def create_survey_token_in_mapp(user_data, survey_id, invite=False,
message=None):
        try:
            access_token = get_access_token()

            mutation = '''
                mutation createTokenLeaderboard(
                $first_name: String!,
                $last_name: String!,
```

```
            $email: String,
            $mobileNumber: String,
            $responseSourceFor_id: Int,
            $respondentType: String,
            $survey_id: Int!,
            $invite: Boolean,
            $message: String,
            ) {
            createTokenLeaderboard(
                firstName: $first_name,
                lastName: $last_name,
                email: $email,
                mobileNumber: $mobileNumber,
                responseSourceForId: $responseSourceFor_id,
                respondentType: $respondentType,
                surveyId: $survey_id,
                invite: $invite,
                message: $message
            ) {
                token
            }
            }
            '''

        variables = {
            "first_name": user_data.get("first_name"),
            "last_name": user_data.get("last_name"),
            "email": user_data.get("email"),
            "mobileNumber": user_data.get("mobileNumber"),
```

```
            "survey_id": survey_id,
            "invite": invite,
            "message": message,
            "responseSourceFor_id":
user_data.get("responseSourceFor_id"),
            "respondentType": user_data.get("respondentType","self"),
        }


        headers = {
            "Authorization": f"Bearer {access_token}",
            "Content-Type": "application/json",
        }

        response = requests.post(
            GraphQL_Playground,
            json={"query": mutation, "variables": variables},
            headers=headers,
        )
        response.raise_for_status()

        result = response.json()

        if errors := result.get("errors"):
            raise Exception(f"Error from MApp API: {errors}")

        token_data = result["data"]["createTokenLeaderboard"]["token"]
    except Exception as e:
        print (e)
```

```
    return token_data
def fetch_all_surveys_leaderboard():
    """
    Fetch all surveys leaderboard data from MApp GraphQL API
    Returns: List of {id, name} or None on error
    """
    try:
        access_token = get_access_token()

        query = '''
            query {
                allSurveysLeaderboard {
                    id
                    name
                }
            }
        '''

        headers = {
            "Authorization": f"Bearer {access_token}",
            "Content-Type": "application/json",
        }

        response = requests.post(
            GraphQL_Playground,
            json={"query": query},
            headers=headers,
        )
        response.raise_for_status()
```

```
        result = response.json()

        if errors := result.get("errors"):
            raise Exception(f"Error from MApp API: {errors}")

        surveys = result["data"]["allSurveysLeaderboard"]
        survey_list = []

        for survey in surveys:
            survey_id = from_global_id(survey["id"])
            if survey_id:
                survey_list.append({
                    "id": survey_id,
                    "name": survey["name"]
                })


        return survey_list

    except Exception as e:
        print(f"Error fetching surveys: {e}")
        return None

def get_cached_surveys():
    """Return cached dict OR fetch + cache"""
    cache_key = 'mapp_surveys_dict'
    survey_dict = cache.get(cache_key)
```

```
        if not survey_dict:
            survey_dict = Activity.fetch_all_surveys_leaderboard()
            cache.set(cache_key, survey_dict, 300)

        return survey_dict

    def fetch_mapp_tool_used_date(userEmail,
moduleName,dateStart,dateEnd ):
        access_token = get_access_token()

        query = '''
            query ModuleUsage($userEmail: String!, $moduleName: String!,
$dateStart: DateTime, $dateEnd: DateTime) {
                moduleUsage(
                    userEmail: $userEmail
                    moduleName: $moduleName
                    dateStart: $dateStart
                    dateEnd: $dateEnd
                )
            }
        '''
        variables = {
            'userEmail': userEmail,
            'moduleName': moduleName,
            'dateStart': datetime.combine(dateStart, time.min).isoformat() +
'Z' if dateStart else None,
            'dateEnd': datetime.combine(dateEnd, time.max).isoformat() +
'Z' if dateEnd else None
        }
```

```
        headers = {
            "Authorization": f"Bearer {access_token}",
            "Content-Type": "application/json",
        }

        response = requests.post(
            GraphQL_Playground,
            json={"query": query, "variables": variables},
            headers=headers
        )

        response.raise_for_status()

        result = response.json()
        if "errors" in result:
            raise Exception(f"GraphQL errors: {result['errors']}")

        responses = result.get("data", {}).get("moduleUsage", {})

        return responses

class Program(models.Model):

    JOURNEY_MODE_CHOICES = [
        ('open_all',   'Everything open immediately'),
        ('date_based', 'Unlock by calendar dates'),
        ('sequential', 'Step-by-step (wait for previous)'),
    ]
```

```
    VISIBILITY_SCOPE_CHOICES = [
        ('all',             'Apply rules to sessions, topics and activities'),
        ('activities_only', 'Apply rules only to activities'),
    ]
    STATUS_CHOICES = [
        ('new', 'New'),
        ('in_progress', 'In Progress'),
        ('inactive', 'Inactive'),
        ('completed', 'Completed'),
    ]

    name = models.CharField(max_length=255)
    creator = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='created_programs')
    client = models.ForeignKey(Client, on_delete=models.CASCADE,
related_name='programs')
    start_date = models.DateTimeField()
    end_date = models.DateTimeField()
    status = models.CharField(max_length=20,
choices=STATUS_CHOICES, default='new')
    max_participants = models.PositiveIntegerField()
    pre_activity = models.ManyToManyField(Activity, blank=True,
related_name='pre_program')
    post_activity = models.ManyToManyField(Activity, blank=True,
related_name='post_program')
    #account_manager =
models.ManyToManyField(User,related_name='account_admin',
blank=True)
```

```
    certificate_template_id = models.CharField(max_length=255,
null=True, blank=True)
    content_unlock_mode = models.CharField(
        max_length=20,
        choices=JOURNEY_MODE_CHOICES,
        default='open_all',
        help_text='How the program journey unlocks
(open_all/date_based/sequential).'
    )

    unlock_scope = models.CharField(
        max_length=20,
        choices=VISIBILITY_SCOPE_CHOICES,
        default='activities_only',
        help_text='Where to apply journey rules: all nodes or only
activities.'
    )
    program_uuid = models.UUIDField(unique=True, null=True,
blank=True)
    description = models.TextField()
    def __str__(self):
        return self.name

class UserProgramPreference(models.Model):
    user = models.OneToOneField(User,
on_delete=models.CASCADE,db_column='employee_id',to_field='id')
    default_program =
models.ForeignKey(Program,on_delete=models.CASCADE,db_column='
default_program_id',to_field='id',null=True,blank=True)
```

```
    client =  models.ForeignKey(Client,
on_delete=models.CASCADE,db_column='client_id',to_field='id',
null=True,blank=True)
    role = models.CharField(max_length=50, null=True, blank=True)
    role_program_json = models.JSONField(null=True, blank=True)
class Topic(models.Model):
    name = models.CharField(max_length=255)
    creator = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='created_topics')
    pre_activity = models.ManyToManyField(Activity, blank=True,
related_name='pre_topics')
    post_activity = models.ManyToManyField(Activity, blank=True,
related_name='post_topics')
    start_date = models.DateTimeField()
    end_date = models.DateTimeField()
    description = models.TextField()

    def __str__(self):
        return self.name

class Session(models.Model):

    SESSION_STATUS_CHOICES = [
        ('planned', 'Planned'),
        ('in_progress', 'In Progress'),
        ('inactive', 'Inactive'),
        ('cancelled', 'Cancelled'),
        ('rescheduled', 'Rescheduled'),
        ('completed', 'Completed'),
```

```
    ]

    program = models.ForeignKey(Program,
on_delete=models.CASCADE, related_name='sessions')
    topic = models.ManyToManyField(Topic,
related_name='topic_sessions',blank=True)
    start_date = models.DateTimeField()
    end_date = models.DateTimeField()
    location = models.CharField(
        max_length=100,
        choices=cities,
        blank=True
    )
    name = models.CharField(max_length=255)
    delivery_mode = models.CharField(max_length=100,
choices=SESSION_TYPE_CHOICES)
    pre_activity = models.ManyToManyField(Activity,  blank=True,
related_name='pre_session')
    post_activity = models.ManyToManyField(Activity,  blank=True,
related_name='post_session')
    session_uuid = models.UUIDField(unique=True, null=True,
blank=True)
    status = models.CharField(max_length=20,
choices=SESSION_STATUS_CHOICES, default='planned')
    description = models.TextField()
    def __str__(self):
        return f"{self.name} on {self.start_date}"

class ParticipantActivityStatus(models.Model):
```

```
    ACTIVITY_STATUS_CHOICES = [
        ('not_started', 'Not Started'),
        ('in_progress', 'In Progress'),
        ('completed', 'Completed'),
    ]
    participant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='activity_progress')
    activity = models.ForeignKey(Activity, on_delete=models.CASCADE,
related_name='user_progress')
    status = models.CharField(max_length=20,
choices=ACTIVITY_STATUS_CHOICES, default='not_started')
    completion_date = models.DateTimeField(null=True, blank=True)

    def __str__(self):
        return f"{self.participant.username} - {self.activity} - {'Completed' if
self.status == 'Completed' else 'In Progress'}"

class ParticipantProgramEnrollment(models.Model):
    participant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='program_enrollments')
    program = models.ForeignKey(Program,
on_delete=models.CASCADE, related_name='enrolled_employees')
    enrollment_date = models.DateField(auto_now_add=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["participant", "program"],
                name="uniq_participant_program",
```

```
            )
        ]
    def __str__(self):
        return f"{self.participant.username} enrolled in
{self.program.name}"

class ParticipantReplacement(models.Model):
    """Store all replacement history - acts as archive for replaced
participants"""

    PARTICIPANT_REPLACEMENT_STATUS_CHOICES = [
        ('replace', 'Replace'),
        ('unenroll', 'Unenroll'),
        ('remove','Remove') #this choice will be just for UI purpose and
will not save this into this model (in future if required we can do this)
    ]

    program = models.ForeignKey(Program,
on_delete=models.CASCADE,
related_name='participant_replacements')
    old_participant = models.ForeignKey(User,
on_delete=models.CASCADE, related_name='replaced_from_programs')
    new_participant = models.ForeignKey(User,
on_delete=models.CASCADE, related_name='replaced_into_programs',
null=True,blank=True)
    action = models.CharField(max_length=20,
choices=PARTICIPANT_REPLACEMENT_STATUS_CHOICES)
    old_enrollment_date = models.DateField(help_text="When old
participant originally enrolled")
```

```
    replacement_date = models.DateTimeField(auto_now_add=True)

    reason = models.TextField()

    replaced_by = models.ForeignKey(User,
on_delete=models.SET_NULL, null=True,
related_name='replacements_made')

    class Meta:
        ordering = ['-replacement_date']
        indexes = [
            models.Index(fields=['program', 'replacement_date']),
            models.Index(fields=['old_participant', 'program']),
            models.Index(fields=['new_participant', 'program']),
        ]

    def __str__(self):
        return f"{self.old_participant} → {self.new_participant} in
{self.program.name}"

class ParticipantSessionAttendance(models.Model):
    ATTENDANCE_STATUS_CHOICES = [
        ('present', 'Present'),
        ('absent', 'Absent'),
        ('excused', 'Excused'),
    ]
    participant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='session_attendance')
    session = models.ForeignKey(Session, on_delete=models.CASCADE,
```

```
related_name='attendees')
    status = models.CharField(max_length=20,
choices=ATTENDANCE_STATUS_CHOICES)
    attendance_date = models.DateField(auto_now_add=True)
    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=['participant', 'session'],
                name='unique_participant_per_session',
            )
        ]
    def __str__(self):
        return f"{self.participant.username} - {self.session} - {self.status}"

class ParticipantProgramAttendance(models.Model):
    ATTENDANCE_STATUS_CHOICES = [
        ('present', 'Present'),
        ('absent', 'Absent'),
        ('excused', 'Excused'),
    ]
    participant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='program_attendance')
    program = models.ForeignKey(Program,
on_delete=models.CASCADE, related_name='attendees')
    status = models.CharField(max_length=20,
choices=ATTENDANCE_STATUS_CHOICES)
    attendance_date = models.DateField(auto_now_add=True)
    class Meta:
        constraints = [
```

```
            models.UniqueConstraint(
                fields=['participant', 'program'],
                name='unique_participant_per_program',
            )
        ]
    def __str__(self):
        return f"{self.participant.username} - {self.program} - {self.status}"

class ParticipantActivityResponse(models.Model):
    STATUS_CHOICES = [("Pending", "Pending"), ("Complete",
"Complete"), ("Expired", "Expired")]

    participant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='activity_responses')
    activity = models.ForeignKey(Activity, on_delete=models.CASCADE,
related_name='responses')
    response_data = models.JSONField(null=True, blank=True)
    submission_date = models.DateTimeField(auto_now_add=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    status =
models.CharField(max_length=20,choices=STATUS_CHOICES,null=True,
blank=True)
    last_polled_at = models.DateTimeField(null=True, blank=True)
    score = models.FloatField(null=True, blank=True)
    topic =
models.ForeignKey(Topic,null=True,blank=True,on_delete=models.CASC
ADE,related_name="activity_responses",)
    session =
```

```
models.ForeignKey(Session,null=True,blank=True,on_delete=models.CA
SCADE,related_name="activity_responses",)
    program =
models.ForeignKey(Program,null=True,blank=True,on_delete=models.C
ASCADE,related_name="activity_responses",)
    activty_status =
models.CharField(max_length=20,choices=ACTIVITY_CHOICES,null=Tru
e, blank=True)
    token_link = models.CharField(max_length=200,null=True,
blank=True)

    class Meta:
        constraints = [
            # Existing constraint (topic-specific)
            models.UniqueConstraint(
                fields=['participant', 'activity', 'topic', 'activty_status'],
                name='unique_participant_activity_topic'
            ),
            # NEW: Exactly ONE context (topic OR session OR program)
            models.CheckConstraint(
                check=(
                    # Topic only
                    (Q(topic__isnull=False) & Q(session__isnull=True) &
Q(program__isnull=True)) |
                    # Session only
                    (Q(topic__isnull=True) & Q(session__isnull=False) &
Q(program__isnull=True)) |
                    # Program only
                    (Q(topic__isnull=True) & Q(session__isnull=True) &
```

```
Q(program__isnull=False))
            ),
            name="exactly_one_context",
        ),
        # NEW: Session uniqueness (add similar for program if needed)
        models.UniqueConstraint(
            fields=['participant', 'activity', 'session', 'activty_status'],
            name='unique_participant_activity_session',
            condition=Q(session__isnull=False)
        ),
        models.UniqueConstraint(
            fields=['participant', 'activity', 'program', 'activty_status'],
            name='unique_participant_activity_program',
            condition=Q(program__isnull=False)
        ),
    ]
  #assignment =
models.ForeignKey(ActivityAssignment,null=True,blank=True,on_delete
=models.CASCADE,related_name="responses",)
  # class Meta:
  #     constraints = [
  #         models.CheckConstraint(
  #             check=(
  #                 ~(
  #                     models.Q(topic__isnull=False) &
  #                     models.Q(assignment__isnull=False)
  #                 )
  #             ),
  #             name="no_both_topic_and_assignment",
```

```
    #         ),
    #     ]
    def __str__(self):
        return f"{self.participant.username} - {self.activity} Response"

    def fetch_mapp_survey_responses_on_refresh(user_email, survey_ids,
response_date_gte=None,query_called = False):
        query = '''
        query fetchLeaderboardAnswersMap($userEmail: String!,
$surveyIds: [ID!]!, $responseDateGte: DateTime) {
            fetchLeaderboardAnswersMap(
                userEmail: $userEmail
                surveyIds: $surveyIds
                responseDateGte: $responseDateGte
            )
            }
        '''
        variables = {
            "userEmail": user_email,
            "surveyIds": [survey_ids],
            "responseDateGte": response_date_gte.isoformat() if
response_date_gte else None
        }

        # Get OAuth token
        access_token = get_access_token()

        headers = {
            "Authorization": f"Bearer {access_token}",
```

```
            "Content-Type": "application/json",
        }

        response = requests.post(
            GraphQL_Playground,
            json={"query": query, "variables": variables},
            headers=headers
        )

        response.raise_for_status()

        result = response.json()
        if "errors" in result:
            raise Exception(f"GraphQL errors: {result['errors']}")

        responses = result.get("data",
{}).get("fetchLeaderboardAnswersMap", {})

        return responses

    def fetch_mapp_survey_responses(user_email, survey_ids,
response_date_gte=None,query_called = False):
        query = '''
        query fetchLeaderboardAnswersMap($userEmail: String!,
$surveyIds: [ID!]!, $responseDateGte: DateTime) {
            fetchLeaderboardAnswersMap(
                userEmail: $userEmail
                surveyIds: $surveyIds
                responseDateGte: $responseDateGte
```

```
            )
            }
        '''
        variables = {
            "userEmail": user_email,
            "surveyIds": [survey_ids],
            "responseDateGte": response_date_gte.isoformat() if
response_date_gte else None
        }

        # Get OAuth token
        access_token = get_access_token()

        headers = {
            "Authorization": f"Bearer {access_token}",
            "Content-Type": "application/json",
        }

        response = requests.post(
            GraphQL_Playground,
            json={"query": query, "variables": variables},
            headers=headers
        )

        response.raise_for_status()

        result = response.json()
        if "errors" in result:
            raise Exception(f"GraphQL errors: {result['errors']}")
```

```
        responses = result.get("data",
{}).get("fetchLeaderboardAnswersMap", {})

        return responses

    def fetch_survey_status_for_360(createdBy_Email,
survey_id,responseSourceFor):
        query = '''
        query fetchTokens($responseSourceFor: ID!) {
            getAllTokensListLeaderboard(responseSourceFor:
$responseSourceFor) {
                user {
                id
                username
                }
                respondentType
                valid
            }
            }

        '''
        variables = {
            "responseSourceFor":responseSourceFor
        }
        access_token = get_access_token()

        headers = {
            "Authorization": f"Bearer {access_token}",
```

```
            "Content-Type": "application/json",
        }

        response = requests.post(
            GraphQL_Playground,
            json={"query": query, "variables": variables},
            headers=headers
        )
        response.raise_for_status()
        result = response.json()

        if "errors" in result:
            raise Exception(f"GraphQL errors: {result['errors']}")

        status_data = []

        tokens = result["data"]["getAllTokensListLeaderboard"]

        for token in tokens:
            username = token["user"]["username"]
            status = "pending" if token["valid"] else "completed"
            status_data.append({
                "username": username,
                "status": status
            })

        return status_data

class ScorecardConfiguration(models.Model):
```

```
    program = models.ForeignKey(Program,
on_delete=models.CASCADE, related_name='scorecards')
    name = models.CharField(max_length=255)
    is_draft = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def total_percentage(self):
        total = self.scorecard_activities.aggregate(
            total=models.Sum('weight_percentage')
        )['total'] or 0
        return total

    def update_is_draft(self):
        total = self.total_percentage()
        self.is_draft = (int(total) != 100)
        self.save(update_fields=['is_draft'])

    def _get_linked_item_dates(sca):
        """
        Returns (start_date, end_date) of the linked item
(session/topic/program).
        """
        if sca.session:
            return sca.session.start_date, sca.session.end_date
        elif sca.topic:
            return sca.topic.start_date, sca.topic.end_date
        elif sca.program:
            return sca.program.start_date, sca.program.end_date
```

```
        return None, None


    def _is_future_scorecard_activity(sca):
        """
        Determines if a scorecard activity should show 'NA' because its
        linked event is still in the future.

        Rules:
        - Attendance:    NA if now < linked_item.end_date
        - Pre activity:  NA if now < linked_item.start_date - 1 day
        - Post activity: NA if now < linked_item.end_date
        """
        now = timezone.now()
        start_date, end_date =
ScorecardConfiguration._get_linked_item_dates(sca)

        if start_date is None and end_date is None:
            # No linked item dates found — treat as available (not future)
            return False

        if sca.is_attendance:
            # Attendance: only score after the linked item has ended
            return now < end_date

        # Activity (including mapp_tool)
        activity_status = (sca.activity_status or "").strip().lower()

        if activity_status == "pre":
```

```
            # Pre-activity: open 1 day before linked item starts
            return now < (start_date - timedelta(days=1))
        else:
            # Post-activity: open after linked item ends
            return now < end_date

    def build_leaderboard_dict(program_id: int) -> Dict[str, Any]:
        try:
            program = Program.objects.get(id=program_id)
            scorecard = program.scorecards.filter(is_draft=False).order_by("-
created_at").first()
            if scorecard is None:
                return {"columns": [], "rows": [],'success_rate':0}
            columns = [
                {"key": "rank", "label": "Rank", "type": "number", "width": 80},
                {"key": "name", "label": "Name", "type": "string"},
                {"key": "email", "label": "Email", "type": "string"},
            ]

            scorecard_activities = list(
                scorecard.scorecard_activities
                .select_related("activity", "session", "program", "topic")
            )

            # Build columns and pre-compute which ones are future (NA)
            activity_columns = []
            future_flags = {}  # key -> True if future (NA)

            for sca in scorecard_activities:
```

```
            if sca.is_attendance:
                if sca.session:
                    key = f"att_{sca.session.name}"
                    label = f"Att: {sca.session.name}
({sca.weight_percentage}%)"
                elif sca.program:
                    key = f"att_program"
                    label = f"Program Att ({sca.weight_percentage}%)"
                else:
                    continue
            else:
                status_suffix = f"_{sca.activity_status}" if sca.activity_status
else ""
                if sca.activity.mapp_tool:
                    key = f"mapp_{sca.activity.mapp_tool}"
                    label = f"MApp
{MAPP_TOOL_LABEL_MAP[sca.activity.mapp_tool].title()}
({sca.weight_percentage}%)"
                else:
                    key = f"act_{sca.activity.name}{status_suffix}"
                    label = f"{sca.activity.name} {sca.activity_status or ''}
({sca.weight_percentage}%)"

            is_future =
ScorecardConfiguration._is_future_scorecard_activity(sca)
            future_flags[key] = is_future

            # Use "mixed" type if this column might contain "NA" strings
            col_type = "mixed" if is_future else "number"
```

```
            activity_columns.append({"key": key, "label": label, "type":
col_type})

        active_columns = [col for col in activity_columns if not
future_flags.get(col["key"], False)]
        na_columns = [col for col in activity_columns if
future_flags.get(col["key"], False)]
        activity_columns = active_columns + na_columns

        columns.extend(activity_columns)
        columns.append({"key": "total_score", "label": "Total", "type":
"number"})

        enrollments =
ParticipantProgramEnrollment.objects.filter(program=program).select_r
elated("participant")
        participants = [e.participant for e in enrollments]
        participant_ids = [p.id for p in participants]

        if not participants:
            return {"columns": columns, "rows": [],'success_rate':0}

        # Attendance maps
        session_att_qs = ParticipantSessionAttendance.objects.filter(
            participant_id__in=participant_ids
        ).select_related('session')
        program_att_qs = ParticipantProgramAttendance.objects.filter(
            participant_id__in=participant_ids,
            program=program
```

```
        ).select_related('program')

        attendance_map = {}
        for att in session_att_qs:
            attendance_map[('session', att.participant_id, att.session_id)]
= att.status

        for att in program_att_qs:
            attendance_map[('program', att.participant_id, program.id)] =
att.status

        # Activities
        activity_resp_qs = ParticipantActivityResponse.objects.filter(
            participant_id__in=participant_ids
        ).select_related("activity", "session", "topic", "program")
        activity_score_map = {}
        for r in activity_resp_qs:
            if r.status == 'Complete' and r.score is not None:
                activity_score_map[(r.participant_id, r.activity_id)] =
float(r.score)

        invitees_all_done_map, invite_ratio_map =
build_360_maps(scorecard_activities, participants)

        # MApp tool data map
        mapp_data_map = {}

        mapp_modules = set(
            sca.activity.mapp_tool for sca in scorecard_activities
```

```
            if sca.activity and sca.activity.mapp_tool
        )
        for p in participants:
            p_email = p.email
            for module in mapp_modules:
                try:
                    data = Activity.fetch_mapp_tool_used_date(
                        userEmail=p_email,
                        dateStart=program.start_date,
                        dateEnd=program.end_date,
                        moduleName=module
                    )
                    mapp_data_map[(p.id, module)] = data or {}
                except Exception as e:
                    mapp_data_map[(p.id, module)] = {}

        # Process rows
        temp_rows = []
        for p in participants:
            row = {
                "name": p.get_full_name() or p.username,
                "email": p.email,
            }
            total = 0

            p_attendance_map = {
                key: status for key, status in attendance_map.items() if
key[1] == p.id
            }
```

```
            for sca in scorecard_activities:
                # Generate key (consistent with columns)
                if sca.is_attendance:
                    if sca.session:
                        key = f"att_{sca.session.name}"
                        status_key = ('session', p.id, sca.session_id)
                        status = p_attendance_map.get(status_key)
                    elif sca.program:
                        key = f"att_program"
                        status_key = ('program', p.id, program_id)
                        status = p_attendance_map.get(status_key)
                    else:
                        continue
                else:
                    status_suffix = f"_{sca.activity_status}" if
sca.activity_status else ""
                    if sca.activity.mapp_tool:
                        key = f"mapp_{sca.activity.mapp_tool}"
                    else:
                        key = f"act_{sca.activity.name}{status_suffix}"
                    status = None

                # ── CHECK IF FUTURE → assign "NA" and skip scoring
──
                if future_flags.get(key, False):
                    row[key] = "NA"
                    continue
```

```
                base = float(sca.weight_percentage)
                ctx = {"base": base}

                if sca.is_attendance:
                    ctx.update({
                        "present": status == "present",
                        "excused": status == "excused",
                        "absent": status == "absent",
                        "status": status or "unknown",
                    })
                else:
                    raw_score = activity_score_map.get((p.id, sca.activity_id),
0.0)

                    activity_filter = {
                        "participant_id": p.id,
                        "activity": sca.activity,
                        "activty_status": sca.activity_status
                    }
                    if sca.program:
                        activity_filter["program"] = sca.program
                    if sca.session:
                        activity_filter["session"] = sca.session
                    if sca.topic:
                        activity_filter["topic"] = sca.topic

                    completed =
ParticipantActivityResponse.objects.filter(**activity_filter).filter(
                        status="Complete"
```

```
                ).exists()

                all_invitees_done = invitees_all_done_map.get((p.id,
sca.activity_id), False)
                invite_ratio = invite_ratio_map.get((p.id, sca.activity_id),
0.0)

                if sca.activity.mapp_tool:
                    raw_data = mapp_data_map.get((p.id,
sca.activity.mapp_tool), {})
                    mapp_data = json.loads(raw_data) if
isinstance(raw_data, str) else raw_data
                    ctx.update(mapp_data)

                ctx.update({
                    "raw_score": raw_score,
                    "completed": completed,
                    "all_invitees_done": all_invitees_done,
                    "invite_ratio": invite_ratio,
                    "present": status == "present",
                    "excused": status == "excused",
                    "on_time": True,
                    "days_late": 0,
                })

            if sca.formula:
                if sca.activity and sca.activity.mapp_tool and
len(mapp_data) < 1:
                    cell_score = 0
```

```
                    else:
                        cell_score = eval_formula(sca.formula, ctx)
                else:
                    if sca.is_attendance:
                        cell_score = base if ctx["present"] else 0.0
                    else:
                        cell_score = (raw_score / 100.0 * base) if
ctx["completed"] else 0.0

                row[key] = round(cell_score, 2)
                total += cell_score

            row["total_score"] = round(total, 2)
            temp_rows.append(row)

        # Sort + rank (stable ties by name)
        temp_rows.sort(key=lambda r: (-r["total_score"],
r["name"].lower()))
        rows = []
        last_score = None
        current_rank = 0
        for row in temp_rows:
            score = row["total_score"]
            if score != last_score:
                current_rank += 1
                last_score = score
            row["rank"] = current_rank
            rows.append(row)
```

```
        except Exception as e:
            print(e)
            return {"columns": [], "rows": [],'success_rate':0}

        # Swap email column with total
        columns[next(i for i, c in enumerate(columns) if c["key"] ==
"total_score")], \
        columns[next(i for i, c in enumerate(columns) if c["key"] ==
"email")] = \
        columns[next(i for i, c in enumerate(columns) if c["key"] ==
"email")], \
        columns[next(i for i, c in enumerate(columns) if c["key"] ==
"total_score")]



        if rows:
            sum_of_scores = sum(row["total_score"] for row in rows)
            max_possible = len(rows) * 100.0  # each can get up to 100
            success_rate = (sum_of_scores / max_possible) * 100.0
        else:
            success_rate = 0

        return {
            "columns": columns,
            "rows": rows,
            "success_rate": round(success_rate, 2),
        }

    def __str__(self):
```

```
        return f"Scorecard Configuration for {self.program.name}"

class ScorecardActivity(models.Model):
    scorecard = models.ForeignKey(ScorecardConfiguration,
on_delete=models.CASCADE,related_name='scorecard_activities')

    # Level (program/session/topic)
    level = models.CharField(max_length=20, choices=LEVEL_CHOICES)

    # Target object
    program = models.ForeignKey(Program, null=True, blank=True,
on_delete=models.PROTECT)
    session = models.ForeignKey(Session, null=True, blank=True,
on_delete=models.PROTECT)
    topic = models.ForeignKey(Topic, null=True, blank=True,
on_delete=models.PROTECT)

    # ACTIVITY component (optional)
    activity = models.ForeignKey(Activity, null=True, blank=True,
on_delete=models.PROTECT,related_name = "scorecard_activities")
    activity_status =
models.CharField(max_length=20,choices=ACTIVITY_CHOICES,null=Tru
e, blank=True)
    is_attendance = models.BooleanField(default=False,
        help_text="True=Attendance scoring, False=Activity scoring (if
activity set)")

    weight_percentage = models.DecimalField(max_digits=5,
decimal_places=2)
```

```
    # Formula can reference both activity AND attendance variables
    formula = models.TextField(
        null=True, blank=True,
        help_text=(
            "Activity vars: base, raw_score, present, completed\n"
            "Attendance vars: attendance_points, participation_points,
engagement_score"
        )
    )

    # class Meta:
    #     constraints = [
    #         models.UniqueConstraint(
    #             fields=["scorecard", "activity"],
    #             name="uniq_scorecard_activity",
    #             condition=Q(activity__isnull=False),
    #         ),
    #         models.UniqueConstraint(
    #             fields=["scorecard", "session"],
    #             name="uniq_scorecard_session",
    #             condition=Q(session__isnull=False),
    #         ),
    #     ]

    def clean(self):
        has_activity = self.activity is not None
        has_session = self.session is not None
        # if has_activity == has_session:
```

```
        #     raise ValidationError(
        #         "Exactly one of 'activity' or 'session' must be set."
        #     )
        if self.activity and not self.activity.mapp_tool:
            validate_formula(self.formula)


    def __str__(self):
        if self.activity:
            name = self.activity.name
        elif self.session:
            name = self.session.name
        else:
            name = "Unbound item"
        return f"{name} ({self.weight_percentage}%)"


class PaymentNotificationRule(models.Model):

    class TargetType(models.TextChoices):
        PROGRAM = 'program', 'Program'
        SESSION = 'session', 'Session'
        TOPIC   = 'topic',   'Topic'

    class When(models.TextChoices):
        BEFORE_START    = 'before_start',   'Before Start'
        AFTER_START     = 'after_start',     'After Start'
        BEFORE_END      = 'before_end',      'Before End'
        AFTER_END       = 'after_end',       'After End'
```

```
        ON_COMPLETION  = 'on_completion',   'On Completion'

    program = models.ForeignKey(
        Program, on_delete=models.CASCADE,
        related_name='payment_notification_rules'
    )

    target_type = models.CharField(max_length=20,
choices=TargetType.choices)
    target_id = models.PositiveIntegerField(
        null=True, blank=True,
        help_text='NULL when target_type is "program".'
    )

    when = models.CharField(
        max_length=20, choices=When.choices,
        help_text='Single dropdown for AM: before_start, after_end, etc.'
    )
    offset = models.CharField(
        max_length=20, default='0d',
        help_text='Human-friendly: "7d", "12h", "30m", "0d". Parsed to
seconds in BE.'
    )

    created_by = models.ForeignKey(
        User, on_delete=models.SET_NULL, null=True,
        related_name='created_payment_notification_rules'
    )
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    updated_at = models.DateTimeField(auto_now=True)

    context = models.TextField(null=True, blank=True,help_text
="Context to be included in the finance reminder emails." )


    def parse_offset(value: str) -> timedelta:
        """
        Converts AM-friendly offset string to timedelta.

        '7d'  → 7 days
        '12h' → 12 hours
        '30m' → 30 minutes
        '0d'  → zero
        """
        value = value.strip().lower()
        if value in ('0', '0d', '0h', '0m'):
            return timedelta(0)

        match = re.fullmatch(r'(\d+)\s*([dhm])', value)
        if not match:
            raise ValueError(f'Invalid offset format: "{value}". Use e.g. "7d",
"12h", "30m".')

        amount, unit = int(match.group(1)), match.group(2)

        if unit == 'd':
            return timedelta(days=amount)
        elif unit == 'h':
```

```
            return timedelta(hours=amount)
        elif unit == 'm':
            return timedelta(minutes=amount)

    class Meta:
        db_table = 'payment_notification_rule'
        indexes = [
            models.Index(fields=['program',]),
            models.Index(fields=['target_type', 'target_id']),
        ]

    def clean(self):
        from django.core.exceptions import ValidationError

        if self.target_type == self.TargetType.PROGRAM and self.target_id
is not None:
            raise ValidationError('target_id must be NULL when target_type
is "program".')
        if self.target_type in (self.TargetType.SESSION,
self.TargetType.TOPIC) and not self.target_id:
            raise ValidationError('target_id is required for session/topic
rules.')
        if self.target_type == self.TargetType.TOPIC and self.when ==
self.When.ON_COMPLETION:
            raise ValidationError('Topic has no status; "on_completion" is not
supported.')

class PaymentNotificationRun(models.Model):
    class State(models.TextChoices):
```

```
        SCHEDULED = 'scheduled', 'Scheduled'
        SENT      = 'sent',      'Sent'
        SKIPPED   = 'skipped',   'Skipped'
        FAILED    = 'failed',    'Failed'
        PENDING    = 'pending',  'Pending'

    rule = models.ForeignKey(
        PaymentNotificationRule, on_delete=models.CASCADE,
        related_name='runs'
    )

    fire_at = models.DateTimeField(
        help_text='Computed from rule.when ± parsed offset.'
    )

    state = models.CharField(
        max_length=20, choices=State.choices, default=State.SCHEDULED
    )
    sent_at = models.DateTimeField(null=True, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def compute_fire_at(rule, target_obj):
        offset = PaymentNotificationRule.parse_offset(rule.offset)

        anchor_map = {
            'before_start':  ('start_date', -1),
            'after_start':   ('start_date',  1),
```

```
            'before_end':    ('end_date',   -1),
            'after_end':     ('end_date',    1),
        }

        if rule.when == 'on_completion':
            # event-driven: fire_at = now() + offset at the moment status →
completed
            from django.utils import timezone
            return timezone.now() + offset

        field_name, direction = anchor_map[rule.when]
        anchor_dt = getattr(target_obj, field_name)
        return anchor_dt + (offset * direction)

    class Meta:
        db_table = 'payment_notification_run'
        indexes = [
            models.Index(fields=['state', 'fire_at']),
        ]

class ReminderStatus(models.TextChoices):
    PENDING = "pending", "Pending"          # Not yet picked up by
scheduler
    SCHEDULED = "scheduled", "Scheduled"    # Countdown task
dispatched
    SENT = "sent", "Sent"                   # Successfully delivered
    FAILED = "failed", "Failed"
class Reminder(models.Model):
    """
```

```
    One row per reminder to send. For standalone programs, links to
    the program directly. For multi-session, links to the session.
    """
    program = models.ForeignKey(
        Program, on_delete=models.CASCADE, related_name="reminders"
    )
    session = models.ForeignKey(
        Session, on_delete=models.CASCADE,
        null=True, blank=True, related_name="reminders"
    )
    target_time = models.DateTimeField(
        help_text="When the reminder should fire (start_time - 1 hour)"
    )
    status = models.CharField(
        max_length=20,
        choices=ReminderStatus.choices,
        default=ReminderStatus.PENDING,
        db_index=True,
    )
    celery_task_id = models.CharField(max_length=255, null=True,
blank=True)
    scheduled_at = models.DateTimeField(null=True, blank=True)
    sent_at = models.DateTimeField(null=True, blank=True)



    def reschedule_program(program, new_start_time, session = None):
        """
        When an admin changes a program or session time,
        reset affected reminders back to 'pending' so the
```

```
        scheduler picks them up with the corrected countdown.
        """
        Reminder.objects.filter(
            program=program, session=session
        ).exclude(status="sent").update(
            target_time=new_start_time - timedelta(hours=1),
            status="pending",
            celery_task_id=None,
        )

    class Meta:
        indexes = [
            models.Index(fields=["status", "target_time"]),
        ]
        # Prevent duplicate reminders for the same event
        constraints = [
            models.UniqueConstraint(
                fields=["program", "session"],
                name="unique_reminder_per_event",
            )
        ]

    def __str__(self):
        label = self.session or self.program
        return f"Reminder for {label} @ {self.target_time}"

class Review(models.Model):

    feedback_response = models.JSONField()
```

```
    additional_feedback = models.TextField(blank=True, null=True)
    submitted_by = models.ForeignKey(User,
on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    program = models.ForeignKey(Program,
on_delete=models.CASCADE,null=True, blank=True)
    session = models.ForeignKey(Session, on_delete=models.CASCADE,
null=True, blank=True)
    topic = models.ForeignKey(Topic, on_delete=models.CASCADE, null
= True,blank=True)

    def __str__(self):
        return f"Review by {self.submitted_by.username} - submited at:
{self.created_at}"

    def rating(self, program_id,session_id):
        from program.feedback_survey_questions import feedback_form
        rating_map = {
            "Very dissatisfied": 1,
            "Dissatisfied": 2,
            "Neither satisfied nor dissatisfied": 3,
            "Satisfied": 4,
            "Very satisfied": 5,
        }

        reviews = Review.objects.all()
        rating_question_ids = {
            q["id"] for q in feedback_form[0]['questions'] if q.get("rating")
        }
```

```
        if program_id:
            reviews = reviews.filter(program_id=program_id)

        if session_id:
            reviews = reviews.filter(session_id=session_id)

        total_weighted_score = 0
        total_responses = 0
        total_score = 0
        for review in reviews:
            responses = review.feedback_response or []

            for qid, answer in responses.items():

                if qid in rating_question_ids and answer in rating_map:
                    total_score += rating_map[answer]
                    total_responses += 1


        avg_rating = (
            total_score / total_responses if total_responses else 0
        )

        return round(avg_rating, 2)
this is my program models.py
import graphene
import django_filters
from ..models import *
```

```
from client.models import Client
from graphene import relay
from django.db.models import Q
from graphene_django.types import DjangoObjectType
from leaderboard.customConnections.extendedConnections import
ExtendedConnection
from django.contrib.auth import get_user_model
from graphene_django.filter import DjangoFilterConnectionField
from django.core.exceptions import ObjectDoesNotExist
import json
from ..journey_helpers import *
from ..journey_builder import compute_visibility
User = get_user_model()

class ProgramActivityType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    reason = graphene.String()
    total_responses = graphene.Int()
    class Meta:
        model = Activity
        interfaces = (relay.Node,)
        filter_fields = {
            "type": ["exact"],
            "description": ["exact", "icontains", "istartswith"],
        }
        connection_class = ExtendedConnection
    def resolve_visible(self, info):
        return compute_activity_vis(self, info.context.user)["accessible"]
```

```
    def resolve_accessible(self, info):
        return compute_activity_vis(self, info.context.user)["accessible"]
    def resolve_reason(self, info):
        return getattr(self, '_journey_reason', None)

    def resolve_total_responses(self, info):
        return ParticipantActivityResponse.objects.filter(activity = self,
status= "Complete").count()

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        activity_ids = Activity.objects.filter(
            Q(pre_program__enrolled_employees__participant=user) |
            Q(post_program__enrolled_employees__participant=user) |

Q(pre_session__program__enrolled_employees__participant=user) |

Q(post_session__program__enrolled_employees__participant=user) |

Q(pre_topics__topic_sessions__program__enrolled_employees__participa
nt=user) |

Q(post_topics__topic_sessions__program__enrolled_employees__particip
ant=user)
```

```
            ).distinct().values_list('id', flat=True)

        return queryset.filter(id__in=activity_ids).distinct()

class CustomUserFilterSet(django_filters.FilterSet):
    class Meta:
        model = User
        fields = {
            "id": ["exact"],
            "username": ["exact", "icontains", "istartswith"],
            "email": ["exact", "icontains"],
            "first_name": ["exact", "icontains", "istartswith"],
            "last_name": ["exact", "icontains", "istartswith"],
        }

class UserType(DjangoObjectType):
    class Meta:
        model = User
        fields = "__all__"
        filterset_class = CustomUserFilterSet
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection

class ActivityWithStageType(graphene.ObjectType):
    activity = graphene.Field(lambda: ProgramActivityType)
    stage = graphene.String()


class ProgramType(DjangoObjectType):
```

```
    activities = graphene.List(ActivityWithStageType)

    # Expose ManyToMany as a connection field
    account_manager = DjangoFilterConnectionField(UserType)
    pre_activities = graphene.List(ProgramActivityType)
    post_activities = graphene.List(ProgramActivityType)

    class Meta:
        model = Program
        fields = "__all__"
        interfaces = (relay.Node,)
        filter_fields = {
            "name": ["exact", "icontains", "istartswith"],
            "creator__id": ["exact"],
            "start_date": ["exact", "lt", "gt"],
            "end_date": ["exact", "lt", "gt"],
            "status": ["exact"],
            "client__id": ["exact"],
        }
        connection_class = ExtendedConnection

    def resolve_account_manager(self, info, **kwargs):
        return self.client.admins.filter(is_staff=True)

    def resolve_activities(self, info):
        sessions = self.sessions.all()
        topics = set()
        for session in sessions:
            for t in session.topic.all():
```

```
            topics.add(t)

        seen = set()
        results = []

        for topic in topics:
            for pre_act in topic.pre_activity.all():
                if pre_act.id not in seen:
                    results.append(
                        ActivityWithStageType(activity=pre_act, stage="Pre")
                    )
                    seen.add(pre_act.id)

            for post_act in topic.post_activity.all():
                if post_act.id not in seen:
                    results.append(
                        ActivityWithStageType(activity=post_act, stage="Post")
                    )
                    seen.add(post_act.id)

        return results
    def resolve_sessions(self, info):
        """Override sessions resolution to inject context"""
        sessions = self.sessions.all().order_by('start_date')

        # Inject program context
        for session in sessions:
            session._journey_program = self
```

```
        return sessions
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        return queryset.filter(Q(creator=user) |
Q(enrolled_employees__participant=user) | Q(client__id__in =
info.context.user.client_admins.all())).distinct()

class SessionType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()

    pre_activities = graphene.List(ProgramActivityType)
    post_activities = graphene.List(ProgramActivityType)
    rating = graphene.Int()
    class Meta:
        model = Session
        interfaces = (relay.Node,)
        filter_fields = {
            "program__id": ["exact"],
            "topic__id": ["exact"],
            "start_date": ["exact", "lt", "gt"],
            "end_date": ["exact", "lt", "gt"],
            "location": ["exact", "icontains", "istartswith"],
            "status": ["exact"],
        }
```

```
        connection_class = ExtendedConnection

    def resolve_visible(self, info):
        return compute_session_vis(self, info.context.user)["accessible"]

    def resolve_accessible(self, info):
        return compute_session_vis(self, info.context.user)["accessible"]
    def resolve_rating(self, info):
        return Review().rating(program_id=None,session_id=self.id)

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset

        return
queryset.filter(Q(program__enrolled_employees__participant=user) |
Q(attendees__participant=user)).distinct()

class ProgramActivityTemplateType(DjangoObjectType):
    class Meta:
        model = ActivityTemplate
        interfaces = (relay.Node,)
        filter_fields = {
            "type": ["exact"],
            "description": ["exact", "icontains", "istartswith"],
            "is_feedback":["exact"]
        }
```

```
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return None



class TopicType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    pre_activities = graphene.List(ProgramActivityType)
    post_activities = graphene.List(ProgramActivityType)
    class Meta:
        model = Topic
        interfaces = (relay.Node,)
        filter_fields = {
            "name": ["exact", "icontains", "istartswith"],
            "creator__id": ["exact"],
        }
        connection_class = ExtendedConnection
    def resolve_visible(self, info):
        return compute_topic_vis(self, info.context.user)["accessible"]

    def resolve_accessible(self, info):
        return compute_topic_vis(self, info.context.user)["accessible"]
```

```
preActivityResponses = DjangoFilterConnectionField(
    'program.graphql.types.ParticipantActivityResponseType'
)
postActivityResponses = DjangoFilterConnectionField(
    'program.graphql.types.ParticipantActivityResponseType'
)

def resolve_preActivityResponses(self, info, **kwargs):
    return ParticipantActivityResponse.objects.filter(
        topic=self,
        activty_status="pre_activity",
        participant=info.context.user
    )

def resolve_postActivityResponses(self, info, **kwargs):
    return ParticipantActivityResponse.objects.filter(
        topic=self,
        activty_status="post_activity",
        participant=info.context.user
    )

@classmethod
def get_queryset(cls, queryset, info):
    user = info.context.user
    if user.is_staff or user.client_admins.exists():
        return queryset

    enrolled_programs =
```

```
Program.objects.filter(enrolled_employees__participant=user)

        sessions = Session.objects.filter(program__in=enrolled_programs)

        topics_in_sessions =
Topic.objects.filter(topic_sessions__in=sessions).distinct()

        return queryset.filter(Q(id__in=topics_in_sessions.values_list('id',
flat=True)) | Q(creator=user)).distinct()
class ParticipantActivityStatusType(DjangoObjectType):
    class Meta:
        model = ParticipantActivityStatus
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "activity__id": ["exact"],
            "status": ["exact"],
            "completion_date": ["exact", "lt", "gt"],
        }
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)
```

```
class ParticipantProgramEnrollmentType(DjangoObjectType):
    attendance_percentage = graphene.Float()

    class Meta:
        model = ParticipantProgramEnrollment
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "program__id": ["exact"],
            "enrollment_date": ["exact", "lt", "gt"],
        }
        connection_class = ExtendedConnection

    def resolve_attendance_percentage(self, info):
        participant = self.participant
        program = self.program
        total_sessions = Session.objects.filter(program=program).count()
        attended_sessions = ParticipantSessionAttendance.objects.filter(
            participant=participant, session__program=program,
status="present"
        ).count()
        if total_sessions == 0:
            return 0
        return round((attended_sessions / total_sessions) * 100, 2)

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
```

```
            return queryset
        return queryset.filter(participant=user)

class ParticipantReplacementType(DjangoObjectType):

    class Meta:
        model = ParticipantReplacement
        interfaces = (relay.Node,)
        filter_fields = {
            "old_participant__id": ["exact"],
            "new_participant__id": ["exact"],
            "program__id": ["exact"],
            "old_enrollment_date": ["exact", "lt", "gt"],
            "replacement_date": ["exact", "lt", "gt"],
            "action":["exact"]
        }
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)
class ParticipantSessionAttendanceType(DjangoObjectType):
    class Meta:
        model = ParticipantSessionAttendance
        fields = ('id', 'status', 'participant', 'session','attendance_date')
        # participant = graphene.Field(UserType)
```

```
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "session__id": ["exact"],
            "status": ["exact"],
            "attendance_date": ["exact", "lt", "gt"],

        }

        connection_class = ExtendedConnection
        # def resolve_participant(self, info):
        #     return self.participant  # Your ForeignKey to User
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class ParticipantProgramAttendanceType(DjangoObjectType):
    class Meta:
        model = ParticipantProgramAttendance
        fields = ('id', 'status', 'participant', 'program','attendance_date')
        # participant = graphene.Field(UserType)
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "program__id": ["exact"],
            "status": ["exact"],
```

```
            "attendance_date": ["exact", "lt", "gt"],

        }

        connection_class = ExtendedConnection
        # def resolve_participant(self, info):
        #     return self.participant  # Your ForeignKey to User
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class ParticipantActivityResponseType(DjangoObjectType):
    class Meta:
        model = ParticipantActivityResponse
        interfaces = (relay.Node,)
        filter_fields = {
            "participant__id": ["exact"],
            "activity__id": ["exact"],
            "topic__id":["exact"],
            "session__id":["exact"],
            "program__id":["exact"],
            "submission_date": ["exact", "lt", "gt"],
        }
        connection_class = ExtendedConnection

    fetch = graphene.String()
```

```
    def resolve_fetch(self, info):
        if self.status != "Complete" and
Activity.objects.get(id=self.activity_id).mapp_survey:
            user_email = self.participant.email
            survey_ids = Activity.objects.get(
                id=self.activity_id
            ).mapp_survey
            response_date_gte = self.created_at

            if survey_ids:
                responses = (

ParticipantActivityResponse.fetch_mapp_survey_responses_on_refresh(
                        user_email, survey_ids, response_date_gte,
query_called=True
                    )
                )
                if isinstance(responses, str):
                    responses = json.loads(responses)
                if responses and len(responses) > 0:
                    self.response_data = responses
                    self.status = "Complete"
                    self.save()

        return self.status
    @classmethod
    def get_queryset(cls, queryset, info):
        user = info.context.user
```

```
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)

class UserProgramPreferenceType(DjangoObjectType):
    class Meta:
        model=UserProgramPreference
        fields=(
            "default_program",
            "user",
            "client",
            "role",
            "role_program_json"
        )
        filter_fields = {
            "default_program": ['exact'],
            "user": ['exact'],
            "client":['exact'],
            "role":['exact']
        }
        interfaces = (relay.Node, )
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        if not info.context.user.is_staff:
            return queryset.filter(Q(user = info.context.user))
        else:
            return queryset
```

```
class ScorecardConfigurationType(DjangoObjectType):
    class Meta:
        model = ScorecardConfiguration

        filter_fields = {
            "program__id": ["exact"],
            "name": ["exact", "icontains", "istartswith"],
            "is_draft": ["exact"],
            "created_at": ["exact", "lt", "gt"],
            "updated_at": ["exact", "lt", "gt"],
            "program__client__id": ["exact"],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection

    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        if not info.context.user.is_staff:
            return queryset.filter(
                    Q(program__client__in=info.context.user.client_admins.all())
|

Q(program__enrolled_employees__participant=info.context.user)
                ).distinct()
        else:
            return queryset

class ScorecardActivityType(DjangoObjectType):
```

```
    class Meta:
        model = ScorecardActivity

        filter_fields = {
            "scorecard__id": ["exact"],
            "activity__id": ["exact"],
            "session__id":["exact"],
            "topic__id":["exact"],
            "program__id":["exact"],
            "weight_percentage": ["exact", "lt", "gt"],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        if not info.context.user.is_staff:
            return queryset.filter(

Q(scorecard__program__client__id__in=info.context.user.client_admins.al
l()) |

Q(scorecard__program__enrolled_employees__participant=info.context.
user)
                ).distinct()
        else:
            return queryset


class PaymentNotificationRuleType(DjangoObjectType):
```

```
    class Meta:
        model = PaymentNotificationRule
        fields = '__all__'
        filter_fields = {
            'program__id': ['exact'],
            'target_type': ['exact'],
            'when': ['exact'],
            'created_by__id': ['exact'],
            'created_at': ['gte', 'lte'],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)



class PaymentNotificationRunType(DjangoObjectType):
    class Meta:
        model = PaymentNotificationRun
        fields = '__all__'
        filter_fields = {
            'rule__id': ['exact'],
            'rule__program__id': ['exact'],
            'state': ['exact'],
```

```
            'fire_at': ['gte', 'lte'],
            'sent_at': ['gte', 'lte'],
            'created_at': ['gte', 'lte'],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
            return queryset
        return queryset.filter(participant=user)
class ReviewType(DjangoObjectType):
    class Meta:
        model = Review
        fields = '__all__'
        filter_fields = {
            'additional_feedback': ['exact'],
            'submitted_by__id': ['exact'],
            'program__id': ['exact'],
            'session__id': ['exact'],
            'created_at': ['gte', 'lte'],
        }
        interfaces = (relay.Node,)
        connection_class = ExtendedConnection
    @classmethod
    def get_queryset(cls, queryset, info, **kwargs):
        user = info.context.user
        if user.is_staff or user.client_admins.exists():
```

```
            return queryset
        return queryset.filter(participant=user)
this is progrm/graphql/types.py
from leaderboard.decorators import graphql_permission_required
import graphene
from django.utils.dateparse import parse_datetime
from django.contrib.auth import get_user_model
from ...models import Program, Activity,ACTIVITY_TYPE_CHOICES
from ..types import ProgramActivityType
from leaderboard.events import send_subscription_and_notification
from django.db.models import Q
from django.db.models import Prefetch

User = get_user_model()

class CreateActivity(graphene.Mutation):
    class Arguments:
        type = graphene.String(required=True)
        name = graphene.String(required = True)
        template_id = graphene.Int()
        description = graphene.String()
        deadline = graphene.String()
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        mapp_tool = graphene.String()
        is_feedback = graphene.Boolean()
        learning_resource_id = graphene.Int()
```

```
    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, type,
name,template_id=None,description=None, deadline=None,
mapp_survey=None, is_360=None,mapp_survey_name = None,
is_feedback=False,mapp_tool = None,learning_resource_id = None):
        try:
            deadline_parsed = parse_datetime(deadline) if deadline else
None

            activity = Activity(
                type=type,
                name = name,
                template_id = template_id,
                description=description if description else '',
                deadline=deadline_parsed,
                mapp_survey=mapp_survey,
                is_360=is_360,
                mapp_survey_name = mapp_survey_name,
                is_feedback = is_feedback,
                mapp_tool = mapp_tool,
                learning_resource_id = learning_resource_id
            )
            activity.full_clean()
            activity.save()
```

```
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "CREATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return CreateActivity(activity=activity, success=True,
message="Activity created successfully")
        except Program.DoesNotExist:
            return CreateActivity(activity=None, success=False,
message="Program not found.")
        except Exception as e:
            return CreateActivity(activity=None, success=False,
message=str(e))


class UpdateActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
```

```
        template_id = graphene.Int()
        name = graphene.String()
        type = graphene.String()
        description = graphene.String()
        deadline = graphene.String()  # ISO 8601 datetime string
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        is_feedback = graphene.Boolean()
        mapp_tool = graphene.String()
        learning_resource_id = graphene.Int()

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id, template_id=None,name=None,
type=None, description=None, deadline=None, mapp_survey=None,
is_360=None,mapp_survey_name = None,
is_feedback=None,mapp_tool = None,learning_resource_id = None):
        try:
            activity = Activity.objects.get(pk=id)

            # Handle type with choices validation
            if type is not None:
                activity.type = type
```

```
        # Handle deadline parsing
        if deadline is not None:
            deadline_parsed = parse_datetime(deadline)
            if not deadline_parsed:
                return UpdateActivity(activity=None, success=False,
message="Invalid deadline format.")
            activity.deadline = deadline_parsed

        # Update simple fields (None will set NULL)
        if name is not None:
            activity.name = name
        if description is not None:
            activity.description = description
        activity.mapp_survey = mapp_survey
        activity.mapp_survey_name = mapp_survey_name
        activity.mapp_tool = mapp_tool
        activity.is_360 = is_360
        if template_id:
            activity.template_id = template_id

        if is_feedback is not None:
            activity.is_feedback = is_feedback
        if learning_resource_id is not None:
            activity.learning_resource_id = learning_resource_id
        activity.full_clean()
        activity.save()
        send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
```

```
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": activity,
                "mutation": "UPDATE",
            },
            # notification_event_type="survey_token_created",
            # notification_payload={
            #     "object": token,
            #     "user_id": user.id,
            # },
        )

        participants = User.objects.filter(
            program_enrollments__program__in=Program.objects.filter(
                Q(pre_activity=activity) |
                Q(post_activity=activity) |
                Q(sessions__pre_activity=activity) |
                Q(sessions__post_activity=activity) |
                Q(sessions__topic__pre_activity=activity) |
                Q(sessions__topic__post_activity=activity)
            )
        ).distinct()

        for participant in participants:
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
```

```
                group=participant.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "UPDATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
        return UpdateActivity(activity=activity, success=True,
message="Activity updated successfully")
    except Activity.DoesNotExist:
        return UpdateActivity(activity=None, success=False,
message="Activity not found.")
    except Exception as e:
        return UpdateActivity(activity=None, success=False,
message=str(e))


class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
```

```
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity.delete()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return DeleteActivity(success=True, message="Activity deleted
successfully")
        except Activity.DoesNotExist:
            return DeleteActivity(success=False, message="Activity not
found")


class Mutation(graphene.ObjectType):
    create_activity = CreateActivity.Field()
```

```
    update_activity = UpdateActivity.Field()
    delete_activity = DeleteActivity.Field()

"""
Example:
mutation {
  createActivity(
    type: "quiz"
    description: "A quiz for the program"
    deadline: "2025-12-10T17:00:00Z"
    mappSurvey: 123
    is360: true
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360

    }
  }
}

mutation {
```

```
  updateActivity(
    id: "5"
    type: "audio"
    description: "Updated description"
    deadline: "2025-12-15T10:30:00Z"
    mappSurvey: 456
    is360: false
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360
    }
  }
}

mutation {
  deleteActivity(id: "5") {
    success
    message
```

i need many to many field make changes while creating activity i should be uld be able to add multiple files

```
  }
```

```
}
"""
```

this is program/graphql/Activity/mutations.py

Perfect — now we are at the **correct stage**.
And because your `Activity` model already has:

Python Run

that supports **only one file**.

You want:

> while creating activity, I should be able to add **multiple files**

✅ So now the **correct solution** is:

- **keep** `learning_resource` **as-is** (for backward compatibility, team-safe)
- **add a NEW ManyToMany field** on `Activity`
- expose it in GraphQL
- update `createActivity` and `updateActivity` to accept multiple attachment IDs

This is the **safest team-project approach** because it does **not break old code** that may still use `learning_resource`.

✅ FINAL SAFE PLAN (DO THIS EXACTLY)

We will change only these files:

1. `program/models.py`
2. `program/graphql/types.py`
3. `program/graphql/Activity/mutations.py`

And then run:

4. `makemigrations`
5. `migrate`

🚨 IMPORTANT TEAM-SAFE RULE

❌ Do NOT remove this existing field:

```python
learning_resource = models.ForeignKey(SecureAttachment, on_delete=models.CASCADE, null=True, blan
```

Python Run

Keep it.

Because:

- old frontend/backend may still use it
- templates may still depend on it
- removing it can break existing logic

✅ Add a new field:

Python Run

This gives you **multiple files** without breaking anything.

STEP 1) UPDATE `program/models.py`

File:

`program/models.py`

You already have import:

Python Run

✅ Good. No change needed there.

Find your `Activity` model

Currently it has:

```
learning_resource = models.ForeignKey(SecureAttachment, on_delete=models.CASCADE, null=True, blan
```

Python Run

Add this NEW field immediately below `learning_resource`

Replace this block:

```
learning_resource = models.ForeignKey(SecureAttachment, on_delete=models.CASCADE, null=True, blan
```

Python Run

With this exact block:

```python
learning_resource = models.ForeignKey(
    SecureAttachment,
    on_delete=models.CASCADE,
    null=True,
    blank=True
)

# NEW: supports multiple files on one activity
attachments = models.ManyToManyField(
    SecureAttachment,
    blank=True,
    related_name="activity_attachments"
)
```

✅ Final exact section in `Activity` should look like this

```
class Activity(models.Model):
    mapp_survey=models.IntegerField(null=True, blank=True)
    mapp_survey_name = models.CharField(max_length=255, blank=True, null=True,verbose_name="MAp
    mapp_tool = models.CharField(max_length=50, choices=MAPP_TOOL_CHOICES,null=True, blank=True
    type = models.CharField(max_length=20, choices=ACTIVITY_TYPE_CHOICES)
    description = models.TextField(null=True, blank=True)
    deadline = models.DateTimeField(null=True, blank=True)
    is_360 = models.BooleanField(null=True, blank=True)
    name = models.CharField(max_length=200)
    is_feedback = models.BooleanField(default=False)
    template = models.ForeignKey(
            ActivityTemplate,
            on_delete=models.PROTECT,
            null=True,
            blank=True,
            related_name="activities"
        )
    learning_resource = models.ForeignKey(
        SecureAttachment,
        on_delete=models.CASCADE,
        null=True,
        blank=True
```

STEP 2) CREATE MIGRATION

Run exactly:

Bash

You should get something like:

- `program/migrations/00xx_activity_attachments.py`

Then run:

Bash

✅ HOW TO VERIFY MIGRATION WORKED

Run:

Bash

Then:

```python
from program.models import Activity
```

Python Run

Expected output:

Python Run

Python Run

STEP 3) UPDATE `program/graphql/types.py`

Now we need to expose multiple attachments in GraphQL **without causing type conflicts**.

Since we already learned type-name collisions can happen, we will use a **unique local GraphQL type name**.

File:

`program/graphql/types.py`

At the top, add this import:

Add below existing imports:

Python Run

So near top it should include:

```python
from ..models import *
from client.models import Client
```

Python Run

Now create a NEW unique GraphQL type

Add this class above `ProgramActivityType` (important)

```
class ActivityAttachmentFileType(DjangoObjectType):
    class Meta:
        model = SecureAttachment
        fields = (
            "id",
            "file_name",
            "description",
            "asset_type",
            "sharepoint_url",
            "external_url",
            "created_at",
            "updated_at",
            "is_active",
        )
```

⚠️ Use this exact unique name:

- `ActivityAttachmentFileType`

This avoids schema conflicts.

Now update `ProgramActivityType`

CURRENT:

```python
class ProgramActivityType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    reason = graphene.String()
    total_responses = graphene.Int()
```

CHANGE TO:

```python
class ProgramActivityType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    reason = graphene.String()
    total_responses = graphene.Int()
    attachments = graphene.List(ActivityAttachmentFileType)
```

Now add resolver for attachments

Inside `ProgramActivityType`, add this method:

```python
def resolve_attachments(self, info):
```

Python Run

✅ Final `ProgramActivityType` beginning should look like:

```
class ActivityAttachmentFileType(DjangoObjectType):
    class Meta:
        model = SecureAttachment
        fields = (
            "id",
            "file_name",
            "description",
            "asset_type",
            "sharepoint_url",
            "external_url",
            "created_at",
            "updated_at",
            "is_active",
        )


class ProgramActivityType(DjangoObjectType):
    visible = graphene.Boolean()
    accessible = graphene.Boolean()
    reason = graphene.String()
    total_responses = graphene.Int()
    attachments = graphene.List(ActivityAttachmentFileType)
```

STEP 4) UPDATE `program/graphql/Activity/mutations.py`

Now we make `createActivity` and `updateActivity` accept **multiple attachment IDs**.

File:

`program/graphql/Activity/mutations.py`

4A) Add import

At top, add:

```python
```

Python Run

So imports become:

```python
from ...models import Program, Activity,ACTIVITY_TYPE_CHOICES
```

Python Run

STEP 5) UPDATE `CreateActivity`

In `CreateActivity.Arguments`, add this new field:

```python
```

Python Run

So `Arguments` becomes:

```
class Arguments:
    type = graphene.String(required=True)
    name = graphene.String(required = True)
    template_id = graphene.Int()
    description = graphene.String()
    deadline = graphene.String()
    mapp_survey = graphene.Int()
    is_360 = graphene.Boolean()
    mapp_survey_name = graphene.String()
    mapp_tool = graphene.String()
    is_feedback = graphene.Boolean()
    learning_resource_id = graphene.Int()
    attachment_ids = graphene.List(graphene.Int)
```

Now update `mutate()` signature

CURRENT:

```
def mutate(root, info, type, name, template_id=None, description=None, deadline=None, mapp_survey=N
```

Python Run

REPLACE WITH:

```
def mutate(
    root,
    info,
    type,
    name,
    template_id=None,
    description=None,
    deadline=None,
    mapp_survey=None,
    is_360=None,
    mapp_survey_name=None,
    is_feedback=False,
    mapp_tool=None,
    learning_resource_id=None,
    attachment_ids=None,
):
```

After `activity.save()`, add attachment set logic

Find this:

```
activity.full_clean()
```

‹/› Python Run

Replace with:

```
activity.full_clean()
activity.save()

# NEW: attach multiple files
if attachment_ids:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

✅ Final important part of `CreateActivity` should be:

```
activity.full_clean()
activity.save()

# NEW: attach multiple files
if attachment_ids:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

STEP 6) UPDATE `UpdateActivity`

We also need update support so you can later edit activity and replace/clear files.

In `UpdateActivity.Arguments`, add:

Python Run

So it becomes:

```python
class Arguments:
    id = graphene.ID(required=True)
    template_id = graphene.Int()
    name = graphene.String()
    type = graphene.String()
    description = graphene.String()
    deadline = graphene.String()
    mapp_survey = graphene.Int()
    is_360 = graphene.Boolean()
    mapp_survey_name = graphene.String()
    is_feedback = graphene.Boolean()
    mapp_tool = graphene.String()
    learning_resource_id = graphene.Int()
    attachment_ids = graphene.List(graphene.Int)
```

Update `mutate()` signature

CURRENT:

```
def mutate(root, info, id, template_id=None, name=None, type=None, description=None, deadline=None
```

Python Run

REPLACE WITH:

```
def mutate(
    root,
    info,
    id,
    template_id=None,
    name=None,
    type=None,
    description=None,
    deadline=None,
    mapp_survey=None,
    is_360=None,
    mapp_survey_name=None,
    is_feedback=None,
    mapp_tool=None,
    learning_resource_id=None,
    attachment_ids=None,
):
```

After `activity.save()`, add this:

Find this:

```python
activity.full_clean()
```

Python Run

Replace with:

```python
activity.full_clean()
activity.save()

# NEW: update multiple files
# If frontend sends [] => clears all
# If frontend omits field => leaves unchanged
if attachment_ids is not None:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

STEP 7) RESTART SERVER

Bash

STEP 8) TEST CREATE ACTIVITY WITH MULTIPLE FILES

Go to main GraphQL endpoint (`/api` probably) and run this:

```
mutation {
  createActivity(
    type: "assignment"
    name: "Activity With Multiple Files"
    description: "Testing many to many attachments"
    attachmentIds: [1, 2]
  ) {
    success
    message
    activity {
      id
      name
      attachments {
        id
        fileName
        assetType
        sharepointUrl
        externalUrl
      }
    }
  }
}
```

✅ Expected response

```
{
  "data": {
    "createActivity": {
      "success": true,
      "message": "Activity created successfully",
      "activity": {
        "id": "123",
        "name": "Activity With Multiple Files",
        "attachments": [
          {
            "id": "1",
            "fileName": "file1.pdf"
          },
          {
            "id": "2",
            "fileName": "video-link"
          }
        ]
      }
    }
  }
}
```

STEP 9) TEST `activityById`

Your `activityById` already returns `ProgramActivityType`, so now test:

```
query {
  activityById(id: 123) {
    id
    name
    attachments {
      id
      fileName
      assetType
      sharepointUrl
      externalUrl
    }
  }
}
```

If attachments come back → integration is complete.

STEP 10) TEST UPDATE ACTIVITY WITH MULTIPLE FILES

Replace files:

```
mutation {
  updateActivity(
    id: "123"
    attachmentIds: [2, 3]
  ) {
    success
    message
    activity {
```

```
      id
      name
      attachments {
        id
        fileName
      }
    }
  }
}
```

Clear all files:

```
mutation {
  updateActivity(
    id: "123"
    attachmentIds: []
  ) {
    success
    message
    activity {
      id
      name
      attachments {
        id
        fileName
      }
    }
  }
}
```

🔥 VERY IMPORTANT FRONTEND NOTE

GraphQL argument name in Python:

```python
```

In GraphQL query it becomes:

```graphql
```

✅ Use:

- `attachmentIds`

❌ Not:

- `attachment_ids`

✅ COPY-PASTE READY PATCHES

---

1) `program/models.py` — only exact change inside `Activity`

```python
learning_resource = models.ForeignKey(
    SecureAttachment,
    on_delete=models.CASCADE,
    null=True,
    blank=True
)

# NEW: supports multiple files on one activity
attachments = models.ManyToManyField(
    SecureAttachment,
    blank=True,
    related_name="activity_attachments"
)
```

---

2) `program/graphql/types.py` — add import

Python Run

---

3) `program/graphql/types.py` — add class above `ProgramActivityType`

```python
class ActivityAttachmentFileType(DjangoObjectType):
    class Meta:
        model = SecureAttachment
        fields = (
            "id",
```

```
            "file_name",
            "description",
            "asset_type",
            "sharepoint_url",
            "external_url",
            "created_at",
            "updated_at",
            "is_active",
        )
```

---

4) `program/graphql/types.py` — update `ProgramActivityType`

Add field:

Python Run

Add resolver:

```python
def resolve_attachments(self, info):
```

Python Run

---

5) `program/graphql/Activity/mutations.py` — add import

Python Run

---

6) `CreateActivity.Arguments`

Python Run

---

7) `CreateActivity.mutate()` signature

```python
def mutate(
    root,
    info,
    type,
    name,
    template_id=None,
    description=None,
    deadline=None,
    mapp_survey=None,
    is_360=None,
    mapp_survey_name=None,
    is_feedback=False,
    mapp_tool=None,
    learning_resource_id=None,
    attachment_ids=None,
):
```

8) `CreateActivity` after save

```
activity.full_clean()
activity.save()

# NEW: attach multiple files
if attachment_ids:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

9) `UpdateActivity.Arguments`

Python Run

10) `UpdateActivity.mutate()` signature

```
def mutate(
    root,
```

```
    info,
    id,
    template_id=None,
    name=None,
    type=None,
    description=None,
    deadline=None,
    mapp_survey=None,
    is_360=None,
    mapp_survey_name=None,
    is_feedback=None,
    mapp_tool=None,
    learning_resource_id=None,
    attachment_ids=None,
):
```

---

11) `UpdateActivity` after save

```
activity.full_clean()
activity.save()

# NEW: update multiple files
if attachment_ids is not None:
    attachments = SecureAttachment.objects.filter(
        id__in=attachment_ids,
        is_active=True
    )
    activity.attachments.set(attachments)
```

⚠️ ONE IMPORTANT OPTIONAL IMPROVEMENT (RECOMMENDED)

Right now if frontend sends invalid IDs like `[9999]`, your code silently ignores them.

If you want strict validation, use this instead:

```
if attachment_ids:
    attachments = list(
        SecureAttachment.objects.filter(id__in=attachment_ids, is_active=True)
    )
    if len(attachments) != len(set(attachment_ids)):
        return CreateActivity(
            activity=None,
            success=False,
            message="One or more attachment IDs are invalid."
        )
    activity.attachments.set(attachments)
```

But for now, keep simple version first.

✅ BEST TEAM-SAFE RESULT

After these changes:

- old single file support via `learning_resource` still works
- new multiple file support via `attachments` works
- no breaking schema rename
- no duplicate type conflicts
- no need to touch existing attachment REST APIs

🚀 NEXT BEST STEP (I can give this next)

Once backend is done, I can now give you the **exact frontend changes** for your `program management page.tsx`:

I can give you:

1. exact GraphQL query to fetch attachments from `/api`
2. exact dropdown UI for **multi-select**
3. exact `createActivity` mutation payload using `attachmentIds`
4. exact line where to read:

TypeScript

so your previous `"No activity id returned"` issue is fixed too

👉 Best next reply from you:

Send me:

- your **current createActivity GraphQL mutation string** from frontend
- the part where you submit activity in `page.tsx`

And I'll give you **copy-paste frontend code only**, with:

- **multi-file dropdown**
- **attachmentIds**
- **safe response parsing**

- **exact replacement blocks**

are backend steps and changes done if yes let me know what frontend things we are going to create update what files do you need to fetch all attachment files from backend

Yes ✅ **backend part is done** (for the multi-file activity attachment feature) **if you applied all the changes I gave**:

- `program/models.py` → added `attachments = ManyToManyField(...)`
- ran `makemigrations` + `migrate`
- `program/graphql/types.py` → added `attachments` on `ProgramActivityType`
- `program/graphql/Activity/mutations.py` → added `attachmentIds` in:
  - `createActivity`
  - `updateActivity`

✅ Backend checklist (quick verify)

Before frontend, just confirm these 3 things:

1) Migration applied

```
python manage.py makemigrations
```

Bash

2) This mutation works in GraphQL

```
mutation {
  createActivity(
    type: "assignment"
    name: "Test Multi File"
    attachmentIds: [1,2]
  ) {
    success
    message
    activity {
      id
      name
      attachments {
        id
        fileName
      }
    }
  }
}
```

3) Existing query returns attachments

```
query {
  activityById(id: 123) {
    id
```

```
    name
    attachments {
      id
      fileName
    }
  }
}
```

If these work → **backend is fully ready**.

🚀 FRONTEND: What we are going to create/update

We only need to do **4 frontend parts**.

---

1) Fetch all uploaded attachments from backend

We will call your GraphQL `/api` query:

```
query {
  attachments {
    id
    fileName
    assetType
    description
    sharepointUrl
    externalUrl
  }
}
```

This will fill the dropdown list.

---

2) Add multi-select attachment dropdown in Activity Create form

In your **Program Management page** activity form:

- show **all uploaded files**
- allow selecting **multiple files**
- store selected IDs in state

Example UI:

- checkbox list
- multi-select dropdown
- searchable selector (best)
- simple `<select multiple>` (fastest)

👉 Since your page is already huge, **best approach = create a reusable component file**.

---

3) Send `attachmentIds` in createActivity mutation

When activity is created, frontend should send:

GraphQL

⚠️ GraphQL uses **camelCase**:

- backend arg = `attachment_ids`
- frontend mutation = `attachmentIds`

4) Send `attachmentIds` in updateActivity mutation too

When editing activity:

- load already attached files
- pre-select them
- update with new list
- allow clearing all

✅ Best frontend structure (recommended)

Because your `page.tsx` is already very large (you already said this), the cleanest approach is:

Create 2 small reusable files

File 1 (NEW)

`components/ActivityAttachmentSelector.tsx`

This component will:

- fetch all attachments
- render multi-select UI
- return selected IDs to parent

Props:

- `selectedAttachmentIds`
- `setSelectedAttachmentIds`

File 2 (UPDATE)

`program management page.tsx`

We will:

- import `ActivityAttachmentSelector`
- add state:

TypeScript

- render selector in create activity form
- render selector in edit activity form
- pass `attachmentIds` in mutations

📁 Files I need from you for frontend

To give you **exact copy-paste code**, I need only these frontend parts:

---

1) Your current `page.tsx` (important parts only)

From your **program management page.tsx**, send these exact sections:

A. Your create activity mutation string

Something like:

```
TypeScript
```

or gql block / fetch body / Apollo mutation — whatever you use.

B. Your update activity mutation string

Something like:

```
TypeScript
```

C. Your activity form state

The section where you store:

- activity name
- description
- type
- deadline
- etc.

Usually looks like:

```
TypeScript
```

or separate states.

D. Your onSubmit / createActivityAndAttachToProgram function

This is very important because your earlier error was:

> `No activity id returned from createActivity`

So I need the function where you call:

- `createActivity`
- then attach to program/session/topic

Likely functions like:

- `createActivityAndAttachToProgram`
- `onSubmit`
- `handleSubmit`

E. The JSX section where activity create form is rendered

The part where fields are shown:

- name
- type
- description
- deadline
- etc.

This is where I'll tell you exactly where to insert the attachment selector.

🔥 Optional but very helpful (if you already have this file)

If you have a file like:

- `uploadedfileslist.tsx`
- `UpdateAttachmentDialog.tsx`
- any existing file list component

Then send that too.

Because if you already fetch files somewhere, I can **reuse existing logic** instead of creating duplicate API code.

✅ What I will give you after you send those

Once you send those frontend parts, I will give you:

1. New file:

- `components/ActivityAttachmentSelector.tsx`
  - fully copy-paste ready

2. Exact changes in `page.tsx`

- exact import line
- exact new state lines
- exact place in JSX to paste component
- exact mutation update (`attachmentIds`)
- exact response parsing fix for:

TypeScript

3. Edit mode support

- prefill selected attachments for existing activity
- update mutation integration

4. Minimal safe approach

- no breaking current UI
- no touching unrelated logic

✅ Short answer

Yes — backend is done if you applied the changes + migrations successfully.

For frontend, I need these from you:

1. **create activity mutation**
2. **update activity mutation**
3. **createActivityAndAttachToProgram / onSubmit function**
4. **activity form JSX section**
5. (optional) existing `uploadedfileslist.tsx` or attachment-fetch logic

👉 Send me these next:

Just paste from `page.tsx`:

- `CREATE_ACTIVITY_MUTATION`
- `UPDATE_ACTIVITY_MUTATION`
- `createActivityAndAttachToProgram`
- activity form JSX

Then I'll give you **exact frontend code only**, step-by-step, with **replace this / paste below this** style — no confusion.

in backend checklist step 2 i see this response {

```
  "errors": [
    {
      "message": "Authentication required.",
      "locations": [
        {
          "line": 2,
          "column": 3
        }
      ],
      "path": [
        "createActivity"
      ]
    }
  ],
  "data": {
    "createActivity": null
```

```
  }
}
```

Perfect — **this is NOT a backend code problem** ✅
This means your backend schema/mutation is loaded correctly, but the mutation is protected by auth.

✅ What this error means

You got:

```
{
  "errors": [
    {
      "message": "Authentication required."
    }
  ],
  "data": {
    "createActivity": null
  }
}
```

Because in your mutation file you have:

```python
</> Python
```

on:

- `CreateActivity`
- `UpdateActivity`
- `DeleteActivity`

So GraphQL is saying:

> "Mutation exists and works, but you are not logged in / token not passed."

✅ GOOD NEWS

This actually confirms:

- `createActivity` mutation is registered
- backend schema is loading
- no schema crash
- your changes are not broken
- only auth is missing in GraphQL Playground / request

👉 So **backend structure is okay**.

🔥 IMPORTANT

This means:

✅ **Your backend changes are likely correct enough to proceed to frontend**

But before frontend, we should do **one proper authenticated test** from your actual frontend (or GraphQL with token) so we confirm `attachmentIds` works.

✅ Why this happens in your project

Your mutation has:

```python
```

That decorator checks:

- `info.context.user.is_authenticated`
- or auth header token/session (depending on your setup)

If you run mutation directly in GraphiQL/Playground without auth cookie/token → it fails.

✅ What you should do now (best options)

OPTION 1 (BEST): Test from your frontend app

This is the easiest and safest.

Because your frontend is already logged in and already calling `/api`.

So instead of testing in Playground, we do the real test in frontend by:

- adding `attachmentIds` to existing create activity mutation
- submit activity from UI
- verify files attached

👉 This is the best next step.

OPTION 2: Test in GraphQL Playground with auth (only if you want)

If your backend uses JWT / token in headers, you can add auth header in GraphiQL/Playground.

Example:

```
{
  "Authorization": "Bearer YOUR_ACCESS_TOKEN"
```

JSON

or if your project uses session auth and you're logged in in same browser, it may work automatically depending on setup.

But honestly for your project:

👉 **Do frontend test now.**

✅ So answer to your question:

Is backend done?

YES — backend is functionally ready enough to move to frontend

Because:

- schema is loading

- mutation exists
- auth is the only blocker in manual GraphQL test
- that is expected because of `@graphql_permission_required`

⚠️ One thing I want you to verify before frontend (very important)

Please test this **read-only query** in GraphQL `/api` (or your frontend if easier):

```
query {
  attachments {
    id
    fileName
    assetType
    description
    sharepointUrl
    externalUrl
  }
}
```

If this query returns attachment list:

✅ then frontend can fetch dropdown data immediately.

🚀 FRONTEND NEXT — what I need from you NOW

Since backend is okay, now send me **these exact frontend pieces** from your `page.tsx`:

---

1) `CREATE_ACTIVITY_MUTATION`

The full mutation string / gql block

---

2) `UPDATE_ACTIVITY_MUTATION`

The full mutation string / gql block

---

3) `createActivityAndAttachToProgram(...)`

(or whatever exact function creates activity first, then links it)

This is the one that earlier gave:

> `No activity id returned from createActivity`

I need this exact function.

---

4) Activity form JSX section

The part where you render:

- activity name
- activity type
- description
- deadline
- etc.

This is where I will insert the attachment selector.

---

5) (Optional but very useful)

If you already have file list logic in:

- `uploadedfileslist.tsx`
- `UpdateAttachmentDialog.tsx`

send that too.

Because if you already fetch uploaded files somewhere, I'll **reuse that existing fetch** instead of creating duplicate logic.

✅ Short conclusion

Your backend is NOT failing.

It is only saying:

> **You must be logged in to run createActivity**

That is expected because of:

```python
```

So yes:

Proceed to frontend now ✅

👉 Next message from you

Please paste:

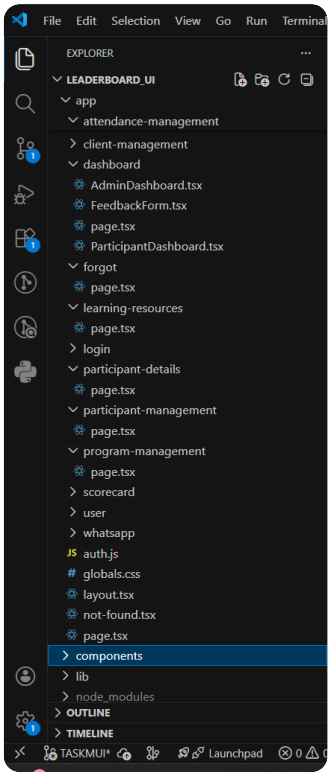
1. `CREATE_ACTIVITY_MUTATION`
2. `UPDATE_ACTIVITY_MUTATION`
3. `createActivityAndAttachToProgram`
4. activity form JSX
5. if possible `uploadedfileslist.tsx`

Then I'll give you:

- exact new component file
- exact page.tsx changes
- exact mutation changes (`attachmentIds`)
- exact fix for `"No activity id returned"` issue too

If you want, after you send it, I'll do it in **your preferred style**:

"Step 1 replace this block / Step 2 paste below this line" only.



Pasted text.txt
Document

this is my frontend project structure
// lib/queries.ts
// All GraphQL operations used across the Leaderboard App

/* ----------
AUTHENTICATION: TOKEN REFRESH
---------- */

```
export const REFRESH_TOKEN_MUTATION = `
mutation RefreshToken($refreshToken: String!) {
  refreshToken(refreshToken: $refreshToken) {
    success
    token
    refreshToken
    errors
  }
}
`;

export const VERIFY_TOKEN_QUERY = `
mutation VerifyToken($token: String!) {
  verifyToken(token: $token) {
    success
    errors
  }
}
`;

export const PASSWORD_RESET_MUTATION = `
mutation PasswordReset($token: String!, $newPassword1: String!,
$newPassword2: String!) {
  passwordReset(token: $token, newPassword1: $newPassword1,
newPassword2: $newPassword2) {
    success
    errors
  }
}
```

```
;

/* ---------------------------------------------------
   SURVEY TOKEN
--------------------------------------------------- */

export const CREATE_SURVEY_TOKEN_QUERY =
query CreateSurveyToken($activityId: Int!, $invite: Boolean, $message:
String, $topicId: Int, $sessionId: Int, $programId: Int, $activtyStatus:
String, $userInput: UserSurveyInput) {
  createSurveyToken(
    activityId: $activityId
    invite: $invite
    message: $message
    topicId: $topicId
    sessionId: $sessionId
    programId: $programId
    activtyStatus: $activtyStatus
    userInput: $userInput
  )
}
;

/* ---------------------------------------------------
   360 SURVEY STATUS
--------------------------------------------------- */

export const FETCH_360_STATUS =
query Fetch360Status($activityId: Int!, $responseId: Int!) {
```

```
  fetchSurveyStatusFor360(activityId: $activityId, responseSourceFor:
$responseId) {
    username
    status
  }
}
;

/* ---------------------------------------------------
   ME / PROFILE
--------------------------------------------------- */

// Full version used by dashboard/sidebar
export const ME_ENROLLMENTS =
  query MeEnrollments {
    me {
      id
      firstName
      lastName
      email
      isStaff
      phoneNumber
      dateJoined
      isActive
      clientAdmins {
        totalCount
        edges {
          node {
            id
```

```
        programs {
          totalCount
          edges {
            node {
              id
            }
          }
        }
      }
    }
  }
  programEnrollments {
    totalCount
    edges {
      node {
        id
        program{
          id
          name
        }
      }
    }
  }
  userprogrampreference {
    id
    defaultProgram {
      id
      name
      scorecards{
```

```
          edges{
            node{
              id
              name
            }
          }
        }
        client{
          id
          contactEmail
        }
      }
      client{
        id
      }
      role
      roleProgramJson
    }
  }
}
;

// Lightweight version used by scorecard page to keep defaultProgram
+ scorecards in sync
// after program preference changes, without fetching the full
enrollment graph.
export const ME_DEFAULT_PROGRAM_SCORECARDS =
  query MeDefaultProgramScorecards {
    me {
```

```
    id
    firstName
    lastName
    email
    userprogrampreference {
      id
      defaultProgram {
        id
        name
        scorecards {
          edges {
            node {
              id
              name
            }
          }
        }
        client {
          id
          contactEmail
        }
      }
      client {
        id
      }
      role
      roleProgramJson
    }
  }
```

```
  }
;

/* ---------------------------------------------------
   ADMIN: PROGRAM DETAILS BY ID
--------------------------------------------------- */

export const PROGRAM_BY_ID =
query programByID($id: ID!) {
  Program(id: $id) {
    id
    name
    description
    client {
      id
      name
      contactEmail
      phoneNumber
      address
    }
    startDate
    endDate
    status
    maxParticipants
    contentUnlockMode
    unlockScope
    creator {
      id
      username
```

```
      firstName
      lastName
    }
    preActivity {
      edges {
        node {
          id
          type
          description
          deadline
          mappSurvey
          mappSurveyName
          mappTool
          is360
          isFeedback
          name
          visible
          accessible
          totalResponses
        }
      }
    }
    postActivity {
      edges {
        node {
          id
          type
          description
          deadline
```

```
        mappSurvey
        mappSurveyName
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
  }
  sessions {
    edges {
      node {
        id
        name
        description
        deliveryMode
        startDate
        endDate
        location
        status
        rating
        visible
        accessible
        program {
          id
          name
```

```
        }
        preActivity {
          edges {
            node {
              id
              type
              description
              deadline
              mappSurvey
              mappSurveyName
              mappTool
              is360
              name
              visible
              accessible
              totalResponses
            }
          }
        }
        postActivity {
          edges {
            node {
              id
              type
              description
              deadline
              mappSurvey
              mappSurveyName
              mappTool
```

```
            is360
            name
            visible
            accessible
            totalResponses
          }
        }
      }
      topic {
        edges {
          node {
            id
            name
            description
            creator {
              id
              username
              firstName
              lastName
            }
            startDate
            endDate
            visible
            accessible
            preActivity {
              edges {
                node {
                  id
                  type
```

```
        description
        deadline
        mappSurvey
        mappSurveyName
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
  }
  postActivity {
    edges {
      node {
        id
        type
        description
        deadline
        mappSurvey
        mappSurveyName
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
```

```
                totalResponses
              }
            }
          }
        }
      }
      attendees {
        edges {
          node {
            id
            status
            attendanceDate
            participant {
              id
              username
              firstName
              lastName
              email
            }
          }
        }
      }
    }
  }
}
attendees {
  edges {
    node {
```

```
        id
        status
        attendanceDate
        participant {
          id
          firstName
          lastName
          email
        }
      }
    }
  }
  participantReplacements {
    edges {
      node {
        id
        oldParticipant {
          id
          username
          firstName
          lastName
          email
        }
        newParticipant {
          id
          username
          firstName
          lastName
          email
```

```
        }
      }
    }
  }
  enrolledEmployees {
    pageInfo {
      hasNextPage
      endCursor
    }
    edges {
      node {
        id
        participant {
          id
          username
          firstName
          lastName
          email
        }
      }
    }
  }
 }
}
;

/* ---------------------------------------------------
  PROGRAM STATUS ENUM (for status label display; single source of truth from backend)
```

```
   ---------------------------------------------------- */

export const PROGRAM_STATUS_ENUM = `
query ProgramStatusEnum {
  programStatus: __type(name: "ProgramStatus") {
    enumValues {
      name
      description
    }
  }
}
`;

/* ----------------------------------------------------
   ADMIN: PROGRAM MANAGEMENT ENUMS (for dropdowns)
   ---------------------------------------------------- */

export const PROGRAM_MANAGEMENT_ENUMS = `
query ProgramManagementEnums {
  programStatus: __type(name: "ProgramStatus") {
    enumValues {
      name
      description
    }
  }
  journeyMode: __type(name: "ProgramContentUnlockMode") {
    enumValues {
      name
      description
```

```
    }
  }
  visibilityScope: __type(name: "ProgramUnlockScope") {
    enumValues {
      name
      description
    }
  }
  activityTypes: __type(name: "ActivityType") {
    enumValues {
      name
      description
    }
  }
  mappToolList: __type(name: "ActivityTemplateMappTool") {
    enumValues {
      name
      description
    }
  }
  sessionLocationList: __type(name: "SessionLocation") {
    enumValues {
      name
      description
    }
  }
  sessionTypeList: __type(name: "SessionDeliveryMode") {
    enumValues {
      name
```

```
      description
    }
  }
  sessionStatusList: __type(name: "SessionStatus") {
    enumValues {
      name
      description
    }
  }
}
;

/* ---------------------------------------------------
   PARTICIPANT: DASHBOARD ENUMS (status + location labels)
   --------------------------------------------------- */

export const PARTICIPANT_DASHBOARD_ENUMS =
query ParticipantDashboardEnums {
  sessionLocationList: __type(name: "SessionLocation") {
    enumValues {
      name
      description
    }
  }
  sessionStatusList: __type(name: "SessionStatus") {
    enumValues {
      name
      description
    }
```

```
  }
}
;

/* ---------------------------------------------------
   PARTICIPANT: DASHBOARD ENROLLMENT
--------------------------------------------------- */

export const PARTICIPANT_PROGRAM_ENROLLMENT = `
query ParticipantProgramEnrollment($id: ID!) {
  participantProgramEnrollmentById(id: $id) {
    id
    enrollmentDate
    participant {
      id
      username
      email
      firstName
      lastName
      lastLogin
      dateJoined
      phoneNumber
      isActive
      isStaff
      sessionAttendance {
        edges {
          node {
            id
            status
```

```
        attendanceDate
        session {
          id
        }
      }
    }
  }
  activityResponses {
    edges {
      node {
        id
        topic {
          id
        }
        activity {
          id
        }
        program {
          id
        }
        session {
          id
        }
        activtyStatus
        responseData
        submissionDate
        score
        tokenLink
        status
```

```
        }
      }
    }
  }
  program {
    id
    name
    description
    status
    startDate
    endDate
    maxParticipants
    creator {
      id
      username
      email
    }
    client {
      id
      name
    }
    preActivity {
      edges {
        node {
          id
          type
          description
          deadline
          mappSurvey
```

```
        mappSurveyName
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
  }
  postActivity {
    edges {
      node {
        id
        type
        description
        deadline
        mappSurvey
        mappSurveyName
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
```

```
    }
    sessions {
      edges {
        node {
          id
          name
          description
          deliveryMode
          startDate
          endDate
          location
          status
          visible
          accessible
          rating
          preActivity {
            edges {
              node {
                id
                type
                description
                deadline
                mappSurvey
                mappSurveyName
                mappTool
                is360
                isFeedback
                name
                visible
```

```
        accessible
        totalResponses
      }
    }
  }
  postActivity {
    edges {
      node {
        id
        type
        description
        deadline
        mappSurvey
        mappSurveyName
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
  }
  topic {
    edges {
      node {
        id
        name
```

```
        description
        startDate
        endDate
        visible
        accessible
        preActivity {
          edges {
            node {
              id
              type
              description
              deadline
              mappSurvey
              mappSurveyName
              mappTool
              is360
              isFeedback
              name
              visible
              accessible
              totalResponses
            }
          }
        }
        postActivity {
          edges {
            node {
              id
              type
```

```
                    description
                    deadline
                    mappSurvey
                    mappSurveyName
                    mappTool
                    is360
                    isFeedback
                    name
                    visible
                    accessible
                    totalResponses
                  }
                }
              }
            }
          }
        }
      }
    }
    attendees {
      edges {
        node {
          id
          status
          attendanceDate
          participant {
            id
            firstName
```

```
            lastName
            email
          }
        }
      }
    }
  }
}
;

/* ---------------------------------------------------
   ACCOUNT-MANAGER: CLIENT LIST
--------------------------------------------------- */

export const MY_CLIENTS =
  query MyClients($name_search: String) {
    me {
      id
      clientAdmins(name_Icontains: $name_search) {
        edges {
          node {
            id
            name
          }
        }
      }
    }
  }
```

```
;

/* ---------------------------------------------------
   ACCOUNT-MANAGER: PROGRAMS IN A SELECTED CLIENT
--------------------------------------------------- */

export const SELECTED_CLIENT_PROGRAMS =
  query SelectedClientPrograms($client_id: ID) {
    me {
      id
      clientAdmins(id: $client_id) {
        edges {
          node {
            id
            name
            programs {
              edges {
                node {
                  id
                  name
                }
              }
            }
          }
        }
      }
    }
  }
;
```

```
/* ---------------------------------------------------
   CLIENT-ADMIN: THEIR PROGRAMS
--------------------------------------------------- */

export const MY_PROGRAMS = 
  query MyPrograms {
    me {
      id
      clientAdmins {
        edges {
          node {
            id
            programs {
              edges {
                node {
                  id
                  name
                  client {
                    id
                    name
                  }
                }
              }
            }
          }
        }
      }
    }
```

```
 }
;

/* ---------------------------------------------------
   PREFERENCES: SET DEFAULT PROGRAM
--------------------------------------------------- */

export const SET_PROGRAM_PREF =
  mutation setDefaultProgram($program_id: ID, $clientId: ID, $role:
String, $roleProgramJson: JSONString) {
    setDefaultProgram(userProgramPreferenceData: {defaultProgramId:
$program_id, clientId: $clientId, role: $role, roleProgramJson:
$roleProgramJson}) {
      userProgramPreference {
        id
        defaultProgram {
          id
        }
        client {
          id
        }
        role
        roleProgramJson
      }
    }
  }
;

export const UPDATE_PROGRAM_PREF =
```

```
  mutation updateDefault($id: ID!, $program_id: ID, $clientId: ID, $role: String, $roleProgramJson: JSONString) {
    updateDefaultProgram(id: $id, defaultProgramId: $program_id, clientId: $clientId, role: $role, roleProgramJson: $roleProgramJson) {
      userProgramPreference {
        id
        defaultProgram {
          id
        }
        client {
          id
        }
        role
        roleProgramJson
      }
    }
  }
;

/* ---------------------------------------------------
   MANAGE PROGRM: LISTS FOR PROGRAM MANAGEMENT
--------------------------------------------------- */
// Updated queries.ts
// Full set of queries provided by user.

export const ALL_PROGRAMS_LIST =
query allProgramsList($id: ID) {
  Programs(creator_Id: $id) {
    edges {
```

```
      node {
        id
        name
      }
    }
  }
}
;

export const ALL_SESSIONS_LIST =
query allSessionsList {
  allSessions {
    edges {
      node {
        id
        name
      }
    }
  }
}
;

export const ALL_TOPICS_LIST =
query allTopicsList($id: ID) {
  allTopics(creator_Id: $id) {
    edges {
      node {
        id
        name
```

```
      }
    }
  }
}
;

export const ALL_ACTIVITIES_LIST =
query allActivitiesList {
  allActivities {
    edges {
      node {
        id
        name
      }
    }
  }
}
;

export const MAPP_SURVEY_LIST =
query mappSurveyList {
  mappSurveys {
    id
    name
  }
}
;

export const ALL_PROGRAMS =
```

```
query allPrograms($id: ID) {
  Programs(creator_Id: $id) {
    edges {
      node {
        id
        name
        client {
          id
        }
        startDate
        endDate
        status
        maxParticipants
      }
    }
  }
}
;

export const ALL_SESSIONS =
query allSessions {
  allSessions {
    edges {
      node {
        id
        name
        location
        type
        startDate
```

```
        endDate
        program {
          id
        }
      }
    }
  }
}
;

export const ALL_TOPICS =
query allTopics($id: ID) {
  allTopics(creator_Id: $id) {
    edges {
      node {
        id
        name
        description
        startDate
        endDate
        creator {
          id
        }
        preActivity {
          edges {
            node {
              id
            }
          }
```

```
        }
        postActivity {
          edges {
            node {
              id
            }
          }
        }
      }
    }
  }
}
;

export const ALL_ACTIVITIES =
query allActivities {
  allActivities {
    edges {
      node {
        id
        name
        type
        deadline
        description
        is360
        isFeedback
        mappSurvey
        mappSurveyName
        mappTool
```

```
      }
    }
  }
}
`;

export const GET_ALL_USERS = `
query getAllUsers {
  users {
    edges {
      node {
        id
        username
        firstName
        lastName
      }
    }
  }
}
`;

/* ---------------------------------------------------
   MANAGE PROGRM: CREATE AND UPDATE MUTATIONS
--------------------------------------------------- */
/* Program */
export const CREATE_PROGRAM_MUTATION = `
mutation CreateProgram($clientId: Int!, $endDate: DateTime!,
$maxParticipants: Int!, $name: String!, $description: String!, $startDate:
DateTime!, $status: String, $contentUnlockMode: String, $unlockScope:
```

```
String, $preActivity: [Int], $postActivity: [Int]) {
  createProgram(
    clientId: $clientId
    endDate: $endDate
    maxParticipants: $maxParticipants
    name: $name
    description: $description
    startDate: $startDate
    status: $status
    contentUnlockMode: $contentUnlockMode
    unlockScope: $unlockScope
    preActivity: $preActivity
    postActivity: $postActivity
  ) {
    success
    message
    program{
      id
    }
  }
}
`;

export const UPDATE_PROGRAM_MUTATION = `
mutation UpdateProgram($id: ID!, $clientId: Int, $endDate: DateTime,
$maxParticipants: Int, $name: String, $description: String, $startDate:
DateTime, $status: String, $contentUnlockMode: String, $unlockScope:
String, $preActivity: [Int], $postActivity: [Int]) {
  updateProgram(
```

```
    id: $id
    clientId: $clientId
    endDate: $endDate
    maxParticipants: $maxParticipants
    name: $name
    description: $description
    startDate: $startDate
    status: $status
    contentUnlockMode: $contentUnlockMode
    unlockScope: $unlockScope
    preActivity: $preActivity
    postActivity: $postActivity
  ) {
    success
    message
    program {
      id
      name
      client {
        id
      }
      startDate
      endDate
      status
      maxParticipants
      contentUnlockMode
      unlockScope
    }
  }
```

```
}
;

/* Session */
export const CREATE_SESSION_MUTATION =
mutation CreateSession($endDate: DateTime!, $startDate: DateTime!,
$location: String!, $name: String!, $description: String!, $programId: Int!,
$topicIds: [Int], $deliveryMode: String!, $preActivity: [Int], $postActivity:
[Int], $status: String) {
  createSession(
    endDate: $endDate
    startDate: $startDate
    location: $location
    name: $name
    description: $description
    programId: $programId
    topicIds: $topicIds
    deliveryMode: $deliveryMode
    preActivity: $preActivity
    postActivity: $postActivity
    status: $status
  ) {
    success
    message
    session {
      id
    }
  }
}
```

```
;

export const UPDATE_SESSION_MUTATION = `
mutation UpdateSession($id: ID!, $endDate: DateTime, $startDate:
DateTime, $location: String, $name: String, $description: String,
$programId: Int, $topicIds: [Int], $deliveryMode: String, $preActivity:
[Int], $postActivity: [Int], $status: String) {
  updateSession(
    id: $id
    endDate: $endDate
    startDate: $startDate
    location: $location
    name: $name
    description: $description
    programId: $programId
    topicIds: $topicIds
    deliveryMode: $deliveryMode
    preActivity: $preActivity
    postActivity: $postActivity
    status: $status
  ) {
    success
    message
    session {
      id
      name
      location
      description
      deliveryMode
```

```
      startDate
      endDate
      status
      program {
        id
      }
    }
  }
}
;

/* Topic */
export const CREATE_TOPIC_MUTATION =
mutation CreateTopic($description: String!, $endDate: String!, $name:
String!, $postActivities: [Int], $preActivities: [Int], $startDate: String!) {
  createTopic(
    description: $description
    endDate: $endDate
    name: $name
    postActivity: $postActivities
    preActivity: $preActivities
    startDate: $startDate
  ) {
    success
    message
    topic {
      id
    }
  }
```

```
}
;

export const UPDATE_TOPIC_MUTATION = `
mutation UpdateTopic($id: ID!, $creatorId: ID, $description: String,
$startDate: DateTime, $endDate: DateTime, $name: String,
$preActivities: [Int], $postActivities: [Int]) {
  updateTopic(
    id: $id
    creatorId: $creatorId
    description: $description
    startDate: $startDate
    endDate: $endDate
    name: $name
    preActivity: $preActivities
    postActivity: $postActivities
  ) {
    success
    message
    topic {
      id
      name
      startDate
      endDate
      creator {
        id
      }
      preActivity {
        edges {
```

```
        node {
          id
        }
      }
    }
    postActivity {
      edges {
        node {
          id
        }
      }
    }
  }
}
;

/* Activity */
export const CREATE_ACTIVITY_MUTATION =
mutation CreateActivity(
  $name: String!
  $type: String!
  $deadline: String
  $description: String
  $is360: Boolean
  $isFeedback: Boolean
  $mappSurvey: Int
  $mappSurveyName: String
  $mappTool: String
```

```
  $templateId: Int
) {
  createActivity(
    name: $name
    type: $type
    deadline: $deadline
    description: $description
    is360: $is360
    isFeedback: $isFeedback
    mappSurvey: $mappSurvey
    mappSurveyName: $mappSurveyName
    mappTool: $mappTool
    templateId: $templateId
  ) {
    success
    message
    activity{
      id
    }
  }
}
`;

export const UPDATE_ACTIVITY_MUTATION = `
mutation UpdateActivity($id: ID!, $name: String, $type: String,
$deadline: String, $description: String, $is360: Boolean, $isFeedback:
Boolean, $mappSurvey: Int, $mappSurveyName: String, $mappTool:
String) {
  updateActivity(
```

```
    id: $id
    name: $name
    type: $type
    deadline: $deadline
    description: $description
    is360: $is360
    isFeedback: $isFeedback
    mappSurvey: $mappSurvey
    mappSurveyName: $mappSurveyName
    mappTool: $mappTool
  ) {
    success
    message
    activity {
      id
      name
      type
      deadline
      description
      is360
      mappSurvey
      mappSurveyName
      mappTool
    }
  }
}
;

export const DELETE_ACTIVITY_MUTATION =
```

```
mutation deleteActivity($id: ID!) {
  deleteActivity(id: $id) {
    success
    message
  }
}
;

/* Activity Templates */
export const ALL_ACTIVITY_TEMPLATES =
query getActivityTemplateList($isFeedback: Boolean = false) {
  allActivitiesTemplate(isFeedback: $isFeedback) {
    edges {
      node {
        id
        name
      }
    }
  }
}
;

export const ACTIVITY_TEMPLATE_BY_ID =
query getActivityTempleteData($id: Int!) {
  activityTemplateById(id: $id) {
    id
    mappSurvey
    mappSurveyName
    mappTool
```

```
    type
    description
    is360
    name
    isFeedback
  }
}
;

export const CREATE_ACTIVITY_TEMPLATE_MUTATION = `
mutation CreateActivityTemplate(
  $description: String
  $is360: Boolean
  $isFeedback: Boolean
  $mappSurvey: Int
  $mappSurveyName: String
  $mappTool: String
  $name: String!
  $type: String!
) {
  createActivityTemplate(
    description: $description
    is360: $is360
    isFeedback: $isFeedback
    mappSurvey: $mappSurvey
    mappSurveyName: $mappSurveyName
    mappTool: $mappTool
    name: $name
    type: $type
```

```
  ) {
    success
    message
    activity {
      id
      name
      mappSurvey
      mappSurveyName
      mappTool
      type
      description
      is360
      isFeedback
    }
  }
}
;

/* Duplicate Mutations */
export const DUPLICATE_PROGRAM_MUTATION = 
mutation DuplicateProgram($programId: Int!, $clientId: Int!) {
  duplicateProgram(programId: $programId, clientId: $clientId) {
    success
    message
    newProgram {
      id
      name
    }
  }
```

```
}
;

export const DUPLICATE_SESSION_MUTATION =
mutation DuplicateSession($sessionId: Int!) {
  duplicateSession(sessionId: $sessionId) {
    success
    message
    newSession {
      id
      name
    }
  }
}
;

export const DUPLICATE_TOPIC_MUTATION =
mutation DuplicateTopic($topicId: Int!) {
  duplicateTopic(topicId: $topicId) {
    success
    message
    newTopic {
      id
      name
    }
  }
}
;
```

```
/* ---------------------------------------------------
   ATTENDANCE MANAGEMENT: QUERIES AND MUTATIONS
--------------------------------------------------- */

export const ALL_SESSIONS_BY_PROGRAM = `
query AllSessions($program_Id: ID, $orderBy: [String]) {
  allSessions(
    program_Id: $program_Id
    orderBy: $orderBy
  ) {
    edges {
      node {
        id
        name
        startDate
        endDate
        location
        status
      }
    }
  }
}
`;

export const SESSION_BY_ID = `
query SessionById($id: ID!) {
  sessionById(id: $id) {
    id
    name
```

```
    status
    program {
      id
      name
      client {
        id
      }
      participantReplacements {
        edges {
          node {
            id
            oldParticipant {
              id
              username
              firstName
              lastName
              email
            }
            newParticipant {
              id
              username
              firstName
              lastName
              email
            }
          }
        }
      }
      enrolledEmployees {
```

```
      edges {
        node {
          id
          participant {
            id
            firstName
            lastName
            email
          }
        }
      }
    }
  }
  attendees {
    edges {
      node {
        id
        attendanceDate
        status
        participant {
          id
          firstName
          lastName
          email
        }
      }
    }
  }
}
```

```
}
;

export const CREATE_PARTICIPANT_SESSION_ATTENDANCE = `
mutation CreateParticipantSessionAttendance($participantId: Int!, $sessionId: Int!, $status: String!) {
  createParticipantSessionAttendance(
    participantId: $participantId
    sessionId: $sessionId
    status: $status
  ) {
    success
    message
    participantSessionAttendance{
      id
      status
    }
  }
}
;

export const UPDATE_PARTICIPANT_SESSION_ATTENDANCE = `
mutation UpdateParticipantSessionAttendance($id: ID!, $status: String) {
  updateParticipantSessionAttendance(id: $id, status: $status) {
    success
    message
    participantSessionAttendance{
      id
      status
```

```
    }
  }
}
`;

export const CREATE_PARTICIPANT_PROGRAM_ATTENDANCE = `
mutation CreateParticipantProgramAttendance(
  $participantId: Int!
  $programId: Int!
  $status: String!
) {
  createParticipantProgramAttendance(
    participantId: $participantId
    programId: $programId
    status: $status
  ) {
    participantProgramAttendance {
      id
    }
    success
    message
  }
}
`;

export const UPDATE_PARTICIPANT_PROGRAM_ATTENDANCE = `
mutation UpdateParticipantProgramAttendance(
  $id: ID!
  $status: String
```

```
) {
  updateParticipantProgramAttendance(
    id: $id
    status: $status
  ) {
    participantProgramAttendance {
      id
    }
    success
    message
  }
}
;

/* ---------------------------------------------------
   ATTENDANCE MANAGEMENT: QUERIES AND MUTATIONS
--------------------------------------------------- */

export const PROGRAM_BY_ID_FOR_PARTICIPANT_MANAGEMENT =
query programById($id: ID!){
  Program(id: $id){
    id
    name
    client{
      id
    }
    enrolledEmployees{
      edges{
        node{
```

```
          id
          participant{
            id
            email
            firstName
            lastName
          }
        }
      }
    }
  }
}
;

export const USERS_FOR_PARTICIPANT_MANAGEMENT = `
query usersForParticipantManagement($name: String, $program_Id: ID,
$emailDomain: String){
  userListByFirstName:users(firstName_Icontains: $name,
email_Icontains: $emailDomain, first: 20){
     edges{
      node{
        id
        firstName
        lastName
        email
        programEnrollments(program_Id: $program_Id){
          totalCount
        }
      }
```

```
    }
  }
  userListByLastName: users(lastName_Icontains: $name,
email_Icontains: $emailDomain, first: 20){
    edges{
     node{
       id
       firstName
       lastName
       email
       programEnrollments(program_Id: $program_Id){
         totalCount
       }
     }
    }
  }
  userListByEmail:users(email_Icontains: $name, first: 20){
    edges{
     node{
       id
       firstName
       lastName
       email
       programEnrollments(program_Id: $program_Id){
         totalCount
       }
     }
    }
  }
```

```
}
;

export const CREATE_PARTICIPANT_PROGRAM_ENROLLMENT =
mutation CreateParticipantProgramEnrollment(
  $participantId: Int!
  $programId: Int!
) {
  createParticipantProgramEnrollment(
    participantId: $participantId
    programId: $programId
  ) {
    message
    success
    participantProgramEnrollment{
      id
    }
  }
}
;

export const DELETE_PARTICIPANT_PROGRAM_ENROLLMENT =
mutation DeleteParticipantProgramEnrollment($id: ID!) {
  deleteParticipantProgramEnrollment(id: $id) {
    success
    message
  }
}
;
```

```
export const CREATE_PARTICIPANT_REPLACEMENT = `
mutation createParticipantReplacement(
  $action: String!
  $newParticipantId: Int
  $oldParticipantId: Int!
  $programId: Int!
  $reason: String!
) {
  createParticipantReplacement(
    action: $action
    newParticipantId: $newParticipantId
    oldParticipantId: $oldParticipantId
    programId: $programId
    reason: $reason
  ) {
    success
    message
    participantReplacement {
      id
    }
  }
}
`;

export const REGISTER_USER = `
mutation RegisterUser(
  $email: String!
  $username: String!
```

```
  $phoneNumber: String!
  $firstName: String!
  $lastName: String!
  $password1: String!
  $password2: String!
) {
  register(
    email: $email
    username: $username
    phoneNumber: $phoneNumber
    firstName: $firstName
    lastName: $lastName
    password1: $password1
    password2: $password2
  ) {
    success
    errors
  }
}
`;

export const REGISTER_INVITE_USER = `
mutation RegisterInviteUser(
  $email: String!
  $firstName: String
  $lastName: String
  $phoneNumber: String
  $username: String!
  $isStaff: Boolean
```

```
) {
  registerInviteUser(
    email: $email
    firstName: $firstName
    lastName: $lastName
    phoneNumber: $phoneNumber
    username: $username
    isStaff: $isStaff
  ) {
    success
    errors
  }
}
;

export const GET_CREATED_USER =
query getCreatedUser($email: String){
  users(email: $email){
    edges{
      node{
        id
      }
    }
  }
}
;

/* ----------------------------
   SCORECARD: Config & Activity
```

```
----------------------------*/

export const CREATE_SCORECARD_CONFIGURATION = `
mutation CreateScorecardConfiguration($programId: ID!, $name:
String!, $draftFlag: Boolean) {
  createScorecardConfiguration(
    programId: $programId
    name: $name
    draftFlag: $draftFlag
  ) {
    ok
    errors
    scorecardConfiguration {
      id
      name
    }
  }
}
`;

export const SCORECARD_CONFIGURATION_BY_ID = `
query scorecardConfigurationById($id: Int!) {
  scorecardConfigurationById(id: $id) {
    id
    name
    isDraft
    scorecardActivities {
      edges {
        node {
```

```
          id
          level
          weightPercentage
          isAttendance
          formula
          activity {
            id
            name
          }
          activityStatus
          session {
            id
            name
          }
          program {
            id
            name
          }
          topic {
            id
            name
          }
        }
      }
    }
  }
}
;
```

```
/* Fetch activities + sessions by program id to build selection list */
export const ACTIVITY_AND_SESSIONS_BY_PROGRAM =
query allActivityByProgramId($id: ID!) {
  Program(id: $id) {
    id
    name
    scorecards {
      edges {
        node {
          id
          name
          isDraft
        }
      }
    }
    preActivity {
      edges {
        node {
          id
          name
        }
      }
    }
    postActivity {
      edges {
        node {
          id
          name
        }
```

```
    }
  }
  sessions {
    edges {
      node {
        id
        name
        preActivity {
          edges {
            node {
              id
              name
            }
          }
        }
        postActivity {
          edges {
            node {
              id
              name
            }
          }
        }
        topic {
          edges {
            node {
              id
              name
              preActivity {
```

```
            edges {
              node {
                id
                name
              }
            }
          }
          postActivity {
            edges {
              node {
                id
                name
              }
            }
          }
        }
      }
    }
  }
  }
}
;

/* Create multiple scorecard activities in one mutation call */
export const CREATE_SCORECARD_ACTIVITY =
mutation CreateScorecardActivity(
  $scCfgId: ID!,
```

```
  $activities: [ScorecardActivityInput!]!
) {
  createScorecardActivity(
    scCfgId: $scCfgId,
    activities: $activities
  ) {
    ok
    errors
    scorecardActivity {
      id
      level
      weightPercentage
      isAttendance
      formula
      activity {
        id
        name
      }
      activityStatus
      session {
        id
        name
      }
      program {
        id
        name
      }
      topic {
        id
```

```
      name
    }
  }
 }
}
;

/* Update a scorecardActivity (id is the scorecardActivity id) */
export const UPDATE_SCORECARD_ACTIVITY = `
mutation UpdateScorecardActivity(
  $id: ID!
  $activityId: ID
  $activityStatus: String
  $isAttendance: Boolean
  $level: String
  $programId: ID
  $sessionId: ID
  $topicId: ID
  $weight: Decimal
  $formula: String
) {
  updateScorecardActivity(
    id: $id
    activityId: $activityId
    activityStatus: $activityStatus
    isAttendance: $isAttendance
    level: $level
    programId: $programId
    sessionId: $sessionId
```

```
    topicId: $topicId
    weight: $weight
    formula: $formula
  ) {
    ok
    errors
    scorecardActivity {
      id
      weightPercentage
      formula
      isAttendance
      level
    }
  }
}
;

/* Delete a scorecardActivity */
export const DELETE_SCORECARD_ACTIVITY =
mutation DeleteScorecardActivity($id: ID!) {
  deleteScorecardActivity(id: $id) {
    ok
    errors
    deletedId
  }
}
;

/* Program leaderboard (used to render the scorecard table once
```

```
weights sum to 100) */
export const PROGRAM_LEADERBOARD = `
query programLeaderboard($programId: Int!) {
  programLeaderboard(programId: $programId)
}
`;

/* ---------------------------------------------------
   WHATSAPP GROUP MANAGEMENT
--------------------------------------------------- */

export const GET_ALL_WHATSAPP_GROUPS = `
query getAllWhatsappGroup {
  waAllGroups {
    id
    name
    program {
      id
    }
    wagroupId
  }
}
`;

export const GET_GROUP_BY_ID = `
query getGroupById($id: ID!) {
  waGroupById(id: $id) {
    id
    name
```

```
    wagroupId
  }
}
;

export const GET_ALL_GROUP_MESSAGES =
query getAllGroupMessages($programId: ID!) {
  waGetMessageByProgramId(programId: $programId)
}
;

export const CREATE_WHATSAPP_GROUP =
mutation createWhatsappGroup($name: String!, $programId: ID!) {
  createWaGroup(name: $name, programId: $programId) {
    ok
    errors
    waGroup {
      id
    }
  }
}
;

export const SEND_MSG_IN_WA_GROUP =
mutation sendMsgInWAGroup($message: String!, $wagId: String!,
$mentions: [String], $quotedMessage: String) {
  sendMessageToGroup(message: $message, wagId: $wagId, mentions:
$mentions, quotedMessage: $quotedMessage) {
    ok
```

```
    errors
  }
}
`;

export const GET_WA_GROUP_INFO = `
query getWAGroupInfo($programId: ID!) {
  waGetGroupinfoByProgramId(programId: $programId)
}
`;

export const MANAGE_GROUP_ADMIN_BY_GROUP_ID = `
mutation manageGroupAdminByGroupID($action: ActionEnum!,
$participants: [String]!, $wagId: String!) {
  manageGroupAdmin(action: $action, participants: $participants,
wagId: $wagId) {
    ok
    errors
  }
}
`;

export const UPDATE_WHATSAPP_GROUP = `
mutation updateWhatsappGroup($id: ID!, $name: String!, $programId:
ID!, $wagId: String) {
  updateWaGroup(id: $id, programId: $programId, name: $name, wagId:
$wagId) {
    ok
    waGroup {
```

```
      id
    }
    errors
  }
}
;

export const MANAGE_WA_GROUP_MEMBER =
mutation manageWAGroupMember($operation: Boolean!,
$participants: [String]!, $wagId: String!) {
  manageMember(operation: $operation, participants: $participants,
wagId: $wagId) {
    errors
    ok
  }
}
;

/**
 * Send media message to WhatsApp group
 * Supported mediaType values (based on Whapi docs:
https://whapi.readme.io/reference/sendmessageimage):
 * - image: For sending images (JPEG, PNG, etc.)
 * - video: For sending videos
 * - short: For short video messages (PTV - Picture-to-Video)
 * - gif: For sending GIF animations
 * - audio: For sending audio files
 * - voice: For sending voice recordings
 * - document: For sending documents (PDF, DOC, etc.)
```

```
 */
export const SEND_WHATSAPP_MEDIA = `
mutation SendWhatsAppMedia(
  $caption: String
  $file: Upload!
  $mediaType: String!
  $to: String!
) {
  sendWhatsappMedia(
    caption: $caption
    file: $file
    mediaType: $mediaType
    to: $to
  ) {
    ok
    errors
  }
}
`;

/* ---------------------------------------------------
   PARTICIPANT DETAILS PAGE (AM/Admin)
--------------------------------------------------- */

export const ALL_PARTICIPANT_PROGRAM_ENROLLMENTS = `
query allParticipantProgramEnrollments($participantId: ID) {
  allParticipantProgramEnrollments(participant_Id: $participantId) {
    edges {
      node {
```

```
      id
      program {
        id
      }
    }
  }
}
}
;

export const UPDATE_PARTICIPANT_ACTIVITY_RESPONSE =
mutation UpdateParticipantActivityResponse($id: ID!, $lastPolledAt:
DateTime, $responseData: JSONString, $score: Float, $status: String,
$topicId: Int) {
  updateParticipantActivityResponse(
    id: $id
    lastPolledAt: $lastPolledAt
    responseData: $responseData
    score: $score
    status: $status
    topicId: $topicId
  ) {
    success
    message
    participantActivityResponse {
      id
    }
  }
}
```

```
;

/**
 * Fetch all participant activity responses - triggers backend to sync
data from external app
 */
export const ALL_PARTICIPANT_ACTIVITY_RESPONSES =
query allParticiantActivityResponses {
  allParticiantActivityResponses {
    edges {
      node {
        participant {
          username
        }
        activity {
          id
        }
        status
        fetch
        responseData
      }
    }
  }
}
;

/* ------------------------------------------------
   SUBSCRIPTIONS: REAL-TIME UPDATES
   These subscriptions are used for real-time data updates
```

```
  to eliminate the need for manual refresh/refetch
---------------------------------------------------- */

// Subscription for real-time participant activity response updates
export const PARTICIPANT_ACTIVITY_RESPONSE_SUBSCRIPTION =
subscription ParticipantActivityResponseSubscription {
  participantActivityResponseSubscription {
    mutation
    response {
      id
      topic {
        id
      }
      activity {
        id
      }
      program {
        id
      }
      session {
        id
      }
      activtyStatus
      responseData
      submissionDate
      score
      tokenLink
      status
    }
```

```
  }
}
;

// Program subscription for real-time program updates
export const PROGRAM_SUBSCRIPTION = 
subscription ProgramSubscription {
  programSubscription {
    mutation
    program {
      id
      name
      status
      startDate
      endDate
      maxParticipants
      contentUnlockMode
      unlockScope
      client {
        id
        name
      }
      preActivity {
        edges {
          node {
            id
            type
            description
            deadline
```

```
        mappSurvey
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
  }
  postActivity {
    edges {
      node {
        id
        type
        description
        deadline
        mappSurvey
        mappTool
        is360
        isFeedback
        name
        visible
        accessible
        totalResponses
      }
    }
  }
```

```
      attendees {
        edges {
          node {
            id
            status
            attendanceDate
            participant {
              id
              firstName
              lastName
              email
            }
          }
        }
      }
    }
  }
}
;

// Session subscription for real-time session updates
export const SESSION_SUBSCRIPTION = 
subscription SessionSubscription {
  sessionSubscription {
    mutation
    session {
      id
      name
      description
```

```
      deliveryMode
      startDate
      endDate
      location
      status
      preActivity {
        edges {
          node {
            id
            type
            description
            deadline
            mappSurvey
            mappTool
            is360
            isFeedback
            name
            visible
            accessible
            totalResponses
          }
        }
      }
      postActivity {
        edges {
          node {
            id
            type
            description
```

```
          deadline
          mappSurvey
          mappTool
          is360
          isFeedback
          name
          visible
          accessible
          totalResponses
        }
      }
    }
    attendees{
      edges{
        node{
          id
          status
          participant{
            id
            firstName
            lastName
            email
          }
        }
      }
    }
  }
}
```

```
;

// Topic subscription for real-time topic updates
export const TOPIC_SUBSCRIPTION = `
subscription TopicSubscription {
  topicSubscription {
    mutation
    topic {
      id
      name
      description
      startDate
      endDate
      preActivity {
        edges {
          node {
            id
            type
            description
            deadline
            mappSurvey
            mappTool
            is360
            isFeedback
            name
            visible
            accessible
            totalResponses
          }
```

```
        }
      }
      postActivity {
        edges {
          node {
            id
            type
            description
            deadline
            mappSurvey
            mappTool
            is360
            isFeedback
            name
            visible
            accessible
            totalResponses
          }
        }
      }
    }
  }
}
;

// Activity subscription for real-time activity updates
export const ACTIVITY_SUBSCRIPTION = `
subscription ActivitySubscription {
  activitySubscription {
```

```
    mutation
    activity {
      id
      mappSurvey
      mappSurveyName
      mappTool
      type
      description
      deadline
      is360
      name
      isFeedback
    }
  }
}
;

// Client subscription for real-time client updates
export const CLIENT_SUBSCRIPTION = `
subscription ClientSubscription {
  clientSubscription {
    mutation
    client {
      id
      name
      contactEmail
      phoneNumber
      address
      admins {
```

```
      edges {
        node {
          id
          username
          email
          firstName
          lastName
        }
      }
    }
  }
}
;

// Participant Program Enrollment subscription
export const PARTICIPANT_PROGRAM_ENROLLMENT_SUBSCRIPTION =
subscription ParticipantProgramEnrollmentSubscription {
  participantProgramEnrollmentSubscription {
    mutation
    enrollment {
      id
      participant {
        id
        username
        email
        firstName
        lastName
      }
```

```
      program {
        id
        name
      }
      enrollmentDate
      attendancePercentage
    }
  }
}
;

// Participant Session Attendance subscription
export const PARTICIPANT_SESSION_ATTENDANCE_SUBSCRIPTION =
subscription ParticipantSessionAttendanceSubscription {
  participantSessionAttendanceSubscription {
    mutation
    attendance {
      id
      participant {
        id
        username
        email
        firstName
        lastName
      }
      session {
        id
        name
      }
```

```
      status
      attendanceDate
    }
  }
}
;

// Participant Program Attendance subscription
export const PARTICIPANT_PROGRAM_ATTENDANCE_SUBSCRIPTION =

subscription ParticipantProgramAttendanceSubscription {
  participantProgramAttendanceSubscription {
    mutation
    attendance {
      id
      participant {
        id
        username
        email
        firstName
        lastName
      }
      program {
        id
        name
      }
      status
      attendanceDate
    }
```

```
  }
}
;

// Participant Activity Status subscription
export const PARTICIPANT_ACTIVITY_STATUS_SUBSCRIPTION =
subscription ParticipantActivityStatusSubscription {
  participantActivityStatusSubscription {
    mutation
    activityStatus {
      id
      participant {
        id
        username
        email
        firstName
        lastName
      }
      activity {
        id
        name
        type
      }
      status
      completionDate
    }
  }
}
;
```

```
// Scorecard Configuration subscription
export const SCORECARD_CONFIGURATION_SUBSCRIPTION = `
subscription ScorecardConfigurationSubscription {
  scorecardConfigurationSubscription {
    mutation
    scorecardconfiguration {
      id
      program {
        id
        name
      }
      name
      isDraft
      createdAt
      updatedAt
    }
  }
}
`;

// Scorecard Activity subscription
export const SCORECARD_ACTIVITY_SUBSCRIPTION = `
subscription ScorecardActivitySubscription {
  scorecardActivitySubscription {
    mutation
    scorecardactivity {
      id
      level
```

```
      weightPercentage
      isAttendance
      formula
      activity {
        id
        name
      }
      activityStatus
      session {
        id
        name
      }
      program {
        id
        name
      }
      topic {
        id
        name
      }
    }
  }
}
;

// WhatsApp Group subscription
export const WA_GROUP_SUBSCRIPTION =
subscription WAGroupSubscription {
  wagroupSubscription {
```

```
    mutation
    wagroup {
      id
      name
      program {
        id
        name
      }
      wagroupId
      createdAt
      updatedAt
      createdBy {
        id
        username
        email
        firstName
        lastName
      }
    }
  }
}
;

// WhatsApp Event subscription (for messages, group actions, etc.)
export const WHAPI_EVENT_SUBSCRIPTION = `
subscription WhapiEventSubscription {
  whapieventSubscription {
    mutation
    whapievent {
```

```
      id
      eventType
      eventAction
      channelId
      itemType
      messageId
      chatId
      fromMe
      sender
      senderName
      body
      status
      code
      groupAction
      groupId
      participants
      extra
      timestamp
      createdAt
    }
  }
}
;

/* ---------------------------------------------------
   FEEDBACK: FORM SCHEMA & SUBMISSION
--------------------------------------------------- */

export const FEEDBACK_JSON =
```

```
query feedbackJson {
  feedbackData
}
;

export const CREATE_REVIEW_MUTATION =
mutation CreateReview($reviewData: ReviewInput!) {
  createReview(reviewData: $reviewData) {
    success
    message
    review {
      id
      feedbackResponse
      additionalFeedback
      program {
        id
        name
      }
      session {
        id
      }
      topic {
        id
      }
    }
  }
}
;
```

```
export const CHECK_ENROLLMENT =
query CheckEnrollment($sessionId: ID!, $programId: ID!, $topicId: ID!,
$participantId: ID!, $hasSession: Boolean!, $hasProgram: Boolean!,
$hasTopic: Boolean!) {
  sessionById(id: $sessionId) @include(if: $hasSession) {
    id
    program {
      id
      enrolledEmployees(participant_Id: $participantId) {
        totalCount
      }
    }
  }
  Program(id: $programId) @include(if: $hasProgram) {
    id
    enrolledEmployees(participant_Id: $participantId) {
      totalCount
    }
  }
  topicById(id: $topicId) @include(if: $hasTopic) {
    id
    topicSessions {
      edges {
        node {
          id
          program {
            enrolledEmployees(participant_Id: $participantId) {
              totalCount
            }
```

```
        }
      }
    }
  }
 }
}
;

/* ---------------------------------------------------
   CLIENT MANAGEMENT: QUERIES AND MUTATIONS
--------------------------------------------------- */

export const CLIENTS_QUERY = `
query clients {
  clients {
    edges {
      node {
        id
        name
        contactEmail
        phoneNumber
        address
        admins {
          edges {
            node {
              id
              firstName
              lastName
              email
```

```
          }
        }
      }
    }
  }
}
;

export const CREATE_CLIENT_MUTATION =
mutation CreateClient($company: String, $email: String!, $name: String!,
$phone: String, $adminIds: [Int]) {
  createClient(
    company: $company
    email: $email
    name: $name
    phone: $phone
    adminIds: $adminIds
  ) {
    success
    message
    client {
      id
    }
  }
}
;

export const UPDATE_CLIENT_MUTATION =
```

```
mutation UpdateClient($id: ID!, $company: String, $email: String,
$name: String, $phone: String, $adminIds: [Int]) {
  updateClient(
    id: $id
    company: $company
    email: $email
    name: $name
    phone: $phone
    adminIds: $adminIds
  ) {
    success
    message
    client {
      id
    }
  }
}
;

export const USERS_QUERY =
query users {
  users {
    edges {
      node {
        id
        firstName
        lastName
        email
        phoneNumber
```

```
      }
    }
  }
}
;

export const ALL_PROGRAMS_LIST_QUERY =
query allProgramsList($id: ID) {
  Programs(creator_Id: $id) {
    edges {
      node {
        id
        name
        client {
          id
          name
        }
      }
    }
  }
}
;
```

this is lib/queries.ts

```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical, FolderOpen, CheckCircle2 } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
```

```
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";
import UpdateAttachmentDialog from
"@/components/UpdateAttachmentDialog";
import { USER_ROLES } from "@/lib/config";

type Option = { id: number; name: string };

type ProgramOption = {
  id: number;
  name: string;
  client?: number;
};

type Assignment = {
  id: number;
  client: Option;
  program: ProgramOption;
  created_at?: string;
};

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
```

```
  programs: ProgramOption[];
  assignments: Assignment[];
  created_at?: string;
  updated_at?: string | null;
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
  role,
  noProgramsForClient,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
  role?: string | null;
  noProgramsForClient?: boolean | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
```

```
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);
  const [fileToUpdate, setFileToUpdate] = useState<Attachment | null>
(null);
  const [updateFile, setUpdateFile] = useState<File | null>(null);
  const [updateUrl, setUpdateUrl] = useState("");
  const [updateLoading, setUpdateLoading] = useState(false);

  const { toasts, showToast, removeToast } = useToast();
  /** 1GB limit */
  const MAX_FILE_SIZE = 1024 * 1024 * 1024;

  /** Allowed MIME prefixes & types */
  const ALLOWED_TYPES = [
    "image/",
    "application/pdf",
    "application/msword",
    "application/vnd.ms-excel",
    "application/vnd.ms-powerpoint",
    "application/vnd.openxmlformats-officedocument",
    "text/plain",
    "video/",
    "audio/",
  ];

  function validateFile(file: File) {
    if (file.size > MAX_FILE_SIZE) {
      return "File size exceeds 1GB limit";
```

```
    }

    const isAllowed = ALLOWED_TYPES.some((type) =>
      file.type.startsWith(type)
    );

    if (!isAllowed) {
      return "Unsupported file type";
    }

    return null;
  }

  function openUpdateModal(file: Attachment) {
    setOpenMenu(null);
    setFileToUpdate(file);
    setUpdateFile(null);
    setUpdateUrl(file.external_url || "");
  }
  function notifyAttachmentsUpdated() {
    localStorage.setItem("attachments_updated_at",
Date.now().toString());
    window.dispatchEvent(new Event("attachments-updated"));
  }

  function normalizeAttachment(raw: any): Attachment {
    return {
      id: Number(raw?.id),
      file_name: raw?.file_name || "",
```

```
    description: raw?.description || "",
    asset_type: raw?.asset_type || "file",
    sharepoint_url: raw?.sharepoint_url || null,
    external_url: raw?.external_url || null,
    clients: Array.isArray(raw?.clients) ? raw.clients : [],
    programs: Array.isArray(raw?.programs) ? raw.programs : [],
    assignments: Array.isArray(raw?.assignments) ? raw.assignments : [],
    created_at: raw?.created_at || undefined,
    updated_at: raw?.updated_at || null,
  };
}

async function fetchFilesREST() {
  try {
    const res = await fetch(`${API}/api/attachments/`, {
      cache: "no-store",
    });

    const data = await res.json().catch(() => []);

    if (!res.ok) {
      throw new Error(data?.error || "Failed to fetch attachments");
    }

    const normalized = Array.isArray(data)
      ? data.map(normalizeAttachment)
      : [];

    // TEMP DEBUG - remove later if you want
```

```
      console.log("ATTACHMENTS API DATA:", normalized);

      setFiles(normalized);
    } catch (err: any) {
      console.error("Failed to fetch attachments:", err);
      showToast(err.message || "Failed to load files", "error");
      setFiles([]);
    }
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey]);

  // IMPORTANT: listen for global updates from other components
  useEffect(() => {
    function handleAttachmentsUpdated() {
      fetchFilesREST();
    }

    window.addEventListener("attachments-updated",
handleAttachmentsUpdated);
    return () => {
      window.removeEventListener("attachments-updated",
handleAttachmentsUpdated);
    };
  }, []);

  async function assignFileToCurrentSelection(file: Attachment) {
```

```
    if (!clientId || !programId) {
      showToast("Please select a Client and Program from the sidebar
first", "error");
      return;
    }

    try {
      const res = await
fetch(`${API}/api/attachments/${file.id}/assign/`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          client_id: Number(clientId),
          program_id: Number(programId),
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(
          data?.program_id?.[0] ||
          data?.non_field_errors?.[0] ||
          data?.error ||
          "Failed to assign file"
        );
      }

      const updatedAttachment = data?.attachment
```

```
      ? normalizeAttachment(data.attachment)
      : null;

    // Update local state immediately for instant UI response
    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === file.id ? updatedAttachment : f))
      );
    }

    setOpenMenu(null);

    showToast(
      data?.created === false
        ? "File already assigned to selected client/program"
        : "File assigned successfully",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Assignment failed", "error");
  }
}

async function unassignFile(file: Attachment) {
```

```
    if (!clientId || !programId) {
      showToast("Please select a Client and Program first", "error");
      return;
    }

    try {
      const res = await
fetch(`${API}/api/attachments/${file.id}/unassign/`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          client_id: Number(clientId),
          program_id: Number(programId),
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(
          data?.program_id?.[0] ||
          data?.non_field_errors?.[0] ||
          data?.error ||
          "Failed to unassign file"
        );
      }

      const updatedAttachment = data?.attachment
        ? normalizeAttachment(data.attachment)
```

```
      : null;

    // Update local state immediately for instant UI response
    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === file.id ? updatedAttachment : f))
      );
    }

    setOpenMenu(null);

    showToast(
      data?.deleted
        ? "File unassigned successfully"
        : "File was already unassigned",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Unassign failed", "error");
  }
}
async function confirmUpdateFile() {
  if (!fileToUpdate) return;
```

```
    if (!updateFile && !updateUrl.trim()) {
      showToast("Please select a new file or provide a new URL", "error");
      return;
    }

    // Prevent ambiguity
    if (updateFile && updateUrl.trim()) {
      showToast("Please provide either a new file or a new URL, not
both", "error");
      return;
    }

    if (updateFile) {
      const validationError = validateFile(updateFile);
      if (validationError) {
        showToast(validationError, "error");
        return;
      }
    }

    setUpdateLoading(true);

    try {
      const fd = new FormData();

      if (updateFile) {
        fd.append("file", updateFile);
      }
```

```
if (updateUrl.trim()) {
  fd.append("external_url", updateUrl.trim());
}

// Keep same existing values
fd.append("name", fileToUpdate.file_name);
fd.append("description", fileToUpdate.description || "");

const res = await fetch(
  `${API}/api/attachments/${fileToUpdate.id}/replace/`,
  {
    method: "POST",
    body: fd,
    credentials: "include",
  }
);

const data = await res.json().catch(() => ({}));

if (!res.ok) {
  throw new Error(data?.error || "Failed to update file");
}

const updatedAttachment = data?.attachment
  ? normalizeAttachment(data.attachment)
  : null;

// instant UI refresh
if (updatedAttachment) {
```

```
        setFiles((prev) =>
          prev.map((f) => (f.id === updatedAttachment.id ?
updatedAttachment : f))
        );
      }

      // close modal + reset state
      setFileToUpdate(null);
      setUpdateFile(null);
      setUpdateUrl("");

      showToast("File updated successfully", "success");

      notifyAttachmentsUpdated();

      // safest final refresh
      await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Update failed", "error");
    } finally {
      setUpdateLoading(false);
    }
  }

  function requestDeleteFile(fileId: number) {
    setOpenMenu(null);
    setFileIdToDelete(fileId);
  }
```

```
async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;

  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles((prev) => prev.filter((f) => f.id !== fileId));
    showToast("File deleted successfully", "success");
    notifyAttachmentsUpdated();

    // optional safety refetch
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}

function isDirectVideoUrl(url: string) {
```

```
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link: youtu.be/VIDEO_ID
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
    }

    // YouTube normal link: youtube.com/watch?v=VIDEO_ID
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      // already embed url
      if (parsed.pathname.includes("/embed/")) return url;
```

```
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? https://player.vimeo.com/video/${videoId} :
null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return https://www.loom.com/embed/${pathParts[shareIndex +
1]};
      }

      const embedIndex = pathParts.findIndex((part) => part ===
"embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
```

```
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
```

```
}

function formatDateTime(value?: string | null) {
  if (!value) return "Never";

  try {
    return new Date(value).toLocaleString("en-IN", {
      dateStyle: "medium",
      timeStyle: "short",
    });
  } catch {
    return value;
  }
}

const filteredFiles = files.filter((f) =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);

// Right side:
// Files assigned to CURRENT selected sidebar client + program
const assigned = filteredFiles.filter((f) => {
  if (!clientId || !programId) return false;

  return (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );
```

```
});

// Left side:
// Files NOT assigned to CURRENT selected sidebar client + program
const available = filteredFiles.filter((f) => {
  if (!clientId || !programId) {
    // show only files with no exact assignments
    return (f.assignments || []).length === 0;
  }

  const isAssignedToCurrentSelection = (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );

  return !isAssignedToCurrentSelection;
});

function FileCard({
  file,
  isAvailableSection,
}: {
  file: Attachment;
  isAvailableSection: boolean;
}) {
  const cardRef = useRef<HTMLDivElement>(null);

  const isTrulyUnassigned = (file.assignments || []).length === 0;
```

```
  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (
        openMenu === file.id &&
        cardRef.current &&
        !cardRef.current.contains(event.target as Node)
      ) {
        setOpenMenu(null);
      }
    }

    document.addEventListener("mousedown", handleClickOutside);
    return () => {
      document.removeEventListener("mousedown",
handleClickOutside);
    };
  }, [openMenu, file.id]);

  return (
    <div
      ref={cardRef}
      className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
    >
      <button
        className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
```

```
        onClick={() => setOpenMenu(openMenu === file.id ? null :
file.id)}
      >
        <MoreVertical size={18} />
      </button>

      <p className="font-semibold pr-8 break-words">{file.file_name}
</p>

      {file.description && (
        <DescriptionDisplay
          html={file.description}
          className="text-xs text-slate-500 dark:text-slate-400"
        />
      )}

      <p className="text-xs">
        Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
      </p>

      <p className="text-xs">
        Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
      </p>

      <p className="text-xs text-slate-500 dark:text-slate-400">
        Last Updated: {formatDateTime(file.updated_at || file.created_at)}
      </p>

      {file.asset_type === "file" && file.sharepoint_url ? (
```

```
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            View File
          </a>
        ) : file.asset_type === "url" && file.external_url ? (
          isPlayableVideoUrl(file.external_url) ? (
            <button
              onClick={() => openUrlVideo(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5
text-xs rounded-lg bg-emerald-600 text-white hover:bg-emerald-700
transition"
            >
              ▶️ Play Video
            </button>
          ) : (
            <a
              href={file.external_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
            >
              Open URL
            </a>
```

```
          )
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}

        {openMenu === file.id && (
          <div
            className="
              absolute right-3 top-10
              w-[180px]
              bg-white dark:bg-slate-900
              border border-slate-200 dark:border-slate-700
              p-3 rounded-lg shadow-lg
              space-y-2 z-50
            "
          >
            {isAvailableSection ? (
              <>
                <button
                  onClick={() => assignFileToCurrentSelection(file)}
                  className="w-full px-3 py-2 text-sm rounded bg-emerald-
600 text-white hover:bg-emerald-700"
                >
                  Assign
                </button>

                {/* Delete only if file is truly global unassigned */}
```

```
            {isTrulyUnassigned && (
              <button
                onClick={() => requestDeleteFile(file.id)}
                className="w-full px-3 py-2 text-sm rounded bg-red-600
text-white hover:bg-red-700"
              >
                Delete
              </button>
            )}
          </>
        ) : (
          <>
            <button
              onClick={() => openUpdateModal(file)}
              className="w-full px-3 py-2 text-sm rounded bg-emerald-
600 text-white hover:bg-emerald-700"
            >
              Update
            </button>

            <button
              onClick={() => unassignFile(file)}
              className="w-full px-3 py-2 text-sm rounded bg-amber-
500 text-white hover:bg-amber-600"
            >
              Unassign
            </button>
          </>
        )}
```

```
          </div>
        )}
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="mb-4 flex justify-center">
        <div className="relative w-full max-w-md">
          <svg
            className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
            />
          </svg>

          <input
            type="text"
            placeholder="Search files..."
            value={search}
```

```
          onChange={(e) => setSearch(e.target.value)}
          className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
        />
      </div>
    </div>

    <ConfirmDialog
      open={fileIdToDelete != null}
      title="Delete file?"
      description="This action cannot be undone."
      confirmLabel="Delete"
      cancelLabel="Cancel"
      variant="danger"
      onConfirm={confirmDeleteFile}
      onCancel={() => setFileIdToDelete(null)}
    />
    <UpdateAttachmentDialog
      open={fileToUpdate != null}
      attachmentName={fileToUpdate?.file_name || ""}
      description={fileToUpdate?.description || ""}
      currentUrl={fileToUpdate?.external_url || null}
      updateUrl={updateUrl}
      setUpdateUrl={setUpdateUrl}
      setUpdateFile={setUpdateFile}
      onClose={() => {
        setFileToUpdate(null);
```

```
          setUpdateFile(null);
          setUpdateUrl("");
        }}
        onConfirm={confirmUpdateFile}
        loading={updateLoading}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Available Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
            <FolderOpen className="h-4 w-4 text-slate-500 dark:text-
slate-400" aria-hidden />
            Available Files
          </h3>
```

```
          <div className="space-y-3 h-[500px] overflow-y-auto">
            {available.length > 0 ? (
              available.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  isAvailableSection={true}
                />
              ))
            ) : (
              <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
                No available files match your search
              </div>
            )}
          </div>
        </div>

        {/* Assigned Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
            <CheckCircle2 className="h-4 w-4 text-emerald-600 dark:text-
emerald-400" aria-hidden />
            Assigned Files
          </h3>

          <div className="space-y-3 h-[500px] overflow-y-auto">
```

```
          {assigned.length > 0 ? (
            assigned.map((file) => (
              <FileCard
                key={file.id}
                file={file}
                isAvailableSection={false}
              />
            ))
          ) : (
            <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
              {
                !clientId || !programId
                  ? (
                    noProgramsForClient
                      ? (
                        role === USER_ROLES.ACCOUNT_MANAGER
                          ? "No programs are available for this client yet. Create
a program in Program management to continue."
                          : "No programs have been created. Please contact the
administrator."
                      )
                      : "Please select a program from the sidebar first."
                  ) : "No assigned files match your search"
              }
            </div>
          )}
        </div>
      </div>
```

```
      </div>
    </div>
  );
}
```

this is components/uploadedfileslist.tsx
i also added programmanagement page.tsx
what i want to do is when i create/update activity in program,session,topic i want to give a dropdown in which all files are shown and i can add files from there in activity so once files are added we can see sdded files in created/updated activity no client program selection here we only need to add file to activity as per our backend changes tell me how are we going to do it if possible we can create seperate file in components and use it in programmanagement page.tsx

Yes — **100% we should do this with a separate reusable component** ✅
And **your backend is ready enough** for this approach.

You already have:

- `components/UploadedFilesList.tsx` → REST API working
- `lib/queries.ts` → central GraphQL queries/mutations
- `app/program-management/page.tsx` → main huge page

So the **cleanest + safest** solution is:

✅ What we are going to do

Goal

When creating/updating an **Activity** (inside Program / Session / Topic flow), show a selector with:

- **all uploaded files**
- no client/program filtering
- allow selecting **multiple files**
- send selected file IDs as:
  - `attachmentIds` in `createActivity`
  - `attachmentIds` in `updateActivity`

Then those files should be visible in the created/updated activity (because backend activity now has attachments).

✅ Best architecture (recommended)

We will do **3 frontend changes only**:

---

1) Create a new reusable component

`components/ActivityAttachmentSelector.tsx`

This will:

- fetch all attachments from REST API (`GET /api/attachments/`)
- show a **multi-select dropdown / checkbox list**

- return selected IDs to parent

👉 We will **reuse the same REST endpoint** from `UploadedFilesList.tsx`

---

2) Update `lib/queries.ts`

Add `attachmentIds` to:

- `CREATE_ACTIVITY_MUTATION`
- `UPDATE_ACTIVITY_MUTATION`

Also request `attachments { ... }` in response (optional but recommended)

---

3) Update `app/program-management/page.tsx`

In activity create/update modal/form:

- add state:

TypeScript

- render:

```
<ActivityAttachmentSelector
  selectedIds={selectedAttachmentIds}
  onChange={setSelectedAttachmentIds}
```

TypeScript

- pass `attachmentIds: selectedAttachmentIds` in:
  - create activity mutation
  - update activity mutation

✅ VERY IMPORTANT

We should NOT use GraphQL to fetch attachments right now

Because:

- your `UploadedFilesList.tsx` already successfully uses:

```
TypeScript
```

- this is already tested in your project
- safer than introducing another GraphQL schema dependency

👉 So for dropdown list, **use REST API**
👉 For activity create/update, **use GraphQL mutation**

This is the best hybrid approach for your current codebase.

🚀 EXACT FILES TO CHANGE

FILE 1) NEW FILE

`components/ActivityAttachmentSelector.tsx`

We will create this new file.

What it does:

- fetches all attachments from:

  TypeScript

- shows:
  - searchable list
  - checkbox multi-select
  - selected count
- returns selected IDs to parent

FILE 2) UPDATE

`lib/queries.ts`

Update `CREATE_ACTIVITY_MUTATION`

Current:

TypeScript

Add:

TypeScript

and inside mutation:

TypeScript

Update `UPDATE_ACTIVITY_MUTATION`

Add:

TypeScript

and:

TypeScript

FILE 3) UPDATE

`app/program-management/page.tsx`

This is where we need to update:

- import new component
- add selected attachment state
- reset selected IDs when modal opens/closes
- prefill selected IDs during edit
- pass `attachmentIds` in mutation variables
- render selector inside activity form modal

✅ EXACT PLAN (step-by-step)

STEP 1 — Create new component

`components/ActivityAttachmentSelector.tsx`

Copy-paste this file exactly:

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { FolderOpen, Search } from "lucide-react";

type Option = { id: number; name: string };

type ProgramOption = {
```

```
  id: number;
  name: string;
  client?: number;
};

type Assignment = {
  id: number;
  client: Option;
  program: ProgramOption;
  created_at?: string;
};

type Attachment = {
```

STEP 2 — Update `lib/queries.ts`

Replace your `CREATE_ACTIVITY_MUTATION` with this:

```
export const CREATE_ACTIVITY_MUTATION = `
mutation CreateActivity(
  $name: String!
  $type: String!
  $deadline: String
  $description: String
  $is360: Boolean
  $isFeedback: Boolean
  $mappSurvey: Int
  $mappSurveyName: String
  $mappTool: String
  $templateId: Int
```

```
  $attachmentIds: [Int]
) {
  createActivity(
    name: $name
    type: $type
    deadline: $deadline
    description: $description
    is360: $is360
    isFeedback: $isFeedback
```

Replace your `UPDATE_ACTIVITY_MUTATION` with this:

```
export const UPDATE_ACTIVITY_MUTATION = `
mutation UpdateActivity(
  $id: ID!
  $name: String
  $type: String
  $deadline: String
  $description: String
  $is360: Boolean
  $isFeedback: Boolean
  $mappSurvey: Int
  $mappSurveyName: String
  $mappTool: String
  $attachmentIds: [Int]
) {
  updateActivity(
    id: $id
    name: $name
    type: $type
    deadline: $deadline
```

```
    description: $description
    is360: $is360
```

STEP 3 — `program-management/page.tsx` (high-level changes)

Since you did **not paste** `page.tsx` **code yet**, I cannot safely tell you **exact line numbers** without risking another mess.

But here is the **correct integration plan**:

A) Add import

At top of `app/program-management/page.tsx`, add:

TypeScript

B) Add state near your activity form states

Somewhere near other activity form states:

TypeScript

C) Reset state when opening “Create Activity” modal

Wherever you open the create activity modal / form, make sure to reset:

TypeScript

D) Prefill state when opening “Edit Activity” modal

When you load activity data into edit form, do:

```
setSelectedAttachmentIds(
  Array.isArray(activity?.attachments)
    ? activity.attachments.map((a: any) => Number(a.id))
    : []
);
```

⚠️ This requires your activity query / program query to include attachments in activity nodes.

E) Render selector in Activity form JSX

In the create/update activity modal, **below Description / Deadline area** (best place), add:

```
<ActivityAttachmentSelector
  selectedIds={selectedAttachmentIds}
```

```
  onChange={setSelectedAttachmentIds}
  label="Attach Files to Activity"
/>
```

TypeScript

F) Pass `attachmentIds` in create mutation variables

Where you call `createActivity`, add:

TypeScript

So your variables object should include:

```
{
  name: ...,
  type: ...,
  deadline: ...,
  description: ...,
  is360: ...,
  isFeedback: ...,
  mappSurvey: ...,
  mappSurveyName: ...,
  mappTool: ...,
  templateId: ...,
  attachmentIds: selectedAttachmentIds,
}
```

G) Pass `attachmentIds` in update mutation variables

Where you call `updateActivity`, add:

TypeScript

⚠️ IMPORTANT for edit mode

Right now your `PROGRAM_BY_ID` query in `lib/queries.ts` **does not include attachments inside activities**.

If you want edit modal to show already selected files, we must add `attachments` in all these places:

- `Program.preActivity.node`
- `Program.postActivity.node`
- `Session.preActivity.node`
- `Session.postActivity.node`
- `Topic.preActivity.node`
- `Topic.postActivity.node`

STEP 4 — Update `PROGRAM_BY_ID` query (very important for edit mode)

In **every activity node** inside `PROGRAM_BY_ID`, add:

```
attachments {
  id
  fileName
`
```

GraphQL

Example:

From this:

```
node {
  id
  type
  description
  deadline
  mappSurvey
  mappSurveyName
  mappTool
  is360
  isFeedback
  name
  visible
  accessible
  totalResponses
}
```

To this:

```
node {
  id
  type
  description
  deadline
  mappSurvey
  mappSurveyName
  mappTool
  is360
  isFeedback
  name
  visible
  accessible
  totalResponses
  attachments {
    id
    fileName
  }
}
```

⚠️ Do this in **all activity blocks** inside `PROGRAM_BY_ID`.

This is required so edit modal can pre-select already attached files.

✅ Final recommended implementation flow

Create activity

- user opens create activity modal
- `selectedAttachmentIds = []`
- selector loads all files from REST
- user picks files

- createActivity mutation sends:

```
TypeScript
```

- backend saves M2M attachments
- refetch program data
- activity now shows attached files

Update activity

- user opens edit activity modal
- page already has activity attachments from `PROGRAM_BY_ID`
- prefill:

```typescript
selectedAttachmentIds = activity.attachments.map(...)
```

- user changes selection
- updateActivity mutation sends updated array
- backend replaces attachment set
- refetch program data
- activity now shows updated files

🔥 Best next step (important)

Now I need **ONLY ONE THING** from you to give exact copy-paste integration for `page.tsx`:

Paste these 3 parts from `app/program-management/page.tsx`:

1. **the function that creates activity**
   (likely `createActivityAndAttachToProgram`)
2. **the function that updates activity**
3. **the activity modal/form JSX block**
   (the popup form where name/type/deadline/description fields are rendered)

✅ Why I need `page.tsx` before exact code

Your `program-management/page.tsx` is likely huge and custom.

If I guess the wrong function name or state structure, you'll again get:

- wrong mutation variable path
- wrong activity ID extraction
- broken modal reset
- `"No activity id returned from createActivity"` again

I don't want to do that.

✅ Short answer

Yes — we can absolutely do it cleanly with a separate component:

- `components/ActivityAttachmentSelector.tsx` ✅

- update `lib/queries.ts` ✅
- small changes in `program-management/page.tsx` ✅

👉 NEXT MESSAGE FROM YOU

Please paste from `app/program-management/page.tsx` only these:

1. `createActivityAndAttachToProgram(...)`
2. activity update function
3. activity modal JSX block

Then I'll give you:

- **exact page.tsx changes**
- in your style:
  - **"replace this block"**
  - **"paste below this line"**
  - **"add this inside variables object"**

No guessing, no breaking.